

Trí tuệ Nhân tạo - Học Máy - Học sâu

AI - ML - DL

Artificial Intelligence & Machine Learning

Biên soạn tổng hợp

Ngày 13 tháng 7 năm 2026

Mục lục

I	Mạng Nơ-ron Nhân tạo Cơ bản	17
1	ANN1: Neural Networks Introduction – Giới thiệu mạng nơ-ron	18
	Mục tiêu bài học	18
1.1	Artificial Neural Networks Overview – Tổng quan về mạng nơ-ron nhân tạo	18
1.1.1	Mạng nơ-ron nhân tạo là gì?	18
1.1.2	ANN học từ dữ liệu như thế nào?	19
1.1.3	Tại sao ANN quan trọng?	20
1.2	ANN Applications – Các lĩnh vực ứng dụng của ANN	20
1.2.1	Xử lý ngôn ngữ tự nhiên	20
1.2.2	Nhận dạng hình ảnh, âm thanh và video	21
1.2.3	Điều khiển và tự động hóa	21
1.2.4	Dự đoán chuỗi thời gian	21
1.2.5	Y khoa, giáo dục và kinh doanh	22
1.3	Supervised Learning & Data Collection – Học có giám sát và quá trình thu thập dữ liệu	22
1.3.1	Khái niệm học có giám sát	22
1.3.2	Quy trình tổng quát	23
1.3.3	Ví dụ: điều khiển thiết bị bằng sóng não	23
1.3.4	Mô tả bài toán	23
1.3.5	Cách thu thập dữ liệu	23
1.3.6	Giai đoạn sử dụng	24
1.4	From Biological to Artificial Neurons – Từ nơ-ron sinh học đến nơ-ron nhân tạo	24
1.4.1	Nơ-ron sinh học	24
1.4.2	Nơ-ron nhân tạo	24
1.4.3	Ý nghĩa của trọng số	25
1.4.4	Ý nghĩa của bias	25
1.5	Activation Functions – Hàm kích hoạt	26
1.5.1	Vai trò của hàm kích hoạt	26
1.5.2	Hàm sigmoid	26
1.5.3	Tính chất	26
1.5.4	Hàm tanh	27
1.5.5	Tính chất	27
1.5.6	So sánh sigmoid và tanh	27
1.6	Parameters, Hyperparameters & Training – Tham số, siêu tham số và huấn luyện mạng	28
1.6.1	Tham số của mô hình	28

1.6.2	Siêu tham số	28
1.6.3	Huấn luyện mạng là gì?	28
1.6.4	Model sau khi huấn luyện	29
1.7	Neural Network Architecture – Cấu trúc mạng nơ-ron	29
1.7.1	Lớp input, lớp ẩn và lớp output	29
1.7.2	Kết nối đầy đủ	29
1.7.3	Mạng truyền thẳng nhiều lớp	30
1.7.4	Deep Learning	30
1.8	Example 1: Stock Price Prediction – Ví dụ 1: Dự đoán giá cổ phiếu	30
1.8.1	Mô tả bài toán	30
1.8.2	Dữ liệu huấn luyện	30
1.8.3	Mạng nơ-ron cho bài toán	31
1.9	Example 2: Self-Driving Car – Ví dụ 2: Xe tự lái	31
1.9.1	Mô tả bài toán	31
1.9.2	Input là ảnh	32
1.9.3	Thu thập dữ liệu	32
1.9.4	Mô hình học những gì được dạy	32
1.10	Example 3: Cat & Dog Classification – Ví dụ 3: Nhận dạng chó và mèo từ ảnh	33
1.10.1	Mô tả bài toán	33
1.10.2	Vì sao đây là bài toán phi tuyến?	33
1.11	Technical Glossary – Bảng thuật ngữ chuyên ngành	33
1.12	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	35
1.12.1	Các ý chính cần nhớ	35
1.12.2	Sơ đồ tư duy ngắn gọn	36
1.13	Review Questions – Câu hỏi ôn tập	36
1.14	Practice Exercises – Bài tập vận dụng	36
	Bài tập 1: Tính output của nơ-ron	36
	Bài tập 2: Xác định input và output	37
	Kết luận	37
2	ANN3: Forward Propagation – Lan truyền tiến	38
	Mục tiêu bài học	38
2.1	Supervised Learning Recap – Nhắc lại bài toán học có giám sát	38
2.1.1	Mạng nơ-ron cần học ánh xạ	38
2.1.2	Dữ liệu trong học có giám sát	39
2.2	Initial Network Parameters – Thông số ban đầu của mạng nơ-ron	40
2.2.1	Tham số của từng nơ-ron	40
2.2.2	Mạng có rất nhiều tham số	40
2.2.3	Khởi tạo ngẫu nhiên	41
2.2.4	Vì sao output ban đầu thường sai?	41
2.3	Forward Propagation – Lan truyền tiến	42
2.3.1	Ý tưởng của lan truyền tiến	42
2.3.2	Sơ đồ lan truyền tiến	42
2.3.3	Lan truyền tiến qua nhiều lớp	42
2.4	Error & Training Concept – Sai số và ý tưởng huấn luyện	43
2.4.1	So sánh output dự đoán với output mong muốn	43
2.4.2	Hàm mất mát	44

2.4.3	Huấn luyện là giảm sai số	44
2.5	Lan truyền ngược sai số	44
2.5.1	Backpropagation là gì?	44
2.5.2	Ý tưởng trực quan	45
2.5.3	Cập nhật từng bước	45
2.5.4	Liên hệ với điều khiển tự động	46
2.6	Quá trình huấn luyện lặp	46
2.6.1	Huấn luyện qua từng mẫu	46
2.6.2	Epoch	47
2.6.3	Vì sao cần lặp nhiều lần?	47
2.6.4	Thời gian huấn luyện	47
2.7	Bài toán phân loại nhiều lớp	48
2.7.1	Khái niệm class	48
2.7.2	Thiết kế output cho bài toán nhiều lớp	48
2.7.3	Cách 1: Một output nhận giá trị 1, 2, 3, 4	48
2.7.4	Cách 2: Hai output theo mã nhị phân	48
2.7.5	Cách 3: Bốn output theo one-hot vector	48
2.8	One-hot encoding	49
2.8.1	One-hot vector là gì?	49
2.8.2	Bảng mã hóa one-hot	49
2.8.3	Output dự đoán của mạng	49
2.9	Đo sai số giữa hai vector	50
2.9.1	Vì sao cần đo sai số giữa hai vector?	50
2.10	Hàm mất mát dựa trên chuẩn L2 - NORM 2	51
2.10.1	Chuẩn L2 của vector	51
2.10.2	Sai số L2 giữa hai vector	51
2.10.3	Ví dụ tính sai số L2	51
2.11	Đo khác biệt bằng góc giữa hai vector	52
2.11.1	Tích vô hướng	52
2.11.2	Góc giữa hai vector	52
2.11.3	Ý nghĩa	52
2.11.4	Cosine similarity	53
2.12	Cross entropy	53
2.12.1	Ý tưởng của cross entropy	53
2.12.2	Cross entropy với one-hot label	54
2.12.3	Ví dụ tính cross entropy	54
2.12.4	So sánh các cách đo sai số	55
2.13	Mô hình học được những gì?	55
2.13.1	Mô hình chỉ học trong phạm vi dữ liệu được dạy	55
2.13.2	Số lượng class trong các bộ dữ liệu lớn	55
2.13.3	Không phải cứ huấn luyện là thành công	56
2.14	Tổng hợp quy trình huấn luyện ANN	56
2.14.1	Quy trình tổng quát	56
2.14.2	Sơ đồ tổng quát	57
2.15	Technical Glossary – Bảng thuật ngữ chuyên ngành	57
2.16	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	58
2.16.1	Các ý chính cần nhớ	58
2.16.2	Công thức quan trọng	59

2.17	Review Questions – Câu hỏi ôn tập	60
2.18	Practice Exercises – Bài tập vận dụng	60
	Bài tập 1: One-hot encoding	60
	Bài tập 2: Tính chuẩn L2	61
	Bài tập 3: Tính cross entropy	61
	Bài tập 4: Xác định output layer	62
	Kết luận	62
3	ANN4: Gradient Descent – Hạ gradient	63
	Mục tiêu bài học	63
3.1	Neural Network Training Recap – Nhắc lại quá trình huấn luyện mạng nơ-ron	63
3.1.1	Từ dữ liệu đến sai số	63
3.1.2	Hàm mất mát	64
3.1.3	Mục tiêu lý tưởng	64
3.2	ANN Training as Optimization – Huấn luyện ANN là một bài toán tối ưu	65
3.2.1	Bản chất của quá trình huấn luyện	65
3.2.2	Số lượng biến rất lớn	66
3.2.3	Không phải lúc nào cũng có lời giải giải tích	66
3.3	Gradient Descent: Single Variable – Gradient Descent trong trường hợp một biến	67
3.3.1	Bài toán minh họa	67
3.3.2	Đạo hàm cho biết hướng đi	67
3.3.3	Ví dụ trực quan	68
3.4	Learning Rate – Learning rate	69
3.4.1	Learning rate là gì?	69
3.4.2	Learning rate quá nhỏ	69
3.4.3	Learning rate quá lớn	69
3.4.4	Learning rate thay đổi theo thời gian	70
3.5	Gradient Descent trong bài toán nhiều biến	70
3.5.1	Từ đạo hàm sang gradient	70
3.5.2	Công thức cập nhật tổng quát	71
3.5.3	Liên hệ với backpropagation	71
3.5.4	Ví dụ cập nhật một trọng số	72
3.6	Cực trị địa phương và cực trị toàn cục	72
3.6.1	Cực trị toàn cục	72
3.6.2	Cực trị địa phương	72
3.6.3	Vấn đề mắc kẹt ở cực trị địa phương	73
3.6.4	Ảnh hưởng trong huấn luyện ANN	73
3.7	Exploration và Exploitation	74
3.7.1	Exploitation là gì?	74
3.7.2	Exploration là gì?	74
3.7.3	Cân bằng exploration và exploitation	74
3.8	Thuật toán di truyền và metaheuristic	74
3.8.1	Metaheuristic là gì?	74
3.8.2	Thuật toán di truyền	75
3.8.3	Liên hệ với ANN	75
3.9	Momentum và ý tưởng vượt cực trị địa - local minimum	75

3.9.1	Hạn chế của Gradient Descent cơ bản	75
3.9.2	Ý tưởng momentum	76
3.9.3	Không đảm bảo tìm được cực trị toàn cục	76
3.10	Lựa chọn hàm mất mát	77
3.10.1	Các hàm mất mát phổ biến	77
3.10.2	Mean Squared Error	77
3.10.3	Cross entropy	77
3.10.4	Vì sao cross entropy thường tốt cho phân loại?	78
3.10.5	Không có hàm mất mát tốt nhất cho mọi bài toán	79
3.11	So sánh các khái niệm quan trọng	79
3.11.1	Backpropagation và Gradient Descent	79
3.11.2	Local minimum và global minimum	79
3.11.3	Exploration và exploitation	79
3.12	Technical Glossary – Bảng thuật ngữ chuyên ngành	80
3.13	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	81
3.13.1	Các ý chính cần nhớ	81
3.13.2	Công thức quan trọng	81
3.14	Review Questions – Câu hỏi ôn tập	82
3.15	Practice Exercises – Bài tập vận dụng	83
	Bài tập 1: Gradient Descent một biến	83
	Bài tập 2: Cập nhật trọng số	84
	Bài tập 3: So sánh learning rate	84
	Bài tập 4: Tính cross entropy	84
	Bài tập 5: Nhận diện loại bài toán	85
	Kết luận	85
4	ANN6: Backpropagation Example – Ví dụ lan truyền ngược	86
	Mục tiêu của tài liệu	86
4.1	Example Description – Mô tả ví dụ	86
4.1.1	Cấu trúc mạng trong ví dụ	86
4.1.2	Dữ liệu ban đầu	87
4.2	Forward Pass Example – Tính xuôi trong ví dụ	87
4.2.1	Tính output của hidden neuron h_1	87
4.2.2	Tính output của hidden neuron h_2	88
4.2.3	Tính output o_1	88
4.2.4	Tính output o_2	88
4.3	Error Calculation Example – Tính sai số của ví dụ	89
4.3.1	Sai số tại output o_1	89
4.3.2	Sai số tại output o_2	89
4.3.3	Sai số tổng	90
4.4	Updating Output Layer Weights – Cập nhật các trọng số lớp output	90
4.4.1	Cập nhật w_5	90
4.4.2	Cập nhật w_6, w_7, w_8	91
4.4.3	Bảng kết quả cập nhật lớp output	92
4.5	Cập nhật bias ở lớp output	93
4.5.1	Bias ứng với output o_1	93
4.5.2	Bias ứng với output o_2	93
4.6	Cập nhật các trọng số phía trước	94

4.6.1	Diễn quan trọng khi tính w_1	94
4.6.2	Sơ đồ đúng khi tính đạo hàm theo w_1	94
4.6.3	Tính $\frac{\partial E_{total}}{\partial out_{h_1}}$	94
4.6.4	Tính gradient của w_1	95
4.6.5	Cập nhật w_2	96
4.7	Cập nhật w_3 và w_4	97
4.7.1	Tính ảnh hưởng qua hidden neuron h_2	97
4.7.2	Cập nhật w_3	97
4.7.3	Cập nhật w_4	98
4.7.4	Bảng kết quả cập nhật lớp hidden	98
4.8	Cập nhật bias ở lớp hidden	99
4.8.1	Bias ứng với h_1	99
4.8.2	Bias ứng với h_2	99
4.9	Tổng kết một lượt cập nhật	100
4.9.1	Bảng trọng số trước và sau cập nhật	100
4.9.2	Bảng bias trước và sau cập nhật	100
4.9.3	Ý nghĩa của một lượt cập nhật	100
4.10	Những lỗi dễ mắc trong ví dụ	100
4.10.1	Lỗi 1: Chỉ tính một nhánh khi cập nhật w_1	100
4.10.2	Lỗi 2: Nhầm dấu khi cập nhật	101
4.10.3	Lỗi 3: Dùng nhầm output của neuron	101
4.11	Bài tập tự kiểm tra theo ví dụ	101
	Bài tập 1: Kiểm tra sai số tổng	101
	Bài tập 2: Tính lại w_5^{new}	102
	Bài tập 3: Vì sao w_7 tăng?	102
	Bài tập 4: Viết đúng biểu thức cho w_1	103
	Kết luận	103
5	ANN7: Training Issues – Vấn đề huấn luyện	104
	Mục tiêu bài học	104
5.1	From Theory to Practice – Từ lý thuyết huấn luyện đến vấn đề thực tế	104
5.1.1	Nhắc lại quá trình huấn luyện	104
5.1.2	Ba vấn đề chính trong bài ANN7	105
5.2	Vanishing Gradient – Vanishing Gradient	105
5.2.1	Vanishing gradient là gì?	105
5.2.2	Ví dụ số đơn giản	106
5.2.3	Vì sao gradient có thể biến mất?	106
5.3	Vanishing Gradient & Sigmoid – Vanishing Gradient và hàm sigmoid	107
5.3.1	Hàm sigmoid	107
5.3.2	Vấn đề của sigmoid trong mạng sâu	107
5.3.3	Sigmoid bị bão hòa	108
5.4	ReLU: Mitigating Vanishing Gradient – ReLU và ý tưởng giảm vanishing gradient	108
5.4.1	Hàm ReLU	108
5.4.2	Vì sao ReLU giúp giảm vanishing gradient?	109
5.4.3	Hạn chế của ReLU	109
5.5	Underfitting và Overfitting	110
5.5.1	Bài toán fitting là gì?	110

5.5.2	Ba trạng thái thường gặp	110
5.6	Ví dụ phân loại 2D	110
5.6.1	Mô tả ví dụ	110
5.6.2	Underfitting trong phân loại	111
5.6.3	fitting trong phân loại	111
5.6.4	Overfitting trong phân loại	111
5.6.5	Vì sao phân loại hoàn hảo chưa chắc là tốt?	112
5.7	Nhiều dữ liệu và sai số đo lường	112
5.7.1	Dữ liệu đo được không hoàn hảo	112
5.7.2	Ví dụ đo chiều dài	113
5.7.3	Liên hệ với overfitting	113
5.8	Ví dụ hồi quy	113
5.8.1	Bài toán hồi quy là gì?	113
5.8.2	Dữ liệu hồi quy có nhiều	114
5.8.3	Underfitting trong hồi quy	114
5.8.4	Good fitting trong hồi quy	114
5.8.5	Overfitting trong hồi quy	115
5.8.6	Ví dụ đa thức bậc cao	115
5.9	Phân loại và hồi quy	115
5.9.1	Bài toán phân loại	115
5.9.2	Bài toán hồi quy	116
5.9.3	So sánh classification và regression	116
5.10	Tổng hợp: Khi nào mô hình học tốt?	116
5.10.1	Mô hình học tốt cần cân bằng	116
5.10.2	Dấu hiệu nhận biết	117
5.10.3	Liên hệ với loss bằng 0	117
5.11	Technical Glossary – Bảng thuật ngữ chuyên ngành	117
5.12	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	118
5.12.1	Các ý chính cần nhớ	118
5.12.2	Công thức quan trọng	118
5.13	Review Questions – Câu hỏi ôn tập	119
5.14	Practice Exercises – Bài tập vận dụng	120
	Bài tập 1: Kiểm tra vanishing gradient	120
	Bài tập 2: Tính đạo hàm sigmoid	120
	Bài tập 3: Nhận diện underfitting và overfitting	120
	Bài tập 4: Phân biệt classification và regression	121
	Kết luận	121
6	ANN8: Overfitting & Underfitting – Quá khớp và dưới khớp	122
	Mục tiêu bài học	122
6.1	Underfitting & Overfitting Recap – Ôn lại Underfitting và Overfitting	122
6.1.1	Underfitting	122
6.1.2	Overfitting	123
6.1.3	So sánh ngắn gọn	123
6.2	Underfitting & Overfitting: Classification – Ví dụ Underfitting và Overfitting trong phân loại	124
6.2.1	Mô tả bài toán phân loại	124
6.2.2	Underfitting trong phân loại	124

6.2.3	Overfitting trong phân loại	124
6.3	Underfitting & Overfitting: Regression – Ví dụ Underfitting và Overfitting trong hồi quy	125
6.3.1	Mô tả bài toán hồi quy	125
6.3.2	Underfitting trong hồi quy	125
6.3.3	Good fitting trong hồi quy	125
6.3.4	Overfitting trong hồi quy	126
6.4	Causes of Underfitting – Nguyên nhân gây Underfitting	126
6.4.1	Mô hình quá đơn giản	126
6.4.2	Huấn luyện chưa đủ	127
6.4.3	Cách xử lý underfitting	127
6.5	Causes of Overfitting – Nguyên nhân gây Overfitting	127
6.5.1	Mô hình quá phức tạp	127
6.5.2	Dữ liệu quá ít	128
6.5.3	Dữ liệu thiếu đa dạng	128
6.6	Giải pháp cho Overfitting: Đơn giản hóa mô hình	128
6.6.1	Giảm độ phức tạp mô hình	128
6.6.2	Loại bỏ bớt đặc trưng	128
6.7	Giải pháp cho Overfitting: Tăng dữ liệu	129
6.7.1	Tại sao cần thêm dữ liệu?	129
6.7.2	Ví dụ ImageNet	129
6.7.3	Ví dụ nhận diện khuôn mặt	129
6.8	Data Augmentation	130
6.8.1	Data augmentation là gì?	130
6.8.2	Cắt ảnh và dịch cửa sổ	130
6.8.3	Lật đối xứng	131
6.8.4	Xoay ảnh	131
6.8.5	Thay đổi màu sắc và độ sáng	131
6.8.6	Augmentation phải phù hợp với bài toán	132
6.8.7	Tác dụng về số lượng dữ liệu	132
6.9	Train, Validation và Test Set	132
6.9.1	Vì sao phải chia dữ liệu?	132
6.9.2	Chia thành hai tập	133
6.9.3	Chia thành ba tập	133
6.9.4	Vai trò của training set	133
6.9.5	Vai trò của validation set	133
6.9.6	Vai trò của test set	134
6.9.7	Ví dụ chia dữ liệu	134
6.10	Nguyên tắc sử dụng Test Set	134
6.10.1	Không được dùng test set nhiều lần để chỉnh mô hình	134
6.10.2	Vì sao đây là lỗi nghiêm trọng?	135
6.10.3	Liên hệ với nghiên cứu khoa học	135
6.10.4	Nếu lỡ dùng test set thì làm sao?	135
6.11	Cross-validation	135
6.11.1	Ý tưởng	135
6.11.2	Ví dụ K-fold cross-validation	136
6.12	Early Stopping – Early Stopping	136
6.12.1	Early stopping là gì?	136

6.12.2	Minh họa	137
6.12.3	Ý nghĩa	137
6.12.4	Không phải cứ early stopping là mô hình tốt	137
6.13	Dropout – Dropout	137
6.13.1	Dropout là gì?	137
6.13.2	Một thời điểm trong dropout là gì?	138
6.13.3	Minh họa dropout	138
6.13.4	Vì sao dropout giúp giảm overfitting?	138
6.13.5	Góc nhìn ensemble	138
6.13.6	Góc nhìn giảm phụ thuộc	139
6.13.7	Dropout và robust model	139
6.13.8	Vì sao không dùng dropout khi test?	139
6.13.9	Vấn đề scale tín hiệu	139
6.13.10	Xác suất dropout	140
6.14	Tổng hợp các kỹ thuật xử lý Overfitting	140
6.14.1	Các kỹ thuật chính	140
6.14.2	Kỹ thuật nào quan trọng nhất?	140
6.15	Technical Glossary – Bảng thuật ngữ chuyên ngành	141
6.16	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	142
6.16.1	Các ý chính cần nhớ	142
6.16.2	Công thức và con số cần nhớ	142
6.17	Review Questions – Câu hỏi ôn tập	143
6.18	Practice Exercises – Bài tập vận dụng	144
	Bài tập 1: Nhận diện lỗi mô hình	144
	Bài tập 2: Chia dữ liệu	144
	Bài tập 3: Data augmentation	144
	Bài tập 4: Chọn augmentation phù hợp	145
	Bài tập 5: Dropout	145
	Kết luận	146
7	ANN9: Optimization Algorithms – Thuật toán tối ưu	147
	Mục tiêu bài học	147
7.1	Gradient Descent Recap – Nhắc lại Gradient Descent trong huấn luyện ANN	148
7.1.1	Mục tiêu của huấn luyện	148
7.1.2	Một vấn đề cần làm rõ	149
7.2	Stochastic Gradient Descent (SGD) – Stochastic Gradient Descent	149
7.2.1	Ý tưởng	149
7.2.2	Ví dụ trực quan	150
7.2.3	SGD và online learning	150
7.2.4	Đặc điểm: đường đi nhiễu	150
7.2.5	Ưu điểm của SGD	151
7.2.6	Nhược điểm của SGD	151
7.3	Batch Gradient Descent – Batch Gradient Descent	151
7.3.1	Ý tưởng	151
7.3.2	Ví dụ	152
7.3.3	Vì sao lấy trung bình gradient?	152
7.3.4	Đặc điểm: ổn định hơn SGD	153

7.3.5	Ưu điểm của Batch Gradient Descent	153
7.3.6	Nhược điểm của Batch Gradient Descent	153
7.4	Contour Lines & Local Minima – Đường đồng mức và cực trị địa phương	153
7.4.1	Đường đồng mức là gì?	153
7.4.2	Không gian hai biến và giá trị hàm	154
7.4.3	Cực trị địa phương	154
7.4.4	Vì sao SGD có thể tránh cực trị địa phương?	154
7.5	Mini-batch Gradient Descent	155
7.5.1	Ý tưởng	155
7.5.2	Batch size	155
7.5.3	Đường đi của mini-batch	156
7.5.4	Vì sao mini-batch được dùng phổ biến?	156
7.6	So sánh ba phương pháp Gradient Descent	157
7.6.1	Bảng so sánh	157
7.6.2	Ba thái cực và giải pháp trung gian	157
7.7	Các thuật ngữ quan trọng	157
7.7.1	Sample, data point và example	157
7.7.2	Feature	158
7.7.3	Batch	158
7.7.4	Batch size	158
7.7.5	Iteration	159
7.7.6	Epoch	159
7.8	Cách tính số iteration trong một epoch	159
7.8.1	Công thức tổng quát	159
7.8.2	Trường hợp SGD	159
7.8.3	Trường hợp Batch Gradient Descent	160
7.8.4	Trường hợp Mini-batch Gradient Descent	160
7.8.5	Nhiều epoch	160
7.9	Tại sao Batch GD là hướng dốc nhất còn SGD thì nhiều?	161
7.9.1	Hàm loss tổng thể	161
7.9.2	Batch Gradient Descent	161
7.9.3	Stochastic Gradient Descent	161
7.9.4	Mini-batch Gradient Descent	162
7.10	Bài tập được giao trong bài ANN9	162
7.10.1	Nội dung bài tập	162
7.10.2	Gợi ý hàm không lồi	163
7.10.3	Hàm Himmelblau	163
7.10.4	Hàm Rastrigin hai biến	163
7.10.5	Gradient tổng quát cho hàm hai biến	163
7.10.6	Ý nghĩa của việc tự lập trình	163
7.11	Pseudocode cho ba phương pháp	164
7.11.1	Pseudocode SGD	164
7.11.2	Pseudocode Batch Gradient Descent	164
7.11.3	Pseudocode Mini-batch Gradient Descent	164
7.12	Technical Glossary – Bảng thuật ngữ chuyên ngành	165
7.13	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	166
7.13.1	Các ý chính cần nhớ	166
7.13.2	Công thức quan trọng	166

7.14	Review Questions – Câu hỏi ôn tập	167
7.15	Practice Exercises – Bài tập vận dụng	168
	Bài tập 1: Tính số iteration	168
	Bài tập 2: Nhận diện phương pháp	169
	Bài tập 3: So sánh ưu nhược điểm	169
	Bài tập 4: Bài tập lập trình theo yêu cầu	169
	Kết luận	170
8	Softmax Function – Hàm Softmax	171
	Mục tiêu bài học	171
8.1	Softmax in Classification – Bối cảnh: Softmax trong bài toán phân loại	171
8.1.1	Bài toán phân loại ảnh	171
8.1.2	Ví dụ hệ thống phân loại ảnh lớn	172
8.2	Two Types of Classifiers – Hai kiểu bộ phân loại	172
8.2.1	Bộ phân loại thông thường	172
8.2.2	Bộ phân loại dựa trên xác suất	173
8.2.3	Ví dụ với 4 class	173
8.3	One-hot Encoding – One-hot Encoding	174
8.3.1	Vì sao cần mã hóa nhãn?	174
8.3.2	One-hot vector	174
8.3.3	Bảng one-hot encoding	175
8.4	Raw Network Output Issue – Vấn đề ở output thô của mạng	175
8.4.1	Vector logits	175
8.4.2	Ví dụ output thô	175
8.5	Softmax Function – Hàm Softmax	176
8.5.1	Công thức tổng quát	176
8.5.2	Vì sao softmax tạo ra xác suất?	177
8.6	Softmax Calculation Example – Ví dụ tính Softmax	177
8.6.1	Dữ liệu ví dụ	177
8.6.2	Bước 1: Tính các giá trị mũ	178
8.6.3	Bước 2: Tính mẫu số chung	178
8.6.4	Bước 3: Tính từng xác suất	178
8.6.5	Diễn giải kết quả	179
8.7	Softmax & Argmax – Softmax và Argmax	179
8.7.1	Hàm max	179
8.7.2	Hàm argmax	179
8.7.3	Argmax dưới dạng one-hot	180
8.7.4	Softmax xấp xỉ argmax theo nghĩa mềm	180
8.8	Why Softmax? – Vì sao gọi là Softmax?	180
8.8.1	Ý nghĩa tên gọi	180
8.8.2	Max cứng và softmax mềm	181
8.8.3	Lưu ý về tên gọi	181
8.9	Key Softmax Properties – Tính chất quan trọng của Softmax	181
8.9.1	Tính chất 1: Output luôn dương	181
8.9.2	Tính chất 2: Tổng output bằng 1	182
8.9.3	Tính chất 3: Giữ thứ tự tương đối	182
8.9.4	Tính chất 4: Khuếch đại sự khác biệt	182
8.10	Softmax at Output Layer – Softmax ở lớp output của mạng nơ-ron	183

8.10.1	Quy trình trong bài toán phân loại	183
8.10.2	Sơ đồ minh họa	183
8.10.3	Dự đoán class cuối cùng	183
8.11	Softmax & Cross Entropy – Softmax và Cross Entropy	184
8.11.1	Vì sao softmax thường đi cùng cross entropy?	184
8.11.2	Ý nghĩa	184
8.12	Softmax Computation Notes – Lưu ý khi tính Softmax	185
8.12.1	Vấn đề số quá lớn	185
8.12.2	Cách ổn định số học	185
8.12.3	Vì sao không thay đổi?	185
8.13	Technical Glossary – Bảng thuật ngữ chuyên ngành	186
8.14	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	186
8.14.1	Các ý chính cần nhớ	186
8.14.2	Công thức quan trọng	187
8.15	Review Questions – Câu hỏi ôn tập	187
8.16	Practice Exercises – Bài tập vận dụng	188
	Bài tập 1: Tính softmax	188
	Bài tập 2: Xác định class dự đoán	189
	Bài tập 3: One-hot vector	189
	Bài tập 4: Cross entropy với softmax	189
	Bài tập 5: Softmax ổn định số học	190
	Kết luận	190

II Mạng Nơ-ron Tích chập 192

1	CNN1: Convolutional Neural Networks – Mạng tích chập	193
	Mục tiêu bài học	193
1.1	Convolutional Neural Network – Mạng nơ-ron tích chập	194
1.1.1	CNN là gì?	194
1.1.2	CNN được thiết kế để giải quyết bài toán gì?	194
1.2	CNN Applications – Ứng dụng của CNN	195
1.2.1	Thị giác máy tính	195
1.2.2	Một số lĩnh vực khác	195
1.3	General CNN Architecture – Cấu trúc tổng quát của CNN	195
1.3.1	Các thành phần chính	195
1.3.2	Số lớp trong CNN	196
1.4	Image Representation – Biểu diễn ảnh trong máy tính	196
1.4.1	Ảnh xám	196
1.4.2	Ảnh màu	197
1.5	Problem with Fully Connected Networks – Vấn đề của mạng fully connected khi xử lý ảnh	197
1.5.1	Flatten ảnh thành vector	197
1.5.2	Số trọng số tăng rất nhanh	197
1.5.3	Liên hệ với overfitting	198
1.6	Key Ideas of CNN – Hai ý tưởng quan trọng của CNN	198
1.6.1	Local connectivity – Kết nối cục bộ	198
1.6.2	Weight sharing – Trọng số dùng chung	199

1.6.3	Tác dụng của hai ý tưởng này	199
1.7	Kernel and Filter – Kernel và bộ lọc	199
1.7.1	Kernel là gì?	199
1.7.2	Kernel học được từ đâu?	200
1.8	Convolution Forward Pass – Tính xuôi trong lớp tích chập	200
1.8.1	Ý tưởng trượt kernel	200
1.8.2	Công thức tại vị trí đầu tiên	200
1.8.3	Các vị trí còn lại	201
1.9	Activation Map – Bản đồ kích hoạt	202
1.9.1	Activation map là gì?	202
1.9.2	Một kernel tạo ra một activation map	202
1.9.3	Kích thước activation map	202
1.10	Simple X/O Classification Example – Ví dụ phân loại chữ X/O	209
1.10.1	Mô tả bài toán	209
1.10.2	Biểu diễn chữ X và chữ O	209
1.10.3	One-hot output	209
1.11	Convolution Example for Letter X – Ví dụ tích chập cho chữ X	210
1.11.1	Hai kernel minh họa	210
1.11.2	Input chữ X	210
1.11.3	Activation map với kernel thứ nhất	210
1.11.4	Activation map với kernel thứ hai	211
1.12	Flatten and Fully Connected Layer – Flatten và lớp kết nối đầy đủ	211
1.12.1	Flatten là gì?	211
1.12.2	Fully connected layer	212
1.13	Where Do CNN Parameters Come From? – Các tham số trong CNN đến từ đâu?	212
1.13.1	Kernel, bias và trọng số	212
1.13.2	Huấn luyện CNN	212
1.14	Why CNN Works Well for Images – Vì sao CNN phù hợp với ảnh?	213
1.14.1	Ảnh có tính cục bộ	213
1.14.2	Đặc trưng có thể xuất hiện ở nhiều vị trí	213
1.14.3	CNN giảm overfitting	213
1.15	ANN vs CNN – So sánh ANN thông thường và CNN	214
1.16	Technical Glossary – Bảng thuật ngữ chuyên ngành	214
1.17	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	215
1.17.1	Các ý chính cần nhớ	215
1.17.2	Công thức quan trọng	215
1.18	Review Questions – Câu hỏi ôn tập	216
1.19	Practice Exercises – Bài tập vận dụng	217
	Bài tập 1: Tính số input	217
	Bài tập 2: So sánh số trọng số	217
	Bài tập 3: Tính convolution	217
	Bài tập 4: Kích thước output	218
	Bài tập 5: Giải thích trực giác	218
	Kết luận	219

2	CNN2: Multichannel Convolution, Pooling, and Padding – Tích chập nhiều kênh, pooling và padding	220
	Mục tiêu bài học	220
2.1	Review of Simple Convolution – Ôn lại phép tích chập đơn giản	221
2.1.1	Trường hợp ảnh xám một kênh	221
2.1.2	Kích thước kernel	221
2.2	Color Image Convolution – Tích chập trên ảnh màu	221
2.2.1	Ảnh màu có nhiều channel	221
2.2.2	Kernel cũng phải có cùng số layer với input	222
2.3	Multi-channel Convolution Formula – Công thức tích chập nhiều kênh	223
2.3.1	Ý tưởng tính toán	223
2.3.2	Ví dụ kernel $3 \times 3 \times 3$	223
2.3.3	Một kernel vẫn tạo ra một activation map	224
2.4	Multiple Kernels – Nhiều kernel trong một lớp convolution	224
2.4.1	Một lớp convolution thường có nhiều kernel	224
2.4.2	Số kernel bằng số activation map đầu ra	224
2.4.3	Output của convolution layer là một khối dữ liệu	225
2.5	Pooling Layer – Lớp pooling	225
2.5.1	Pooling layer là gì?	225
2.5.2	Pooling window và stride	225
2.5.3	Ví dụ của sổ pooling 2×2	226
2.6	Max Pooling – Max pooling	226
2.6.1	Ý tưởng max pooling	226
2.6.2	Ví dụ max pooling	226
2.7	Average Pooling – Average pooling	227
2.7.1	Ý tưởng average pooling	227
2.7.2	Ví dụ average pooling	227
2.8	Pooling as Downsampling – Pooling như một dạng downsampling	228
2.8.1	Downsampling là gì?	228
2.8.2	Mục tiêu của pooling	228
2.8.3	Pooling có làm mất thông tin không?	228
2.9	Repeated Convolution and Pooling – Lặp lại convolution và pooling	229
2.9.1	CNN thường có nhiều lớp	229
2.9.2	Output của lớp trước là input của lớp sau	229
2.10	Convolution on Multiple Activation Maps – Tích chập trên nhiều activation map	229
2.10.1	Input của lớp convolution sau có nhiều layer	229
2.10.2	Kernel của lớp sau cũng phải có 4 layer	230
2.10.3	Một kernel nhiều layer vẫn tạo ra một activation map	230
2.11	Flatten and Fully Connected Layers – Flatten và các lớp fully connected	230
2.11.1	Khi nào cần flatten?	230
2.11.2	Ví dụ flatten	231
2.11.3	Fully connected layer ở cuối CNN	231
2.12	Sliding Order – Thứ tự trượt kernel	231
2.12.1	Trượt ngang trước hay dọc trước có khác không?	231
2.12.2	Sai lầm cần tránh	232
2.13	Output Size Reduction – Kích thước output giảm dần	232

2.13.1	Vì sao output có thể nhỏ dần?	232
2.13.2	Pooling cũng làm kích thước nhỏ lại	232
2.14	Zero Padding – Dệm số 0	232
2.14.1	Zero padding là gì?	232
2.14.2	Mục đích của padding	233
2.14.3	Công thức kích thước output	233
2.14.4	Lưu ý với kernel chẵn	233
2.15	Technical Glossary – Bảng thuật ngữ chuyên ngành	234
2.16	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	235
2.16.1	Các ý chính cần nhớ	235
2.16.2	Công thức quan trọng	235
2.17	Review Questions – Câu hỏi ôn tập	236
2.18	Practice Exercises – Bài tập vận dụng	237
	Bài tập 1: Kích thước kernel nhiều kênh	237
	Bài tập 2: Số activation map	237
	Bài tập 3: Max pooling	237
	Bài tập 4: Average pooling	238
	Bài tập 5: Kích thước output sau convolution	238
	Bài tập 6: Flatten	239
	Kết luận	239
3	CNN3: Convolution, Correlation và xử lý ảnh nhiều kênh trong CNN	240
	Mục tiêu bài học	240
3.1	Convolution and Correlation – Tích chập và tương quan	240
3.1.1	Các thuật ngữ cần phân biệt	240
3.1.2	Cross-correlation – Tương quan chéo	241
3.1.3	Auto-correlation – Tự tương quan	241
3.1.4	Convolution – Tích chập	242
3.2	Convolution vs Correlation – So sánh tích chập và tương quan	242
3.2.1	Correlation không lật hàm	242
3.2.2	Convolution lật hàm trước khi trượt	242
3.2.3	Trong CNN thực tế	242
3.3	How to Compute 1D Convolution – Cách tính tích chập một chiều	243
3.3.1	Công thức tích chập liên tục	243
3.3.2	Ý nghĩa trực giác	243
3.4	Rectangular Pulse Example – Ví dụ hai xung chữ nhật	243
3.4.1	Mô tả ví dụ	243
3.4.2	Khi hai hàm chưa giao nhau	244
3.4.3	Khi bắt đầu giao nhau	244
3.4.4	Khi phần giao nhau lớn nhất	244
3.4.5	Khi trượt ra khỏi nhau	244
3.5	1D and 2D Convolution – Tích chập một chiều và hai chiều	245
3.5.1	Ví dụ với $f(t)$ và $g(t)$ là 1D	245
3.5.2	Ảnh xám là dữ liệu 2D	245
3.5.3	Tích chập 2D trong ảnh	245
3.6	RGB Images in CNN – Ảnh RGB trong CNN	246
3.6.1	Ảnh RGB có ba kênh	246
3.6.2	Có phải tích chập 3D không?	247

3.6.3	Cách tính trên ảnh RGB	247
3.7	RGB Convolution Example – Ví dụ tính trên ảnh RGB	248
3.7.1	Vùng ảnh đang xét	248
3.7.2	Filter tương ứng	248
3.7.3	Tính trên kênh R	248
3.7.4	Tính trên kênh G	249
3.7.5	Tính trên kênh B	249
3.7.6	Cộng các kênh và bias	249
3.8	Activation Map from RGB Filter – Activation map từ filter RGB	249
3.8.1	Một filter tạo ra một activation map	249
3.8.2	Số trọng số của filter RGB	250
3.9	Technical Glossary – Bảng thuật ngữ chuyên ngành	250
3.10	Key Concepts Summary – Tóm tắt kiến thức trọng tâm	251
3.10.1	Các ý chính cần nhớ	251
3.10.2	Công thức quan trọng	251
3.11	Review Questions – Câu hỏi ôn tập	252
3.12	Practice Exercises – Bài tập vận dụng	252
	Bài tập 1: Phân biệt convolution và correlation	252
	Bài tập 2: Tính số trọng số	253
	Bài tập 3: Số activation map	253
	Bài tập 4: Tính trên ảnh RGB	253
	Kết luận	254

Phần I

Mạng Nơ-ron Nhân tạo Cơ bản

Chương 1

ANN1: Neural Networks Introduction – Giới thiệu mạng nơ-ron

Learning Objectives – Mục tiêu bài học

Sau khi học xong tài liệu này, người học cần nắm được:

1. Mạng nơ-ron nhân tạo là gì và vì sao nó được lấy cảm hứng từ nơ-ron sinh học.
2. Khái niệm học có giám sát, dữ liệu đầu vào, nhãn và đầu ra.
3. Vì sao mạng nơ-ron có khả năng xử lý các quan hệ phi tuyến.
4. Mô hình toán học cơ bản của một nơ-ron nhân tạo.
5. Ý nghĩa của trọng số, bias, hàm kích hoạt, tham số và siêu tham số.
6. Cấu trúc của mạng truyền thẳng nhiều lớp.
7. Cách hình dung ANN qua các ví dụ: nhận dạng ảnh, điều khiển bằng sóng não, dự đoán cổ phiếu và xe tự lái.

1.1 Artificial Neural Networks Overview – Tổng quan về mạng nơ-ron nhân tạo

1.1.1 Mạng nơ-ron nhân tạo là gì?

Mạng nơ-ron nhân tạo, tiếng Anh là *Artificial Neural Network* và thường viết tắt là ANN, là một hệ thống tính toán lấy cảm hứng từ mạng nơ-ron sinh học trong não bộ con người.

Một ANN bao gồm nhiều phần tử tính toán nhỏ gọi là **nơ-ron nhân tạo**. Mỗi nơ-ron có:

- nhiều ngõ vào, còn gọi là *input*;
- một ngõ ra, còn gọi là *output*;

- các trọng số biểu diễn mức độ ảnh hưởng của từng ngõ vào;
- một quy tắc tính toán để biến các ngõ vào thành ngõ ra.

Ghi nhớ

Mạng nơ-ron nhân tạo không được lập trình bằng các luật cứng theo kiểu “nếu–thì” cho mọi trường hợp. Thay vào đó, mạng học từ dữ liệu. Khi được cung cấp nhiều ví dụ, mạng tự điều chỉnh các tham số bên trong để tìm ra quan hệ giữa đầu vào và đầu ra.

1.1.2 ANN học từ dữ liệu như thế nào?

Trong bài học, giảng viên nhấn mạnh trường hợp **học có giám sát**. Đây là dạng học trong đó ta cung cấp cho mô hình các cặp dữ liệu:

(input, output)

hoặc viết theo ký hiệu thông dụng:

(x, y)

Trong đó:

- x là dữ liệu đầu vào, ví dụ một ảnh, một đoạn âm thanh, một chuỗi giá cổ phiếu;
- y là nhãn hoặc đầu ra đúng, ví dụ ảnh là “chó”, ảnh là “mèo”, giá cổ phiếu ngày tiếp theo, hoặc lệnh điều khiển xe.

Mục tiêu của quá trình học là tìm được một hàm xấp xỉ:

$$\hat{y} = f(x)$$

sao cho $\hat{y}$, tức đầu ra dự đoán của mô hình, càng gần với y , tức đầu ra đúng, càng tốt.

Ví dụ minh họa

Giả sử ta muốn dạy máy phân loại email thành “spam” và “không spam”.

Dữ liệu đầu vào	Nhãn đúng	Ý nghĩa
Email quảng cáo trúng thưởng	Spam	Email rác
Email từ giảng viên gửi bài tập	Không spam	Email hợp lệ
Email chứa nhiều liên kết lạ	Spam	Email đáng ngờ

Mô hình học từ nhiều ví dụ như vậy. Sau khi học xong, nếu gặp một email mới, mô hình sẽ dự đoán email đó là spam hay không spam.

1.1.3 Tại sao ANN quan trọng?

Một điểm rất quan trọng của ANN là khả năng xử lý các mối quan hệ **phi tuyến**. Trong thực tế, nhiều bài toán không thể được giải quyết tốt bằng một công thức tuyến tính đơn giản.

Ví dụ, khi đưa một ảnh xám vào máy tính và yêu cầu máy xác định đó là ảnh con chó hay con mèo, mỗi điểm ảnh chỉ là một con số biểu diễn độ sáng. Nếu ảnh có kích thước lớn, số lượng điểm ảnh có thể lên đến hàng chục nghìn hoặc hàng triệu.

Câu hỏi đặt ra là: có thể chỉ cộng tuyến tính các giá trị điểm ảnh để kết luận ảnh là chó hay mèo không?

Câu trả lời thường là không. Quan hệ giữa các điểm ảnh và ý nghĩa của bức ảnh là rất phức tạp. Mắt, tai, hình dáng, bối cảnh, cạnh, vùng sáng tối và rất nhiều đặc trưng khác cùng kết hợp lại theo cách phi tuyến. Đây chính là lý do ANN trở nên hữu ích.

Ghi nhớ

ANN đặc biệt mạnh trong những bài toán mà con người khó viết công thức tường minh, nhưng lại có thể cung cấp nhiều ví dụ dữ liệu để máy học.

1.2 ANN Applications – Các lĩnh vực ứng dụng của ANN

1.2.1 Xử lý ngôn ngữ tự nhiên

Trong xử lý ngôn ngữ tự nhiên, ANN có thể được dùng cho các bài toán như:

- phân loại văn bản;
- lọc email spam;
- tóm tắt tài liệu;
- dịch máy;
- sinh văn bản;
- kiểm tra chính tả;
- nhận dạng giọng nói.

Ví dụ minh họa

Một hệ thống tóm tắt văn bản có thể nhận đầu vào là một tài liệu dài 100 trang và sinh ra một bản tóm tắt vài đoạn. Để làm được điều đó, mô hình phải học cách nhận biết đâu là ý chính, đâu là thông tin phụ và cách diễn đạt lại nội dung một cách ngắn gọn.

1.2.2 Nhận dạng hình ảnh, âm thanh và video

ANN được dùng rất nhiều trong xử lý ảnh, âm thanh và video:

- nhận dạng ảnh;
- phân loại ảnh y khoa;
- đọc biển số xe;
- kiểm tra chữ ký;
- nhận dạng ký tự;
- xử lý ảnh siêu âm, ảnh X-quang, ảnh MRI.

Ví dụ minh họa

Trong bài toán đọc biển số xe, ảnh chụp biển số là input. Output có thể là chuỗi ký tự như “51F-123.45”. Hệ thống phải nhận dạng từng ký tự trong ảnh dù ánh sáng, góc chụp và chất lượng ảnh có thể khác nhau.

1.2.3 Điều khiển và tự động hóa

ANN cũng có thể được dùng để điều khiển hệ thống:

- xe tự lái;
- máy bay tự động;
- robot di chuyển theo hướng mong muốn;
- phát hiện lỗi trong hệ thống;
- dự đoán quỹ đạo vật thể;
- điều khiển thiết bị bằng tín hiệu não.

Ví dụ minh họa

Trong xe tự lái, input có thể là ảnh camera phía trước xe. Output có thể gồm góc lái, mức ga và mức phanh. Mạng học từ dữ liệu lái xe của con người để đưa ra hành động phù hợp trong tình huống mới.

1.2.4 Dự đoán chuỗi thời gian

Chuỗi thời gian là loại dữ liệu thay đổi theo thời gian. Ví dụ:

- thời tiết từng ngày;
- giá cổ phiếu;
- nhu cầu điện năng;

- chi phí sản phẩm;
- lượng khách hàng theo từng tháng.

Ví dụ minh họa

Để dự báo thời tiết ngày hôm nay, ta thường phải xét thời tiết các ngày trước đó, độ ẩm, áp suất, hướng gió, nhiệt độ và nhiều yếu tố khác. Đây là một bài toán chuỗi thời gian điển hình.

1.2.5 Y khoa, giáo dục và kinh doanh

Trong y khoa, ANN có thể hỗ trợ phát hiện ung thư, phân tích ảnh y khoa, nhận biết bệnh từ dữ liệu xét nghiệm hoặc hình ảnh.

Trong giáo dục, ANN có thể hỗ trợ xây dựng hệ thống học thích nghi. Người học nhanh có thể nhận bài khó hơn, trong khi người học chậm có thể được ôn lại phần nền tảng.

Trong kinh doanh và ngân hàng, ANN có thể dùng để:

- phân tích hành vi khách hàng;
- phân nhóm khách hàng;
- dự đoán rủi ro tín dụng;
- nghiên cứu thị trường;
- phát hiện giao dịch bất thường.

1.3 Supervised Learning & Data Collection – Học có giám sát và quá trình thu thập dữ liệu

1.3.1 Khái niệm học có giám sát

Học có giám sát, tiếng Anh là *supervised learning*, là phương pháp học máy trong đó mỗi mẫu dữ liệu huấn luyện đều có nhãn đúng đi kèm.

Một tập dữ liệu huấn luyện có dạng:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

Trong đó:

- N là số lượng mẫu dữ liệu;
- x_i là input thứ i ;
- y_i là output hoặc nhãn đúng tương ứng.

Mô hình sẽ học một hàm:

$$f : X \rightarrow Y$$

để biến input thành output.

1.3.2 Quy trình tổng quát

Một quy trình học có giám sát thường gồm các bước sau:

1. Xác định bài toán cần giải.
2. Xác định input và output.
3. Thu thập nhiều cặp dữ liệu input–output.
4. Huấn luyện mô hình trên dữ liệu đã thu thập.
5. Kiểm tra mô hình trên dữ liệu mới.
6. Triển khai mô hình để sử dụng thực tế.

Ghi nhớ

Trong học có giám sát, chất lượng dữ liệu đóng vai trò cực kỳ quan trọng. Nếu dữ liệu sai, thiếu, nhiễu hoặc gán nhãn không nhất quán, mô hình học được cũng có thể sai.

1.3.3 Ví dụ: điều khiển thiết bị bằng sóng não

Trong transcript, giảng viên đưa ra ví dụ về việc sử dụng thiết bị ghi sóng não để điều khiển đèn, máy lạnh hoặc các thiết bị khác.

1.3.4 Mô tả bài toán

Người dùng đội một thiết bị có gắn các điện cực để thu tín hiệu điện não. Khi người dùng nghĩ đến một lệnh, ví dụ “bật đèn”, thiết bị sẽ ghi lại dạng sóng tương ứng.

Ta cần xây dựng một hệ thống học có giám sát để ánh xạ:

Dạng sóng não $\longrightarrow$ Lệnh điều khiển

1.3.5 Cách thu thập dữ liệu

Ta thu thập nhiều cặp dữ liệu:

$$(x_i, y_i)$$

Trong đó:

- x_i là tín hiệu sóng não tại một thời điểm;
- y_i là lệnh tương ứng, ví dụ “bật đèn”, “tắt đèn”, “bật máy lạnh”, “tắt máy lạnh”.

Lần đo	Input: tín hiệu sóng não	Output: lệnh đúng
1	Dạng sóng khi nghĩ “bật đèn”	Bật đèn
2	Dạng sóng khi nghĩ “tắt đèn”	Tắt đèn
3	Dạng sóng khi nghĩ “bật máy lạnh”	Bật máy lạnh
4	Dạng sóng khi nghĩ “tắt máy lạnh”	Tắt máy lạnh

1.3.6 Giai đoạn sử dụng

Sau khi huấn luyện, người dùng đặt thiết bị, suy nghĩ một lệnh. Hệ thống nhận dạng dạng sóng, dự đoán lệnh và gửi tín hiệu đến cơ cấu chấp hành.

Sóng não mới $\rightarrow$ Mô hình ANN $\rightarrow$ Lệnh dự đoán $\rightarrow$ Thiết bị

1.4 From Biological to Artificial Neurons – Từ nơ-ron sinh học đến nơ-ron nhân tạo

1.4.1 Nơ-ron sinh học

Một nơ-ron sinh học trong não người có nhiều kết nối đầu vào và một đầu ra chính. Các tín hiệu đi vào nơ-ron, được xử lý, sau đó truyền tiếp đến các nơ-ron khác.

Điểm quan trọng không chỉ là số lượng nơ-ron, mà là cách các nơ-ron kết nối với nhau. Khi ta học tập, luyện tập và sử dụng một kỹ năng thường xuyên, các kết nối liên quan được củng cố. Ngược lại, những kết nối ít được sử dụng có thể yếu dần.

Ghi nhớ

Việc học không chỉ là ghi nhớ thông tin trừu tượng. Về bản chất, học tập còn gắn với sự thay đổi và củng cố các kết nối trong não bộ.

1.4.2 Nơ-ron nhân tạo

Nơ-ron nhân tạo là mô hình tính toán mô phỏng đơn giản hóa nơ-ron sinh học. Nó nhận nhiều input và sinh ra một output.

Giả sử nơ-ron có n input:

$$x_1, x_2, \dots, x_n$$

Mỗi input có một trọng số tương ứng:

$$w_1, w_2, \dots, w_n$$

Ngoài ra, nơ-ron còn có một hằng số dịch gọi là bias, ký hiệu là b .

Đầu tiên, nơ-ron tính tổng có trọng số:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Viết gọn:

$$z = \sum_{i=1}^n w_i x_i + b$$

Sau đó, giá trị z được đưa qua một hàm kích hoạt φ :

$$y = \varphi(z)$$

Do đó, công thức tổng quát của một nơ-ron nhân tạo là:

$$y = \varphi \left(\sum_{i=1}^n w_i x_i + b \right)$$

Ví dụ minh họa

Giả sử một nơ-ron có hai input:

$$x_1 = 2, \quad x_2 = 3$$

Trọng số và bias là:

$$w_1 = 0.5, \quad w_2 = -1, \quad b = 1$$

Ta tính:

$$z = w_1 x_1 + w_2 x_2 + b$$

$$z = 0.5 \cdot 2 + (-1) \cdot 3 + 1 = 1 - 3 + 1 = -1$$

Nếu dùng hàm kích hoạt sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

thì output là:

$$y = \sigma(-1) = \frac{1}{1 + e^1} \approx 0.269$$

1.4.3 Ý nghĩa của trọng số

Trọng số w_i cho biết input x_i ảnh hưởng mạnh hay yếu đến output.

- Nếu $|w_i|$ lớn, input x_i có ảnh hưởng mạnh.
- Nếu $|w_i|$ nhỏ, input x_i có ảnh hưởng yếu.
- Nếu $w_i > 0$, input làm tăng giá trị tổng z .
- Nếu $w_i < 0$, input làm giảm giá trị tổng z .

1.4.4 Ý nghĩa của bias

Bias b giúp mô hình linh hoạt hơn. Nếu không có bias, hàm tuyến tính bên trong nơ-ron luôn bị ràng buộc đi qua gốc tọa độ trong một số trường hợp đơn giản. Bias cho phép dịch chuyển ngưỡng kích hoạt của nơ-ron.

Ví dụ minh họa

Xét nơ-ron một input:

$$z = wx + b$$

Nếu $w = 1$ và $b = 0$, ta có:

$$z = x$$

Nếu $w = 1$ và $b = -5$, ta có:

$$z = x - 5$$

Khi đó, nơ-ron chỉ có xu hướng kích hoạt mạnh khi x đủ lớn hơn 5. Bias ở đây đóng vai trò như một ngưỡng.

1.5 Activation Functions – Hàm kích hoạt

1.5.1 Vai trò của hàm kích hoạt

Hàm kích hoạt, tiếng Anh là *activation function*, là hàm được đặt sau tổng có trọng số trong nơ-ron.

Nếu không có hàm kích hoạt phi tuyến, nhiều lớp tuyến tính ghép lại vẫn chỉ tương đương một phép biến đổi tuyến tính. Khi đó, mạng khó giải quyết các bài toán phức tạp như nhận dạng ảnh, phân loại văn bản hay điều khiển xe tự lái.

Ghi nhớ

Hàm kích hoạt là thành phần giúp mạng nơ-ron học được các quan hệ phi tuyến.

1.5.2 Hàm sigmoid

Hàm sigmoid được định nghĩa bởi:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Giá trị của sigmoid luôn nằm trong khoảng:

$$0 < \sigma(x) < 1$$

1.5.3 Tính chất

- Khi x rất lớn, $\sigma(x)$ gần bằng 1.
- Khi x rất nhỏ, $\sigma(x)$ gần bằng 0.
- Khi $x = 0$, $\sigma(0) = 0.5$.

Ví dụ minh họa

Tính một vài giá trị của sigmoid:

$$\sigma(0) = \frac{1}{1 + e^0} = 0.5$$

$$\sigma(2) = \frac{1}{1 + e^{-2}} \approx 0.881$$

$$\sigma(-2) = \frac{1}{1 + e^2} \approx 0.119$$

Nếu output cần biểu diễn xác suất, ví dụ xác suất email là spam, sigmoid là một lựa chọn dễ hiểu vì nó cho giá trị từ 0 đến 1.

1.5.4 Hàm tanh

Hàm tanh được định nghĩa bởi:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Giá trị của tanh nằm trong khoảng:

$$-1 < \tanh(x) < 1$$

1.5.5 Tính chất

- Khi x rất lớn, $\tanh(x)$ gần bằng 1.
- Khi x rất nhỏ, $\tanh(x)$ gần bằng -1.
- Khi $x = 0$, $\tanh(0) = 0$.

1.5.6 So sánh sigmoid và tanh

Tiêu chí	Sigmoid	Tanh
Công thức	$\frac{1}{1 + e^{-x}}$	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$
Miền giá trị	$(0, 1)$	$(-1, 1)$
Giá trị tại $x = 0$	0.5	0
Ứng dụng trực quan	Xác suất nhị phân	Tín hiệu có âm và dương

Cảnh báo

Từ “logistic” trong “logistic sigmoid” không liên quan trực tiếp đến ngành logistics và chuỗi cung ứng. Trong ANN, logistic thường chỉ dạng hàm toán học sigmoid.

1.6 Parameters, Hyperparameters & Training – Tham số, siêu tham số và huấn luyện mạng

1.6.1 Tham số của mô hình

Tham số, tiếng Anh là *parameter*, là những đại lượng được mô hình học từ dữ liệu. Trong một nơ-ron nhân tạo, các tham số chính là:

$$w_1, w_2, \dots, w_n, b$$

Tức là:

- các trọng số;
- bias.

Khi mạng có nhiều nơ-ron, ta phải tìm tất cả trọng số và bias của tất cả các nơ-ron.

1.6.2 Siêu tham số

Siêu tham số, tiếng Anh là *hyperparameter*, là những lựa chọn được xác định trước khi huấn luyện, không được học trực tiếp từ dữ liệu theo cách thông thường.

Ví dụ:

- số lớp ẩn;
- số nơ-ron trong mỗi lớp;
- loại hàm kích hoạt;
- tốc độ học;
- số vòng huấn luyện;
- kích thước batch.

Ví dụ minh họa

Nếu ta chọn mạng có 3 lớp ẩn, mỗi lớp 64 nơ-ron và dùng hàm kích hoạt tanh, thì những lựa chọn này là siêu tham số. Trong quá trình huấn luyện, mô hình sẽ học trọng số và bias, nhưng không tự động đổi số lớp từ 3 thành 5 nếu ta không thiết kế cơ chế riêng.

1.6.3 Huấn luyện mạng là gì?

Huấn luyện mạng là quá trình điều chỉnh các tham số sao cho mô hình dự đoán đúng hơn trên dữ liệu huấn luyện.

Giả sử mô hình dự đoán:

$$\hat{y}_i = f(x_i)$$

Nhân đúng là y_i . Ta cần một hàm mất mát, tiếng Anh là *loss function*, để đo sai số giữa $\hat{y}_i$ và y_i .

Với bài toán dự đoán số, một hàm mất mát phổ biến là sai số bình phương trung bình:

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Mục tiêu huấn luyện là tìm các tham số làm cho L nhỏ nhất có thể.

1.6.4 Model sau khi huấn luyện

Sau khi huấn luyện thành công, ta thu được một **model**. Model này đã chứa các tham số học được và có thể được dùng để dự đoán trên dữ liệu mới.

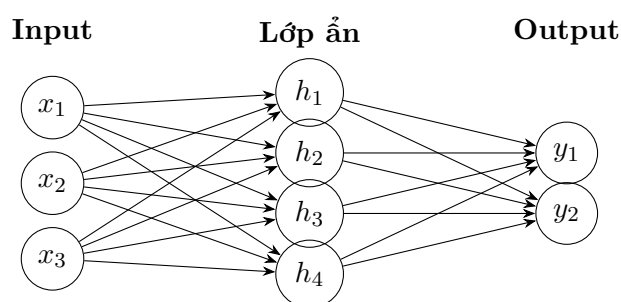
Dữ liệu mới $\rightarrow$ Model đã huấn luyện $\rightarrow$ Dự đoán

1.7 Neural Network Architecture – Cấu trúc mạng nơ-ron

1.7.1 Lớp input, lớp ẩn và lớp output

Một mạng nơ-ron thường gồm ba loại lớp:

- **Lớp input**: nhận dữ liệu đầu vào.
- **Lớp ẩn**: xử lý và biến đổi thông tin.
- **Lớp output**: sinh ra kết quả cuối cùng.



1.7.2 Kết nối đầy đủ

Trong mạng kết nối đầy đủ, tiếng Anh là *fully connected network*, mỗi nơ-ron của một lớp được nối với tất cả nơ-ron ở lớp kế tiếp.

Tuy nhiên, các nơ-ron trong cùng một lớp thường không nối trực tiếp với nhau trong mạng truyền thẳng cơ bản.

1.7.3 Mạng truyền thẳng nhiều lớp

Mạng truyền thẳng, tiếng Anh là *feedforward neural network*, là mạng trong đó thông tin đi theo một chiều:

Input $\rightarrow$ Các lớp ẩn $\rightarrow$ Output

Nếu mạng có nhiều lớp ẩn, ta gọi là mạng truyền thẳng nhiều lớp.

Ghi nhớ

Trong bài học, cấu trúc được khảo sát chủ yếu là mạng truyền thẳng nhiều lớp. Đây là một trong những cấu trúc ANN cơ bản và phổ biến nhất.

1.7.4 Deep Learning

Deep Learning thường được dịch là học sâu, nhưng cần hiểu đúng bản chất: “sâu” ở đây liên quan đến việc mạng có nhiều lớp xử lý liên tiếp.

Một mạng càng có nhiều lớp ẩn thì càng có khả năng học các biểu diễn phức tạp hơn. Tuy nhiên, nhiều lớp hơn không phải lúc nào cũng tốt hơn. Mạng sâu hơn thường cần:

- nhiều dữ liệu hơn;
- phần cứng mạnh hơn;
- kỹ thuật huấn luyện tốt hơn;
- kiểm soát overfitting tốt hơn.

1.8 Example 1: Stock Price Prediction – Ví dụ 1: Dự đoán giá cổ phiếu

1.8.1 Mô tả bài toán

Giả sử ta muốn dự đoán giá cổ phiếu ngày hôm nay dựa vào giá của 5 ngày gần nhất. Input là:

$$x = [p_{t-5}, p_{t-4}, p_{t-3}, p_{t-2}, p_{t-1}]$$

Output là:

$$y = p_t$$

Trong đó p_t là giá cổ phiếu tại ngày cần dự đoán.

1.8.2 Dữ liệu huấn luyện

Ta cần thu thập nhiều cặp dữ liệu:

$$([p_{t-5}, p_{t-4}, p_{t-3}, p_{t-2}, p_{t-1}], p_t)$$

Ngày $t - 5$	Ngày $t - 4$	Ngày $t - 3$	Ngày $t - 2$	Ngày $t - 1$	Ngày t
100	102	101	103	104	105
102	101	103	104	105	107
101	103	104	105	107	106

Trong bảng trên, 5 cột đầu là input, cột cuối là output đúng.

1.8.3 Mạng nơ-ron cho bài toán

Vì input có 5 giá trị, lớp input có thể có 5 nơ-ron. Vì output là một giá cổ phiếu, lớp output có thể có 1 nơ-ron.

$$\mathbb{R}^5 \rightarrow \mathbb{R}$$

Ví dụ minh họa

Nếu 5 giá gần nhất là:

$$[101, 103, 104, 105, 107]$$

mô hình có thể dự đoán:

$$\hat{y} = 106.5$$

Nếu giá thật là:

$$y = 106$$

thì sai số là:

$$\hat{y} - y = 106.5 - 106 = 0.5$$

Sai số bình phương là:

$$(0.5)^2 = 0.25$$

1.9 Example 2: Self-Driving Car – Ví dụ 2: Xe tự lái

1.9.1 Mô tả bài toán

Trong ví dụ xe tự lái, ta cần xây dựng một mô hình nhận dữ liệu từ camera và sinh ra các lệnh điều khiển xe.

Input có thể là ảnh camera phía trước xe. Output có thể gồm:

- góc lái;
- mức ga;
- mức phanh.

Ta có thể viết:

$$\text{Ảnh camera} \rightarrow \text{ANN} \rightarrow \begin{bmatrix} \text{góc lái} \\ \text{ga} \\ \text{phanh} \end{bmatrix}$$

1.9.2 Input là ảnh

Nếu dùng ảnh xám, mỗi pixel có giá trị từ 0 đến 255:

$$0 \leq x_i \leq 255$$

Nếu ảnh có kích thước 100×100 , số pixel là:

$$100 \times 100 = 10\,000$$

Khi đó, input có thể là vector gồm 10.000 giá trị:

$$x = [x_1, x_2, \dots, x_{10000}]$$

1.9.3 Thu thập dữ liệu

Trong quá trình lái xe thật, ta ghi lại đồng thời:

- ảnh camera tại từng thời điểm;
- góc lái tương ứng;
- mức ga tương ứng;
- mức phanh tương ứng.

Mỗi mẫu dữ liệu có dạng:

(ảnh, góc lái, ga, phanh)

1.9.4 Mô hình học những gì được dạy

Một điểm quan trọng trong bài học là: mô hình sẽ học từ dữ liệu mà ta cung cấp. Nếu dữ liệu được thu bởi một tài xế cẩn thận, mô hình có xu hướng học cách lái cẩn thận. Nếu dữ liệu được thu bởi một tài xế lái ẩu, mô hình có thể học hành vi lái ẩu.

Cảnh báo

Mô hình không tự biết thế nào là “lái xe tốt” nếu dữ liệu huấn luyện không phản ánh điều đó. Trong học có giám sát, chất lượng nhãn và chất lượng người tạo dữ liệu ảnh hưởng trực tiếp đến hành vi của mô hình.

Ví dụ minh họa

Hai nhóm dữ liệu khác nhau:

- Dữ liệu A: tài xế giảm tốc khi gặp đèn vàng, giữ làn, tuân thủ tốc độ.
- Dữ liệu B: tài xế vượt ẩu, lấn làn, vượt đèn đỏ.

Nếu huấn luyện hai mô hình trên hai bộ dữ liệu này, hai mô hình có thể cho hành vi rất khác nhau dù kiến trúc mạng giống nhau.

1.10 Example 3: Cat & Dog Classification – Ví dụ 3: Nhận dạng chó và mèo từ ảnh

1.10.1 Mô tả bài toán

Input là một ảnh, output là nhãn:

$$y \in \{\text{chó}, \text{mèo}\}$$

Nếu ảnh là ảnh xám, mỗi pixel là một con số. Tuy nhiên, việc phân loại chó hay mèo không chỉ phụ thuộc vào một pixel riêng lẻ mà phụ thuộc vào cấu trúc tổng thể của ảnh.

1.10.2 Vì sao đây là bài toán phi tuyến?

Nếu ta chỉ dùng tổ hợp tuyến tính:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

thì mô hình chỉ học được ranh giới tuyến tính trong không gian dữ liệu. Nhưng ảnh chó và mèo có sự biến thiên rất phức tạp: tư thế khác nhau, ánh sáng khác nhau, nền khác nhau, giống loài khác nhau.

Do đó, cần các lớp ẩn và hàm kích hoạt phi tuyến để mô hình học được đặc trưng phức tạp hơn.

Ghi nhớ

Bài toán nhận dạng ảnh là ví dụ điển hình cho thấy sức mạnh của mạng nơ-ron: máy học từ rất nhiều ví dụ thay vì con người phải tự viết mọi quy tắc nhận dạng.

1.11 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
ANN	Artificial Neural Network, tức mạng nơ-ron nhân tạo. Đây là hệ thống tính toán lấy cảm hứng từ mạng nơ-ron sinh học.	Mạng phân loại ảnh chó/mèo.
Nơ-ron nhân tạo	Phần tử tính toán cơ bản trong ANN, nhận nhiều input và sinh một output.	Một nơ-ron nhận giá cổ phiếu các ngày trước và tạo tín hiệu dự đoán.
Input	Dữ liệu đầu vào của mô hình.	Ảnh camera, tín hiệu sóng não, giá cổ phiếu 5 ngày trước.
Output	Kết quả đầu ra của mô hình.	Nhãn “chó”, “mèo”, giá dự đoán, góc lái.
Label	Nhãn đúng đi kèm dữ liệu trong học có giám sát.	Email được gán nhãn “spam”.
Học có giám sát	Phương pháp học từ các cặp input–output đúng.	Học từ ảnh và nhãn chó/mèo.
Training	Quá trình huấn luyện mô hình bằng dữ liệu.	Điều chỉnh trọng số để giảm sai số dự đoán.
Model	Mô hình đã học được quan hệ từ dữ liệu.	Mô hình dự đoán giá cổ phiếu sau khi huấn luyện.
Trọng số	Hệ số cho biết mức ảnh hưởng của từng input đến nơ-ron.	$w_1 = 0.8$ nghĩa là input x_1 có ảnh hưởng đáng kể.
Bias	Hằng số cộng thêm vào tổng có trọng số, giúp dịch ngưỡng kích hoạt.	$z = w_1x_1 + w_2x_2 + b$.
Hàm kích hoạt	Hàm biến đổi tổng có trọng số thành output của nơ-ron, thường là hàm phi tuyến.	Sigmoid, tanh.
Sigmoid	Hàm kích hoạt có giá trị từ 0 đến 1.	Dùng để biểu diễn xác suất email là spam.
Tanh	Hàm kích hoạt có giá trị từ -1 đến 1.	Dùng khi cần output có cả giá trị âm và dương.
Tham số	Đại lượng được học từ dữ liệu.	Trọng số và bias.
Siêu tham số	Đại lượng do người thiết kế chọn trước khi huấn luyện.	Số lớp ẩn, số nơ-ron, loại hàm kích hoạt.
Lớp input	Lớp nhận dữ liệu đầu vào.	10.000 pixel ảnh được đưa vào lớp input.
Lớp ẩn	Lớp trung gian xử lý thông tin giữa input và output.	Một mạng có 3 lớp ẩn.
Lớp output	Lớp sinh ra kết quả cuối cùng.	Một nơ-ron output dự đoán giá cổ phiếu.
Kết nối đầy đủ	Mỗi nơ-ron của lớp trước nối với tất cả nơ-ron của lớp sau.	Fully connected layer.

Thuật ngữ	Giải thích	Ví dụ
Mạng truyền thẳng	Mạng trong đó tín hiệu đi từ input qua lớp ẩn đến output, không quay ngược vòng trong lúc dự đoán.	Feedforward neural network.
Deep Learning	Học sâu; thường chỉ các mạng có nhiều lớp ẩn.	Mạng nhiều lớp dùng trong nhận dạng ảnh.
Phi tuyến	Quan hệ không thể biểu diễn tốt bằng một đường thẳng hoặc tổ hợp tuyến tính đơn giản.	Quan hệ giữa pixel ảnh và nhãn chó/mèo.
Pixel	Điểm ảnh, đơn vị nhỏ nhất của ảnh số.	Ảnh xám có pixel giá trị từ 0 đến 255.
Chuỗi thời gian	Dữ liệu được ghi nhận theo thứ tự thời gian.	Giá cổ phiếu từng ngày, nhiệt độ từng giờ.
Cơ cấu chấp hành	Bộ phận thực hiện lệnh điều khiển từ hệ thống.	Đèn, động cơ, phanh, tay lái.
Sóng điện não	Tín hiệu điện sinh ra từ hoạt động của não.	Tín hiệu dùng để điều khiển bật/tắt đèn.
BCI	Brain-Computer Interface, tức giao diện não-máy tính.	Hệ thống dùng sóng não để điều khiển thiết bị.

1.12 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

1.12.1 Các ý chính cần nhớ

1. ANN là hệ thống tính toán lấy cảm hứng từ mạng nơ-ron sinh học.
2. ANN học bằng cách khảo sát dữ liệu và ví dụ.
3. Trong học có giám sát, dữ liệu huấn luyện gồm các cặp input-output.
4. Một nơ-ron nhân tạo tính:

$$y = \varphi \left(\sum_{i=1}^n w_i x_i + b \right)$$

5. Trọng số và bias là tham số được học từ dữ liệu.
6. Hàm kích hoạt giúp mạng học quan hệ phi tuyến.
7. Sigmoid cho output từ 0 đến 1; tanh cho output từ -1 đến 1.
8. Mạng truyền thẳng nhiều lớp gồm input layer, hidden layers và output layer.
9. Mô hình học theo dữ liệu được cung cấp; dữ liệu xấu có thể tạo ra mô hình xấu.

1.12.2 Sơ đồ tư duy ngắn gọn

Dữ liệu $\rightarrow$ Huấn luyện $\rightarrow$ Tham số học được $\rightarrow$ Model $\rightarrow$ Dự đoán trên dữ liệu mới

1.13 Review Questions – Câu hỏi ôn tập

1. Mạng nơ-ron nhân tạo lấy cảm hứng từ đâu?
2. Học có giám sát khác gì so với việc lập trình luật thủ công?
3. Vì sao bài toán phân biệt ảnh chó và mèo thường là bài toán phi tuyến?
4. Viết công thức toán học của một nơ-ron nhân tạo.
5. Trọng số và bias có vai trò gì?
6. Hàm kích hoạt dùng để làm gì?
7. So sánh sigmoid và tanh về miền giá trị.
8. Tham số khác siêu tham số như thế nào?
9. Trong bài toán dự đoán giá cổ phiếu từ 5 ngày trước, input và output là gì?
10. Trong bài toán xe tự lái, vì sao chất lượng tài xế thu thập dữ liệu ảnh hưởng đến mô hình?

1.14 Practice Exercises – Bài tập vận dụng

Bài tập 1: Tính output của nơ-ron

Cho:

$$x_1 = 1, \quad x_2 = 4, \quad w_1 = 2, \quad w_2 = -0.5, \quad b = 1$$

1. Tính $z = w_1x_1 + w_2x_2 + b$.
2. Tính $y = \sigma(z)$ với:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Lời giải gợi ý

$$z = 2 \cdot 1 + (-0.5) \cdot 4 + 1 = 2 - 2 + 1 = 1$$

$$y = \sigma(1) = \frac{1}{1 + e^{-1}} \approx 0.731$$

Bài tập 2: Xác định input và output

Với mỗi bài toán sau, hãy xác định input và output:

1. Nhận dạng biển số xe.
2. Dự đoán thời tiết ngày mai.
3. Điều khiển đèn bằng sóng não.
4. Phân loại email spam.

Gợi ý

Bài toán	Input	Output
Nhận dạng biển số	Ảnh biển số	Chuỗi ký tự biển số
Dự đoán thời tiết	Dữ liệu thời tiết các ngày trước	Thời tiết ngày mai
Điều khiển bằng sóng não	Tín hiệu điện não	Lệnh điều khiển
Phân loại email spam	Nội dung email	Spam hoặc không spam

Kết luận

Bài học ANN1 cung cấp nền tảng đầu tiên về mạng nơ-ron nhân tạo. Trọng tâm của bài là hiểu ANN như một hệ thống học từ dữ liệu, mô phỏng một phần ý tưởng từ nơ-ron sinh học, có khả năng xử lý các quan hệ phi tuyến và được ứng dụng rộng rãi trong nhận dạng, dự đoán, điều khiển, y khoa, giáo dục và kinh doanh.

Muốn hiểu sâu hơn ANN, người học cần tiếp tục nắm các nội dung tiếp theo như lan truyền tiến, hàm mất mát, lan truyền ngược, gradient descent, overfitting, regularization và các kiến trúc mạng chuyên biệt như CNN, RNN, Transformer.

Chương 2

ANN3: Forward Propagation – Lan truyền tiến

Learning Objectives – Mục tiêu bài học

Bài ANN3 tập trung vào quá trình huấn luyện mạng nơ-ron nhân tạo. Sau khi học xong, người học cần nắm được:

1. Vì sao mạng nơ-ron ban đầu chưa có khả năng suy luận.
2. Ý nghĩa của việc khởi tạo ngẫu nhiên trọng số và bias.
3. Quá trình lan truyền tiến từ input đến output.
4. Ý tưởng tổng quát của lan truyền ngược sai số, tiếng Anh là *backpropagation*.
5. Vai trò của dữ liệu, nhãn, output dự đoán và sai số.
6. Cách biểu diễn nhãn phân loại bằng one-hot vector.
7. Một số cách đo sai số giữa hai vector: chuẩn L_2 , góc giữa hai vector và cross entropy.
8. Vì sao huấn luyện mạng nơ-ron là quá trình lặp nhiều lần và có thể tốn nhiều thời gian.

2.1 Supervised Learning Recap – Nhắc lại bài toán học có giám sát

2.1.1 Mạng nơ-ron cần học ánh xạ

Trong các bài trước, ta đã biết mạng nơ-ron nhân tạo được dùng để xấp xỉ một hàm ánh xạ:

$$f : X \rightarrow Y$$

Trong đó:

- X là không gian dữ liệu đầu vào;

- Y là không gian kết quả đầu ra;
- f là quan hệ mà mô hình cần học.

Ví dụ:

Ảnh con chó $\rightarrow$ Nhãn “chó”

hoặc:

Giá cổ phiếu 5 ngày gần nhất $\rightarrow$ Giá cổ phiếu ngày tiếp theo

Ghi nhớ

Ban đầu, mạng nơ-ron chưa biết hàm f . Quá trình huấn luyện chính là quá trình điều chỉnh các tham số bên trong mạng để mạng học được ánh xạ từ input sang output.

2.1.2 Dữ liệu trong học có giám sát

Trong học có giám sát, tiếng Anh là *supervised learning*, ta cung cấp cho hệ thống nhiều cặp dữ liệu:

$$(x_i, y_i)$$

Trong đó:

- x_i là dữ liệu đầu vào, còn gọi là input hoặc data;
- y_i là nhãn đúng, còn gọi là label hoặc target.

Tập dữ liệu huấn luyện có dạng:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

Mục tiêu của mô hình là tạo ra dự đoán:

$$\hat{y}_i = f(x_i)$$

sao cho $\hat{y}_i$ càng gần y_i càng tốt.

Ví dụ minh họa

Bài toán phân loại ảnh gồm 4 lớp: chó, mèo, điều và xe hơi.

Mẫu dữ liệu	Input	Label đúng
1	Ảnh con chó	Chó
2	Ảnh con mèo	Mèo
3	Ảnh con điều	Điều
4	Ảnh xe hơi	Xe hơi

Mạng nơ-ron phải học từ nhiều ảnh đã được gán nhãn. Sau khi huấn luyện, khi gặp ảnh mới, mạng dự đoán ảnh thuộc lớp nào.

2.2 Initial Network Parameters – Thông số ban đầu của mạng nơ-ron

2.2.1 Tham số của từng nơ-ron

Một nơ-ron nhân tạo nhận nhiều đầu vào:

$$x_1, x_2, \dots, x_n$$

Mỗi đầu vào có một trọng số tương ứng:

$$w_1, w_2, \dots, w_n$$

Ngoài ra, nơ-ron còn có bias b . Giá trị trước hàm kích hoạt là:

$$z = \sum_{i=1}^n w_i x_i + b$$

Output của nơ-ron là:

$$a = \varphi(z)$$

Trong đó φ là hàm kích hoạt.

2.2.2 Mạng có rất nhiều tham số

Khi nhiều nơ-ron được kết nối thành một mạng, mỗi nơ-ron có bộ trọng số và bias riêng. Tổng số tham số trong mạng có thể rất lớn.

Ví dụ, nếu một nơ-ron ở lớp hiện tại nhận kết nối từ 5 nơ-ron ở lớp trước, thì nó có:

$$5 \text{ trọng số} + 1 \text{ bias} = 6 \text{ tham số}$$

Nếu mạng có hàng triệu kết nối, tổng số tham số có thể lên đến vài chục triệu hoặc vài trăm triệu.

Ghi nhớ

Huấn luyện mạng nơ-ron thực chất là tìm bộ giá trị phù hợp cho tất cả các trọng số và bias trong mạng.

2.2.3 Khởi tạo ngẫu nhiên

Khi bắt đầu huấn luyện, mạng chưa biết gì về bài toán. Vì vậy, các trọng số và bias thường được khởi tạo bằng các giá trị ngẫu nhiên.

Ký hiệu bộ tham số của mạng là:

$$\theta = \{W, b\}$$

Ban đầu:

$$\theta = \theta_0$$

Trong đó θ_0 là bộ tham số khởi tạo ngẫu nhiên.

Lưu ý

Trong thực tế, việc khởi tạo không phải là ngẫu nhiên hoàn toàn tùy tiện. Có nhiều kỹ thuật khởi tạo đặc biệt để giúp quá trình huấn luyện ổn định hơn. Tuy nhiên, ở mức nhập môn, ta có thể hiểu đơn giản rằng mạng bắt đầu từ một trạng thái ngẫu nhiên.

2.2.4 Vì sao output ban đầu thường sai?

Do các trọng số ban đầu là ngẫu nhiên, nên khi đưa input vào mạng, output cũng thường là một kết quả không có ý nghĩa.

Ví dụ minh họa

Giả sử đưa ảnh con chó vào mạng.

$$x = \text{ảnh con chó}$$

Label đúng là:

$$y = \text{chó}$$

Nhưng vì mạng mới khởi tạo ngẫu nhiên, nó có thể dự đoán:

$$\hat{y} = \text{máy bay}$$

Kết quả này sai vì mạng chưa được huấn luyện.

2.3 Forward Propagation – Lan truyền tiến

2.3.1 Ý tưởng của lan truyền tiến

Lan truyền tiến, tiếng Anh là *forward propagation*, là quá trình đưa dữ liệu từ lớp input đi qua các lớp ẩn và cuối cùng sinh ra output.

$$\text{Input} \rightarrow \text{Lớp ẩn 1} \rightarrow \text{Lớp ẩn 2} \rightarrow \dots \rightarrow \text{Output}$$

Ở mỗi nơ-ron, ta tính:

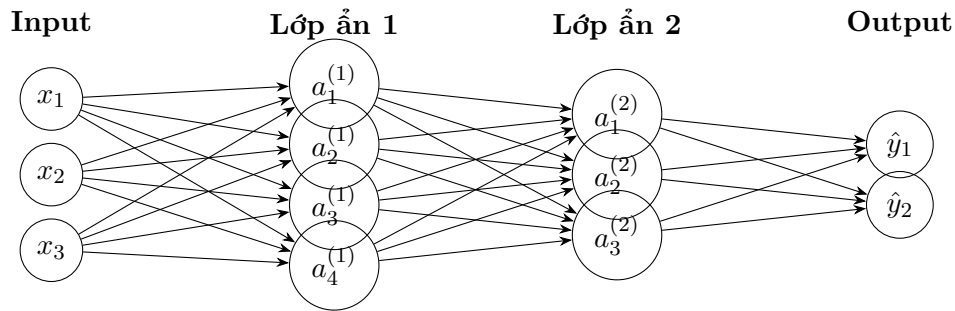
$$z = \sum_{i=1}^n w_i x_i + b$$

sau đó đưa qua hàm kích hoạt:

$$a = \varphi(z)$$

Output của lớp trước trở thành input của lớp sau.

2.3.2 Sơ đồ lan truyền tiến



2.3.3 Lan truyền tiến qua nhiều lớp

Giả sử mạng có L lớp. Với lớp thứ l , ta có:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \varphi(z^{(l)})$$

Trong đó:

- $a^{(0)} = x$ là input ban đầu;
- $W^{(l)}$ là ma trận trọng số của lớp l ;
- $b^{(l)}$ là vector bias của lớp l ;
- $a^{(l)}$ là output của lớp l ;
- output cuối cùng của mạng là $\hat{y} = a^{(L)}$.

Ví dụ minh họa

Xét một mạng đơn giản:

$$x \rightarrow \text{lớp ẩn} \rightarrow \hat{y}$$

Input:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Giả sử lớp ẩn có 2 nơ-ron, dùng hàm kích hoạt tuyến tính để tính đơn giản:

$$W^{(1)} = \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Ta tính:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$z^{(1)} = \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 3.5 \end{bmatrix}$$

Nếu hàm kích hoạt là tuyến tính, output lớp ẩn là:

$$a^{(1)} = \begin{bmatrix} 1 \\ 3.5 \end{bmatrix}$$

2.4 Error & Training Concept – Sai số và ý tưởng huấn luyện

2.4.1 So sánh output dự đoán với output mong muốn

Sau khi lan truyền tiến, mạng tạo ra output dự đoán:

$$\hat{y}$$

Trong học có giám sát, ta biết output đúng:

$$y$$

Sai số được hiểu là sự khác biệt giữa $\hat{y}$ và y .

Sai số = khác biệt giữa output dự đoán và label đúng

Ví dụ minh họa

Nếu ảnh đầu vào là ảnh con chó, label đúng là:

$$y = \text{chó}$$

Nhưng mạng dự đoán:

$$\hat{y} = \text{máy bay}$$

Thì mạng đã sai. Ta cần đo mức độ sai đó bằng một hàm mất mát.

2.4.2 Hàm mất mát

Hàm mất mát, tiếng Anh là *loss function*, là hàm dùng để đo sai số giữa output dự đoán và output mong muốn.

$$\mathcal{L}(\hat{y}, y)$$

Hàm mất mát phải trả về một số vô hướng:

$$\mathcal{L} \in \mathbb{R}$$

Số này càng nhỏ thì dự đoán càng gần với nhãn đúng.

Ghi nhớ

Hàm mất mát biến khái niệm “mô hình dự đoán sai” thành một con số cụ thể. Nhờ đó, thuật toán huấn luyện có thể biết cần điều chỉnh tham số theo hướng nào.

2.4.3 Huấn luyện là giảm sai số

Mục tiêu của huấn luyện là tìm bộ tham số θ sao cho loss nhỏ nhất có thể:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\hat{y}, y)$$

Với toàn bộ tập dữ liệu:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(x_i; \theta), y_i)$$

Mục tiêu là:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

2.5 Lan truyền ngược sai số

2.5.1 Backpropagation là gì?

Lan truyền ngược sai số, tiếng Anh là *backpropagation*, là kỹ thuật dùng sai số ở output để điều chỉnh các tham số bên trong mạng.

Quá trình tổng quát:

Input $\rightarrow$ Lan truyền tiến $\rightarrow$ Output dự đoán

So sánh với label $\rightarrow$ Tính loss

Lan truyền ngược sai số $\rightarrow$ Cập nhật trọng số và bias

Ghi nhớ

Backpropagation là một trong những kỹ thuật phổ biến nhất để huấn luyện mạng nơ-ron hiện nay, nhưng không phải là kỹ thuật duy nhất.

2.5.2 Ý tưởng trực quan

Ban đầu, mạng giống như một đứa trẻ chưa biết làm gì. Các trọng số được khởi tạo ngẫu nhiên nên mạng trả lời sai. Mỗi khi mạng sai, ta dùng mức độ sai đó để chỉnh lại các trọng số một chút.

Sau rất nhiều lần chỉnh như vậy, mạng dần học được cách tạo output gần với label đúng.

Ví dụ minh họa

Quá trình học qua ba mẫu dữ liệu:

1. Đưa ảnh con chó vào. Mạng đoán “máy bay”. So với label “chó”, mạng sai. Ta cập nhật trọng số.
2. Đưa ảnh con mèo vào. Mạng đoán “xe tải”. So với label “mèo”, mạng sai. Ta tiếp tục cập nhật trọng số.
3. Đưa ảnh con voi vào. Mạng đoán “ngôi nhà”. So với label “voi”, mạng sai. Ta lại cập nhật trọng số.

Lặp lại quá trình này với rất nhiều mẫu dữ liệu, mạng có cơ hội học dần quan hệ giữa ảnh và nhãn.

2.5.3 Cập nhật từng bước

Giả sử tại bước huấn luyện thứ t , bộ tham số của mạng là:

$$\theta_t$$

Sau khi tính loss, thuật toán cập nhật tham số:

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

Trong thực tế, nếu dùng gradient descent, công thức thường có dạng:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

Trong đó:

- η là tốc độ học, tiếng Anh là *learning rate*;
- $\nabla_{\theta} J(\theta_t)$ là gradient của hàm mất mát theo tham số;
- dấu trừ thể hiện việc đi theo hướng làm giảm loss.

Lưu ý

Trong transcript, giảng viên chủ yếu trình bày ý tưởng tổng quát: dùng sai số giữa output hiện tại và output mong muốn để quay lại điều chỉnh các trọng số. Để lập trình được backpropagation, cần học thêm các chi tiết toán học như đạo hàm riêng, gradient, chain rule và thuật toán tối ưu.

2.5.4 Liên hệ với điều khiển tự động

Ý tưởng dùng sai lệch giữa kết quả mong muốn và kết quả hiện tại để điều chỉnh hệ thống có liên hệ với tư duy trong điều khiển tự động.

Trong điều khiển phản hồi:

$$e = r - y$$

Trong đó:

- r là giá trị mong muốn;
- y là giá trị thực tế;
- e là sai lệch.

Hệ thống dùng sai lệch e để điều chỉnh tín hiệu điều khiển.

Trong huấn luyện mạng nơ-ron:

$$\text{sai số} = \text{label đúng} - \text{output dự đoán}$$

Mạng dùng sai số này để điều chỉnh trọng số và bias.

2.6 Quá trình huấn luyện lặp

2.6.1 Huấn luyện qua từng mẫu

Với mỗi cặp dữ liệu (x_i, y_i) , ta làm các bước:

1. Đưa x_i vào mạng.
2. Lan truyền tiến để tính $\hat{y}_i$.
3. So sánh $\hat{y}_i$ với y_i .

4. Tính loss.
5. Lan truyền ngược sai số.
6. Cập nhật trọng số và bias.

Sau đó chuyển sang mẫu tiếp theo.

2.6.2 Epoch

Một epoch là một lượt mô hình đi qua toàn bộ tập dữ liệu huấn luyện.

Nếu tập dữ liệu có N mẫu, thì sau một epoch, mạng đã được huấn luyện trên N mẫu đó một lần.

Thông thường, ta cần nhiều epoch:

Training = nhiều epoch

2.6.3 Vì sao cần lặp nhiều lần?

Một lần cập nhật thường chỉ làm thay đổi tham số một chút. Vì vậy, mạng cần rất nhiều lần cập nhật để học được một quan hệ phức tạp.

Ví dụ minh họa

Nếu mạng có hàng triệu tham số và dữ liệu gồm hàng trăm nghìn ảnh, việc chỉ học một vài ảnh là không đủ. Mạng cần nhìn thấy nhiều ảnh chó, nhiều ảnh mèo, nhiều ảnh xe hơi trong nhiều điều kiện khác nhau để học được đặc trưng tổng quát.

2.6.4 Thời gian huấn luyện

Thời gian huấn luyện phụ thuộc vào:

- kích thước tập dữ liệu;
- độ phức tạp của mạng;
- số lượng tham số;
- số epoch;
- cấu hình phần cứng;
- loại GPU hoặc card màn hình;
- cách cài đặt thuật toán.

Lưu ý

Huấn luyện mạng nơ-ron có thể mất vài giờ, vài ngày hoặc thậm chí vài tuần. Với các mô hình lớn, phần cứng mạnh, đặc biệt là GPU, đóng vai trò rất quan trọng.

2.7 Bài toán phân loại nhiều lớp

2.7.1 Khái niệm class

Trong bài toán phân loại, *class* là lớp dữ liệu hoặc nhóm nhãn mà mô hình cần dự đoán.

Ví dụ bài toán có 4 class:

{chó, mèo, điều, xe hơi}

Mỗi ảnh đầu vào được gán thuộc đúng một class.

Ghi nhớ

Nếu bài toán yêu cầu mỗi mẫu chỉ thuộc một lớp duy nhất, ta gọi đó là bài toán phân loại một nhãn, tiếng Anh là *single-label classification*.

2.7.2 Thiết kế output cho bài toán nhiều lớp

Với 4 class, có nhiều cách thiết kế output.

2.7.3 Cách 1: Một output nhận giá trị 1, 2, 3, 4

Ta có thể quy ước:

1 = chó, 2 = mèo, 3 = điều, 4 = xe hơi

Khi đó mạng chỉ có một output.

Tuy nhiên, cách này có vấn đề: các con số 1, 2, 3, 4 có thứ tự số học, nhưng các class không nhất thiết có quan hệ thứ tự. Ví dụ, không có nghĩa là “xe hơi” lớn hơn “chó” theo nghĩa toán học.

2.7.4 Cách 2: Hai output theo mã nhị phân

Với 4 class, ta có thể dùng 2 bit:

00, 01, 10, 11

Ví dụ:

00 = chó, 01 = mèo, 10 = điều, 11 = xe hơi

Cách này tiết kiệm số output hơn nhưng không phải cách phổ biến nhất trong bài toán phân loại cơ bản.

2.7.5 Cách 3: Bốn output theo one-hot vector

Cách phổ biến là dùng số output bằng số class. Với 4 class, ta dùng 4 output.

Mỗi label được biểu diễn bằng một vector có đúng một phần tử bằng 1, các phần tử còn lại bằng 0.

$$\text{chó} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \text{mèo} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \text{điều} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \text{xe hơi} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Ghi nhớ

One-hot vector là cách biểu diễn nhãn rất phổ biến trong bài toán phân loại nhiều lớp.

2.8 One-hot encoding

2.8.1 One-hot vector là gì?

One-hot vector là vector trong đó chỉ có một phần tử mang giá trị 1, các phần tử còn lại mang giá trị 0.

Nếu có K class, one-hot vector sẽ có K phần tử.

Ví dụ với $K = 4$:

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Vector này biểu diễn class thứ 2.

2.8.2 Bảng mã hóa one-hot

Class	Vector one-hot	Ý nghĩa
Chó	$[1, 0, 0, 0]^T$	Phần tử thứ nhất bằng 1
Mèo	$[0, 1, 0, 0]^T$	Phần tử thứ hai bằng 1
Điều	$[0, 0, 1, 0]^T$	Phần tử thứ ba bằng 1
Xe hơi	$[0, 0, 0, 1]^T$	Phần tử thứ tư bằng 1

2.8.3 Output dự đoán của mạng

Trong thực tế, output của mạng thường không ra đúng tuyệt đối các giá trị 0 và 1. Nó có thể ra một vector điểm số hoặc xác suất.

Ví dụ, với ảnh con chó:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Nhưng mạng dự đoán:

$$\hat{y} = \begin{bmatrix} 0.70 \\ 0.20 \\ 0.05 \\ 0.05 \end{bmatrix}$$

Ta có thể hiểu mạng đang cho rằng:

- xác suất là chó: 70%;
- xác suất là mèo: 20%;
- xác suất là điều: 5%;
- xác suất là xe hơi: 5%.

Dự đoán cuối cùng là class có giá trị lớn nhất:

$$\arg \max(\hat{y}) = \text{chó}$$

Ví dụ minh họa

Nếu mạng cho output:

$$\hat{y} = \begin{bmatrix} 0.1 \\ 0.6 \\ 0.2 \\ 0.1 \end{bmatrix}$$

thì phần tử lớn nhất là 0.6, nằm ở vị trí thứ 2. Mạng dự đoán ảnh thuộc class thứ 2, tức là “mèo”.

2.9 Đo sai số giữa hai vector

2.9.1 Vì sao cần đo sai số giữa hai vector?

Trong bài toán phân loại nhiều lớp, label đúng thường là một vector one-hot, còn output dự đoán cũng là một vector.

Ví dụ:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Ta cần một cách đo sự khác biệt giữa hai vector này.

Ghi nhớ

Khi output và label đều là vector, sai số không còn đơn giản là phép trừ giữa hai số. Ta cần dùng các đại lượng như chuẩn vector, góc giữa hai vector hoặc cross entropy.

2.10 Hàm mất mát dựa trên chuẩn L2 - NORM 2

2.10.1 Chuẩn L2 của vector

Với vector:

$$v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

Chuẩn L_2 của v là:

$$\|v\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$

Đây chính là độ dài Euclid của vector.

2.10.2 Sai số L2 giữa hai vector

Với label đúng y và output dự đoán $\hat{y}$, vector sai số là:

$$e = y - \hat{y}$$

Sai số theo chuẩn L_2 là:

$$\|y - \hat{y}\|_2$$

Hoặc thường dùng bình phương chuẩn L_2 :

$$\|y - \hat{y}\|_2^2$$

2.10.3 Ví dụ tính sai số L2

Giả sử:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Khi đó:

$$e = y - \hat{y} = \begin{bmatrix} 1 - 0.7 \\ 0 - 0.1 \\ 0 - 0.1 \\ 0 - 0.1 \end{bmatrix} = \begin{bmatrix} 0.3 \\ -0.1 \\ -0.1 \\ -0.1 \end{bmatrix}$$

Chuẩn L_2 :

$$\|e\|_2 = \sqrt{0.3^2 + (-0.1)^2 + (-0.1)^2 + (-0.1)^2}$$

$$\|e\|_2 = \sqrt{0.09 + 0.01 + 0.01 + 0.01} = \sqrt{0.12} \approx 0.346$$

Bình phương chuẩn L_2 :

$$\|e\|_2^2 = 0.12$$

Ghi nhớ

Sai số L_2 càng nhỏ thì hai vector càng gần nhau.

2.11 Đo khác biệt bằng góc giữa hai vector

2.11.1 Tích vô hướng

Với hai vector:

$$p = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix}, \quad q = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{bmatrix}$$

Tích vô hướng là:

$$p \cdot q = p_1q_1 + p_2q_2 + \cdots + p_nq_n$$

Tích vô hướng cho ra một số vô hướng, không phải một vector.

Lưu ý

Không nhầm tích vô hướng với tích có hướng. Tích vô hướng giữa hai vector cho ra một số thực. Tích có hướng mới cho ra một vector trong không gian ba chiều.

2.11.2 Góc giữa hai vector

Góc θ giữa hai vector p và q được tính bởi:

$$\cos \theta = \frac{p \cdot q}{\|p\|_2 \|q\|_2}$$

Suy ra:

$$\theta = \arccos \left(\frac{p \cdot q}{\|p\|_2 \|q\|_2} \right)$$

2.11.3 Ý nghĩa

- Nếu θ gần 0° , hai vector gần cùng hướng, tức là tương đồng cao.
- Nếu θ gần 90° , hai vector gần vuông góc, tức là ít liên quan theo hướng.

- Nếu θ gần 180° , hai vector gần ngược hướng.

Ví dụ minh họa

Cho:

$$p = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad q = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Ta có:

$$p \cdot q = 1 \cdot 0 + 0 \cdot 1 = 0$$

$$\|p\|_2 = 1, \quad \|q\|_2 = 1$$

Do đó:

$$\cos \theta = \frac{0}{1 \cdot 1} = 0$$

$$\theta = 90^\circ$$

Hai vector vuông góc với nhau.

2.11.4 Cosine similarity

Trong nhiều ứng dụng, thay vì dùng trực tiếp góc, ta dùng cosine similarity:

$$\text{cosine similarity}(p, q) = \frac{p \cdot q}{\|p\|_2 \|q\|_2}$$

Giá trị này càng gần 1 thì hai vector càng giống nhau về hướng.

Một dạng loss có thể xây dựng từ cosine similarity là:

$$\mathcal{L}_{\cos} = 1 - \frac{p \cdot q}{\|p\|_2 \|q\|_2}$$

2.12 Cross entropy

2.12.1 Ý tưởng của cross entropy

Cross entropy là một hàm mất mát rất phổ biến trong bài toán phân loại.

Giả sử:

p = phân phối đúng

q = phân phối dự đoán

Cross entropy giữa p và q được định nghĩa:

$$H(p, q) = - \sum_{i=1}^K p_i \log(q_i)$$

Trong đó:

- K là số class;
- p_i là xác suất đúng của class i ;
- q_i là xác suất mô hình dự đoán cho class i .

2.12.2 Cross entropy với one-hot label

Nếu label đúng là one-hot vector, chỉ có một phần tử $p_c = 1$, các phần tử còn lại bằng 0.

Khi đó:

$$H(p, q) = -\log(q_c)$$

Trong đó q_c là xác suất mô hình gán cho class đúng.

Ghi nhớ

Với one-hot label, cross entropy chỉ quan tâm trực tiếp đến xác suất mà mô hình gán cho class đúng. Nếu mô hình gán xác suất cao cho class đúng, loss nhỏ. Nếu mô hình gán xác suất thấp cho class đúng, loss lớn.

2.12.3 Ví dụ tính cross entropy

Giả sử có 4 class:

{chó, mèo, diều, xe hơi}

Ảnh thật là chó, nên:

$$p = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Mạng dự đoán:

$$q = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Cross entropy:

$$H(p, q) = -[1 \log(0.7) + 0 \log(0.1) + 0 \log(0.1) + 0 \log(0.1)]$$

$$H(p, q) = -\log(0.7) \approx 0.357$$

Nếu mạng dự đoán tệ hơn:

$$q = \begin{bmatrix} 0.1 \\ 0.6 \\ 0.2 \\ 0.1 \end{bmatrix}$$

thì:

$$H(p, q) = -\log(0.1) \approx 2.303$$

Loss lớn hơn nhiều vì mô hình chỉ gán xác suất 10% cho class đúng.

2.12.4 So sánh các cách đo sai số

Cách đo	Ý tưởng	Thường dùng khi
Chuẩn L_2	Đo độ dài vector sai số $y - \hat{y}$	Bài toán hồi quy hoặc so sánh vector đơn giản
Góc giữa hai vector	Đo sự giống nhau về hướng	Bài toán so sánh độ tương đồng vector
Cross entropy	Đo sai khác giữa phân phối đúng và phân phối dự đoán	Bài toán phân loại nhiều lớp

2.13 Mô hình học được những gì?

2.13.1 Mô hình chỉ học trong phạm vi dữ liệu được dạy

Một điểm quan trọng trong transcript là: mô hình học những gì nó được dạy.

Nếu tập dữ liệu chỉ gồm 10 loại con vật, mô hình chỉ được học để phân biệt 10 loại đó. Nếu đưa vào một con vật thứ 11 chưa từng xuất hiện trong dữ liệu huấn luyện, mô hình có thể trả lời sai hoặc trả lời một cách không đáng tin cậy.

Ví dụ minh họa

Giả sử mô hình được huấn luyện với 4 class:

{chó, mèo, diều, xe hơi}

Nếu đưa ảnh một con voi vào, nhưng mô hình không có class “voi”, nó vẫn có thể buộc phải chọn một trong 4 class đã biết. Ví dụ, nó có thể dự đoán nhầm là “chó” hoặc “xe hơi”.

2.13.2 Số lượng class trong các bộ dữ liệu lớn

Trong thực tế, nhiều mô hình phân loại ảnh được huấn luyện trên các tập dữ liệu có rất nhiều class. Ví dụ, một bộ dữ liệu có thể có 1000 class, bao gồm nhiều loại động vật, phương tiện, đồ vật và cảnh vật khác nhau.

Khi số class tăng, output layer cũng thường tăng số nơ-ron tương ứng nếu dùng one-hot encoding.

K class $\Rightarrow K$ output

2.13.3 Không phải cứ huấn luyện là thành công

Huấn luyện mạng nơ-ron không đảm bảo chắc chắn mô hình sẽ tốt. Kết quả phụ thuộc vào nhiều yếu tố:

- dữ liệu có đủ lớn không;
- dữ liệu có đúng và sạch không;
- nhãn có chính xác không;
- kiến trúc mạng có phù hợp không;
- hàm mất mát có phù hợp không;
- tốc độ học có phù hợp không;
- số epoch có đủ không;
- có bị overfitting hay underfitting không.

Cảnh báo

Nếu chỉ hiểu ý tưởng tổng quát của backpropagation mà không hiểu các chi tiết toán học và kỹ thuật, việc tự lập trình hoặc huấn luyện mô hình phức tạp rất dễ thất bại.

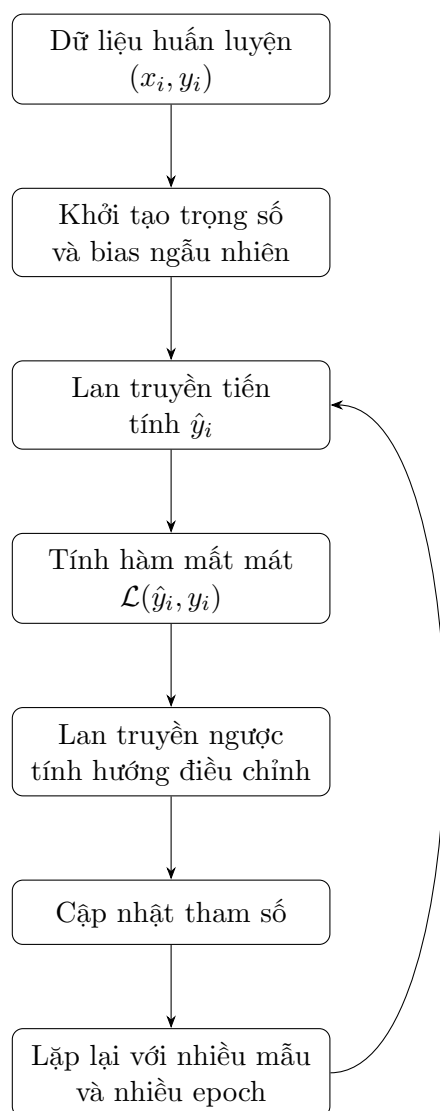
2.14 Tổng hợp quy trình huấn luyện ANN

2.14.1 Quy trình tổng quát

Một quy trình huấn luyện ANN có thể được mô tả như sau:

1. Chuẩn bị tập dữ liệu gồm các cặp (x_i, y_i) .
2. Thiết kế kiến trúc mạng.
3. Chọn hàm kích hoạt.
4. Chọn cách biểu diễn output, ví dụ one-hot vector.
5. Chọn hàm mất mát.
6. Khởi tạo ngẫu nhiên trọng số và bias.
7. Thực hiện lan truyền tiến để tính output dự đoán.
8. Tính loss giữa output dự đoán và label đúng.
9. Dùng backpropagation để tính hướng điều chỉnh tham số.
10. Cập nhật tham số.
11. Lặp lại quá trình trên nhiều lần cho đến khi mô hình đạt chất lượng mong muốn.

2.14.2 Sơ đồ tổng quát



2.15 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
ANN	Artificial Neural Network, mạng nơ-ron nhân tạo.	Mạng phân loại ảnh.
Training	Quá trình huấn luyện mô hình bằng dữ liệu.	Cho mạng học từ ảnh và nhãn.
Supervised learning	Học có giám sát, mô hình học từ các cặp input-label.	Ảnh chó đi kèm nhãn “chó”.
Input	Dữ liệu đầu vào của mạng.	Một ảnh, một vector giá cổ phiếu.
Label	Nhãn đúng của dữ liệu.	Chó, mèo, xe hơi.

Thuật ngữ	Giải thích	Ví dụ
Target Output	Đầu ra mong muốn, thường tương đương label. Kết quả mà mạng tạo ra.	Vector $[1, 0, 0, 0]^T$. $\hat{y} = [0.7, 0.1, 0.1, 0.1]^T$.
Parameter	Tham số được học từ dữ liệu.	Trọng số w và bias b .
Weight	Trọng số, biểu diễn mức ảnh hưởng của một kết nối.	$w_1 = 0.5$.
Bias	Hằng số cộng thêm vào tổng có trọng số.	$z = wx + b$.
Random initialization	Khởi tạo tham số ban đầu bằng giá trị ngẫu nhiên.	Trọng số ban đầu lấy ngẫu nhiên.
Forward propagation	Lan truyền tiến từ input đến output.	Tính lần lượt qua các lớp.
Backpropagation	Lan truyền ngược sai số để cập nhật tham số.	Dùng loss để chỉnh trọng số.
Loss function	Hàm mất mát, đo sai khác giữa dự đoán và label.	Cross entropy.
Error	Sai số giữa output dự đoán và output mong muốn.	$e = y - \hat{y}$.
Gradient	Vector đạo hàm cho biết hướng tăng nhanh nhất của loss.	$\nabla_{\theta} J(\theta)$.
Learning rate	Tốc độ học, quyết định bước cập nhật tham số lớn hay nhỏ.	$\eta = 0.001$.
Epoch	Một lượt đi qua toàn bộ tập dữ liệu huấn luyện.	Huấn luyện 50 epoch.
Class	Lớp dữ liệu trong bài toán phân loại.	Chó, mèo, xe hơi.
Classification	Bài toán phân loại dữ liệu vào các class.	Phân loại ảnh động vật.
One-hot vector	Vector có đúng một phần tử bằng 1, còn lại bằng 0.	$[0, 1, 0, 0]^T$.
One-hot encoding	Cách mã hóa nhãn bằng one-hot vector.	Mèo $\rightarrow [0, 1, 0, 0]^T$.
L_2 norm	Chuẩn Euclid của vector.	$\ v\ _2 = \sqrt{\sum v_i^2}$.
Dot product	Tích vô hướng giữa hai vector, cho ra một số.	$p \cdot q = \sum p_i q_i$.
Cosine similarity	Độ tương đồng theo góc giữa hai vector.	$\frac{p \cdot q}{\ p\ \ q\ }$.
Cross entropy	Hàm mất mát phổ biến cho phân loại.	$-\sum p_i \log q_i$.
GPU	Bộ xử lý đồ họa, thường dùng để tăng tốc huấn luyện mạng lớn.	Card màn hình NVIDIA.

2.16 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

2.16.1 Các ý chính cần nhớ

1. Ban đầu, mạng nơ-ron chưa biết suy luận vì các tham số được khởi tạo ngẫu nhiên.
2. Khi đưa input vào mạng, mạng thực hiện lan truyền tiến để tạo output dự đoán.
3. Output dự đoán được so sánh với label đúng để tính sai số.

4. Hàm mất mát biến sai số thành một số vô hướng.
5. Backpropagation dùng sai số này để quay ngược lại điều chỉnh các trọng số và bias.
6. Quá trình cập nhật tham số được lặp lại rất nhiều lần.
7. Trong phân loại nhiều lớp, label thường được biểu diễn bằng one-hot vector.
8. Các cách đo sai số giữa vector gồm chuẩn L_2 , góc giữa hai vector và cross entropy.
9. Cross entropy đặc biệt phổ biến trong bài toán phân loại.
10. Mô hình chỉ học tốt những gì được thể hiện đủ rõ và đủ đúng trong dữ liệu huấn luyện.

2.16.2 Công thức quan trọng

Lan truyền tiến qua một lớp

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \varphi(z^{(l)})$$

Sai số vector

$$e = y - \hat{y}$$

Chuẩn L2

$$\|e\|_2 = \sqrt{e_1^2 + e_2^2 + \dots + e_n^2}$$

Góc giữa hai vector

$$\cos \theta = \frac{p \cdot q}{\|p\|_2 \|q\|_2}$$

Cross entropy

$$H(p, q) = - \sum_{i=1}^K p_i \log(q_i)$$

Cập nhật tham số theo gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

2.17 Review Questions – Câu hỏi ôn tập

1. Vì sao mạng nơ-ron ban đầu thường cho output sai?
2. Khởi tạo ngẫu nhiên trọng số có ý nghĩa gì?
3. Lan truyền tiến là gì?
4. Backpropagation là gì?
5. Vì sao cần hàm mất mát?
6. Hàm mất mát phải trả về vector hay số vô hướng?
7. One-hot vector là gì?
8. Với 5 class, one-hot vector có bao nhiêu phần tử?
9. Viết công thức chuẩn L_2 của một vector.
10. Tích vô hướng giữa hai vector cho ra số hay vector?
11. Góc giữa hai vector cho biết điều gì?
12. Cross entropy thường dùng cho loại bài toán nào?
13. Vì sao mô hình có thể trả lời sai khi gặp class chưa từng được học?
14. Vì sao huấn luyện mạng nơ-ron có thể mất nhiều giờ hoặc nhiều ngày?
15. Vì sao chỉ hiểu ý tưởng backpropagation là chưa đủ để lập trình hoàn chỉnh?

2.18 Practice Exercises – Bài tập vận dụng

Bài tập 1: One-hot encoding

Cho 5 class:

{chó, mèo, chim, xe hơi, máy bay}

Hãy viết one-hot vector cho từng class theo đúng thứ tự trên.

Lời giải gợi ý

$$\begin{aligned} \text{chó} &= \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, & \text{mèo} &= \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \\ \text{chim} &= \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, & \text{xe hơi} &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, & \text{máy bay} &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \end{aligned}$$

Bài tập 2: Tính chuẩn L2

Cho:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} 0.8 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Tính:

$$\|y - \hat{y}\|_2$$

Lời giải gợi ý

$$e = y - \hat{y} = \begin{bmatrix} 0.2 \\ -0.1 \\ -0.1 \end{bmatrix}$$

$$\|e\|_2 = \sqrt{0.2^2 + (-0.1)^2 + (-0.1)^2}$$

$$\|e\|_2 = \sqrt{0.04 + 0.01 + 0.01} = \sqrt{0.06} \approx 0.245$$

Bài tập 3: Tính cross entropy

Cho label đúng:

$$p = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Mạng dự đoán:

$$q = \begin{bmatrix} 0.2 \\ 0.7 \\ 0.1 \end{bmatrix}$$

Tính cross entropy:

$$H(p, q) = - \sum_{i=1}^3 p_i \log(q_i)$$

Lời giải gợi ý

Vì class đúng là class thứ 2:

$$H(p, q) = -\log(0.7) \approx 0.357$$

Bài tập 4: Xác định output layer

Một bài toán phân loại ảnh có 12 class. Nếu dùng one-hot encoding, lớp output nên có bao nhiêu nơ-ron?

Lời giải gợi ý

Vì có 12 class, one-hot vector có 12 phần tử. Do đó, lớp output nên có 12 nơ-ron.

$$12 \text{ class} \Rightarrow 12 \text{ output neurons}$$

Kết luận

ANN3 giải thích phần cốt lõi của quá trình huấn luyện mạng nơ-ron. Ban đầu, mạng có các trọng số và bias ngẫu nhiên nên output thường sai. Thông qua lan truyền tiến, mạng tạo dự đoán. Dự đoán này được so sánh với label đúng bằng một hàm mất mát. Sau đó, backpropagation dùng sai số để điều chỉnh các tham số của mạng.

Trong bài toán phân loại nhiều lớp, nhãn thường được mã hóa bằng one-hot vector. Khi đó, output và label đều là vector, nên cần các cách đo sai số giữa vector như chuẩn L_2 , góc giữa hai vector hoặc cross entropy. Trong thực tế, cross entropy là lựa chọn rất phổ biến cho bài toán phân loại.

Điều quan trọng cần nhớ là huấn luyện mạng nơ-ron không chỉ là chạy một thuật toán có sẵn. Muốn mô hình hoạt động tốt, người học cần hiểu dữ liệu, cách mã hóa nhãn, hàm mất mát, thuật toán tối ưu, kiến trúc mạng và nhiều chi tiết kỹ thuật khác.

Chương 3

ANN4: Gradient Descent – Hạ gradient

Learning Objectives – Mục tiêu bài học

Bài ANN4 tiếp tục phần huấn luyện mạng nơ-ron, tập trung vào bản chất tối ưu hóa của quá trình học. Sau khi học xong, người học cần nắm được:

1. Nhắc lại vai trò của hàm mất mát trong huấn luyện mạng nơ-ron.
2. Hiểu vì sao huấn luyện ANN là một bài toán tối ưu.
3. Nắm được ý tưởng của phương pháp Gradient Descent.
4. Biết cách dùng đạo hàm để quyết định hướng cập nhật tham số.
5. Hiểu vai trò của learning rate trong quá trình hội tụ.
6. Nhận biết sự khác nhau giữa bài toán một biến và bài toán nhiều biến.
7. Hiểu khái niệm cực trị địa phương và cực trị toàn cục.
8. Nắm được trực giác về exploration, exploitation, momentum và thuật toán di truyền.
9. Biết khi nào nên dùng các hàm mất mát phổ biến như cross entropy hoặc mean squared error.

3.1 Neural Network Training Recap – Nhắc lại quá trình huấn luyện mạng nơ-ron

3.1.1 Từ dữ liệu đến sai số

Trong học có giám sát, ta cung cấp cho mạng nơ-ron các cặp dữ liệu:

$$(x_i, y_i)$$

Trong đó:

- x_i là dữ liệu đầu vào, còn gọi là *data* hoặc *input*;

- y_i là nhãn đúng, còn gọi là *label* hoặc *target*.

Mạng nơ-ron nhận x_i và tạo ra dự đoán:

$$\hat{y}_i = f(x_i; \theta)$$

Trong đó θ là toàn bộ tham số của mạng, gồm các trọng số và bias.

Sai số xuất hiện khi dự đoán $\hat{y}_i$ khác với nhãn đúng y_i .

Sai số = khác biệt giữa y_i và $\hat{y}_i$

Ghi nhớ

Huấn luyện mạng nơ-ron là quá trình dùng sai số giữa output mong muốn và output hiện tại để điều chỉnh các tham số của mạng.

3.1.2 Hàm mất mát

Hàm mất mát, tiếng Anh là *loss function*, là hàm biến sự khác biệt giữa y và $\hat{y}$ thành một số vô hướng.

$$\mathcal{L}(y, \hat{y}) \in \mathbb{R}$$

Giá trị loss càng nhỏ thì mô hình dự đoán càng tốt.

Một số dạng hàm mất mát phổ biến:

- chuẩn L_2 hoặc bình phương sai số;
- góc giữa hai vector;
- cross entropy.

3.1.3 Mục tiêu lý tưởng

Trong trường hợp lý tưởng, nếu mô hình dự đoán hoàn hảo, sai số sẽ tiến về 0.

$$\mathcal{L}(y, \hat{y}) = 0$$

Tuy nhiên, trong thực tế, dữ liệu có thể nhiễu, mô hình có thể chưa đủ tốt, quá trình tối ưu có thể chưa đạt cực trị tốt nhất. Do đó, ta thường chỉ kỳ vọng loss giảm dần đến một mức đủ nhỏ.

Ví dụ minh họa

Giả sử bài toán phân loại ảnh chó và mèo.

Nếu ảnh thật là chó:

$$y = \text{chó}$$

Mạng dự đoán đúng:

$$\hat{y} = \text{chó}$$

thì sai số rất nhỏ.

Nếu mạng dự đoán:

$$\hat{y} = \text{mèo}$$

thì sai số lớn hơn. Hàm mất mát giúp lượng hóa mức sai này thành một con số cụ thể.

3.2 ANN Training as Optimization – Huấn luyện ANN là một bài toán tối ưu

3.2.1 Bản chất của quá trình huấn luyện

Trong mạng nơ-ron, ta cần tìm bộ tham số:

$$\theta = \{W, b\}$$

sao cho hàm mất mát trên tập dữ liệu là nhỏ nhất.

Với một mẫu dữ liệu:

$$\mathcal{L}(y_i, \hat{y}_i)$$

Với toàn bộ tập huấn luyện gồm N mẫu, ta thường xét hàm mục tiêu:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f(x_i; \theta))$$

Mục tiêu huấn luyện là:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

Trong đó:

- $J(\theta)$ là hàm mục tiêu cần tối thiểu hóa;
- θ là bộ tham số cần điều chỉnh;
- θ^* là bộ tham số tối ưu hoặc gần tối ưu.

Ghi nhớ

Bản chất toán học của huấn luyện mạng nơ-ron là bài toán tối ưu hóa: tìm các trọng số và bias để hàm mất mát đạt giá trị nhỏ nhất có thể.

3.2.2 Số lượng biến rất lớn

Trong bài toán tối ưu thông thường, ta có thể chỉ cần tìm một biến x . Nhưng trong mạng nơ-ron, số biến chính là số tham số của mạng.

Một mạng nơ-ron có thể có:

- vài nghìn tham số;
- vài triệu tham số;
- vài chục triệu tham số;
- thậm chí vài trăm triệu hoặc nhiều hơn.

Vì vậy, bài toán tối ưu trong ANN thường là bài toán đa biến quy mô lớn.

3.2.3 Không phải lúc nào cũng có lời giải giải tích

Với một số hàm đơn giản, ví dụ hàm bậc hai, ta có thể tìm cực trị bằng cách giải phương trình đạo hàm bằng 0.

Ví dụ:

$$f(x) = x^2 - 4x + 5$$

Đạo hàm:

$$f'(x) = 2x - 4$$

Cho:

$$f'(x) = 0$$

Ta được:

$$2x - 4 = 0 \Rightarrow x = 2$$

Đây là lời giải giải tích.

Tuy nhiên, trong mạng nơ-ron, hàm mất mát thường rất phức tạp vì phụ thuộc vào nhiều lớp, nhiều hàm kích hoạt, nhiều tham số và dữ liệu lớn. Do đó, ta thường không có lời giải giải tích trực tiếp, mà phải dùng phương pháp tính lặp.

Ghi nhớ

Trong ANN, ta thường không giải trực tiếp được θ^* bằng công thức đóng. Thay vào đó, ta bắt đầu từ một bộ tham số ban đầu và cập nhật lặp để giảm loss.

3.3 Gradient Descent: Single Variable – Gradient Descent trong trường hợp một biến

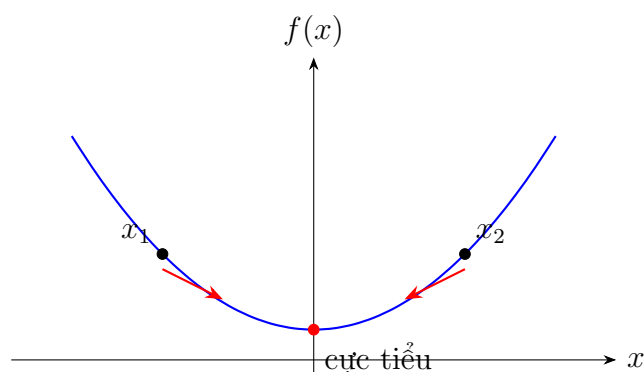
3.3.1 Bài toán minh họa

Giả sử ta cần tìm giá trị x sao cho hàm:

$$f(x)$$

đạt cực tiểu.

Ta có thể hình dung đồ thị của $f(x)$ như một thung lũng. Mục tiêu là tìm điểm thấp nhất của thung lũng.



Nếu ta bắt đầu ở bên trái điểm cực tiểu, ta cần tăng x để đi về phía cực tiểu.

Nếu ta bắt đầu ở bên phải điểm cực tiểu, ta cần giảm x để đi về phía cực tiểu.

Câu hỏi quan trọng là: làm sao biết nên tăng hay giảm x ?

Câu trả lời là: dùng đạo hàm.

3.3.2 Đạo hàm cho biết hướng đi

Đạo hàm $f'(x)$ cho biết độ dốc của hàm tại vị trí hiện tại.

- Nếu $f'(x) < 0$, hàm đang giảm khi x tăng. Muốn giảm $f(x)$, ta nên tăng x .
- Nếu $f'(x) > 0$, hàm đang tăng khi x tăng. Muốn giảm $f(x)$, ta nên giảm x .

Công thức cập nhật của Gradient Descent trong trường hợp một biến là:

$$x_{\text{new}} = x_{\text{old}} - \eta f'(x_{\text{old}})$$

Trong đó:

- x_{old} là giá trị hiện tại;
- x_{new} là giá trị sau khi cập nhật;
- $f'(x_{\text{old}})$ là đạo hàm tại điểm hiện tại;
- η là learning rate.

Ghi nhớ

Dấu trừ trong công thức Gradient Descent thể hiện việc đi ngược hướng tăng của hàm để làm hàm giảm xuống.

3.3.3 Ví dụ trực quan

Giả sử:

$$f(x) = (x - 3)^2$$

Hàm này đạt cực tiểu tại:

$$x = 3$$

Đạo hàm:

$$f'(x) = 2(x - 3)$$

Chọn learning rate:

$$\eta = 0.1$$

Giả sử bắt đầu từ:

$$x_0 = 0$$

Ta cập nhật:

$$x_1 = x_0 - \eta f'(x_0)$$

$$f'(0) = 2(0 - 3) = -6$$

$$x_1 = 0 - 0.1(-6) = 0.6$$

Tiếp tục:

$$f'(0.6) = 2(0.6 - 3) = -4.8$$

$$x_2 = 0.6 - 0.1(-4.8) = 1.08$$

Tiếp tục:

$$f'(1.08) = 2(1.08 - 3) = -3.84$$

$$x_3 = 1.08 - 0.1(-3.84) = 1.464$$

Ta thấy x đang tăng dần và tiến về 3.

Ví dụ minh họa

Bảng vài bước cập nhật:

Bước	x	$f'(x)$	$f(x)$
0	0.000	-6.000	9.000
1	0.600	-4.800	5.760
2	1.080	-3.840	3.686
3	1.464	-3.072	2.360
4	1.771	-2.458	1.511

Giá trị $f(x)$ giảm dần, tức thuật toán đang tiến về cực tiểu.

3.4 Learning Rate – Learning rate

3.4.1 Learning rate là gì?

Learning rate, thường ký hiệu là η , là hệ số quyết định mỗi lần cập nhật tham số sẽ đi một bước lớn hay nhỏ.

Trong công thức:

$$x_{\text{new}} = x_{\text{old}} - \eta f'(x_{\text{old}})$$

đại lượng:

$$\eta f'(x_{\text{old}})$$

quyết định độ lớn của bước cập nhật.

3.4.2 Learning rate quá nhỏ

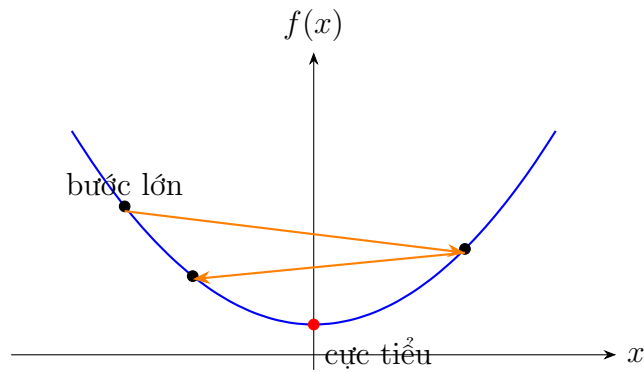
Nếu η quá nhỏ, mỗi lần cập nhật chỉ thay đổi rất ít. Thuật toán vẫn có thể đi đúng hướng nhưng sẽ hội tụ rất chậm.

Ví dụ minh họa

Nếu điểm hiện tại còn rất xa cực tiểu nhưng learning rate quá nhỏ, thuật toán giống như đi từng bước rất ngắn. Kết quả là cần rất nhiều vòng lặp mới đến gần nghiệm.

3.4.3 Learning rate quá lớn

Nếu η quá lớn, bước cập nhật quá dài. Thuật toán có thể nhảy qua điểm cực tiểu, dao động qua lại hoặc thậm chí không hội tụ.



Cảnh báo

Learning rate quá lớn có thể làm quá trình học không ổn định. Loss có thể không giảm mà dao động hoặc tăng lên.

3.4.4 Learning rate thay đổi theo thời gian

Trong nhiều thuật toán, learning rate không cố định. Một chiến lược thường gặp là:

- ban đầu dùng learning rate lớn để đi nhanh;
- về sau giảm learning rate để tinh chỉnh gần điểm cực tiểu.

Ta có thể ký hiệu learning rate tại bước t là:

$$\eta_t$$

và công thức cập nhật:

$$x_{t+1} = x_t - \eta_t f'(x_t)$$

Trong đó η_t có thể giảm dần theo thời gian.

Ghi nhớ

Chọn learning rate là một trong những quyết định quan trọng nhất khi huấn luyện mạng nơ-ron.

3.5 Gradient Descent trong bài toán nhiều biến

3.5.1 Từ đạo hàm sang gradient

Trong mạng nơ-ron, ta không chỉ có một biến x , mà có rất nhiều tham số:

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_m \end{bmatrix}$$

Hàm mục tiêu:

$$J(\theta)$$

là hàm nhiều biến. Khi đó, ta không dùng đạo hàm thông thường mà dùng gradient:

$$\nabla_{\theta} J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \\ \vdots \\ \frac{\partial J}{\partial \theta_m} \end{bmatrix}$$

Gradient là vector gồm các đạo hàm riêng của hàm mất mát theo từng tham số.

3.5.2 Công thức cập nhật tổng quát

Công thức Gradient Descent trong bài toán nhiều biến:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

Nếu xét riêng một trọng số w , ta có:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

Nếu xét bias b :

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial J}{\partial b}$$

Ghi nhớ

Trong ANN, mỗi trọng số và mỗi bias đều được cập nhật dựa trên đạo hàm riêng của hàm mất mát theo chính tham số đó.

3.5.3 Liên hệ với backpropagation

Backpropagation giúp tính các đạo hàm riêng:

$$\frac{\partial J}{\partial w} \quad \text{và} \quad \frac{\partial J}{\partial b}$$

cho toàn bộ mạng một cách hiệu quả. Sau khi có các đạo hàm này, ta dùng thuật toán tối ưu như Gradient Descent để cập nhật tham số.

Tóm lại:

Backpropagation $\Rightarrow$ tính gradient

Gradient Descent $\Rightarrow$ cập nhật tham số

3.5.4 Ví dụ cập nhật một trọng số

Giả sử tại một bước huấn luyện:

$$w = 0.8$$

Gradient theo trọng số này là:

$$\frac{\partial J}{\partial w} = 0.5$$

Chọn learning rate:

$$\eta = 0.1$$

Cập nhật:

$$w_{\text{new}} = 0.8 - 0.1 \cdot 0.5 = 0.8 - 0.05 = 0.75$$

Nếu gradient âm:

$$\frac{\partial J}{\partial w} = -0.5$$

thì:

$$w_{\text{new}} = 0.8 - 0.1(-0.5) = 0.85$$

Như vậy, dấu của gradient quyết định tham số tăng hay giảm.

3.6 Cực trị địa phương và cực trị toàn cục

3.6.1 Cực trị toàn cục

Cực trị toàn cục, tiếng Anh là *global minimum*, là điểm có giá trị hàm mất mát nhỏ nhất trên toàn bộ không gian tìm kiếm.

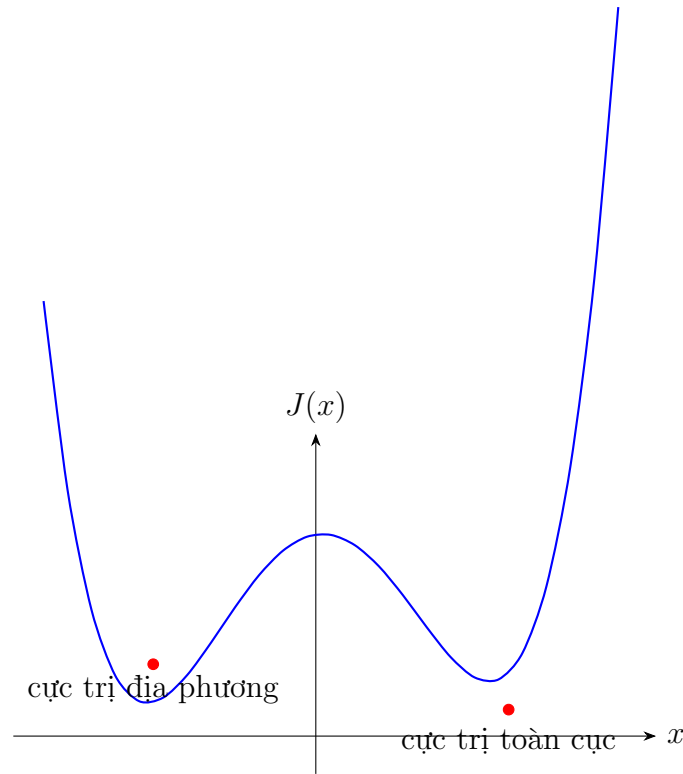
Nếu θ^* là cực tiểu toàn cục thì:

$$J(\theta^*) \leq J(\theta)$$

với mọi θ trong miền xét.

3.6.2 Cực trị địa phương

Cực trị địa phương, tiếng Anh là *local minimum*, là điểm thấp nhất trong một vùng lân cận, nhưng chưa chắc là điểm thấp nhất toàn cục.



3.6.3 Vấn đề mắc kẹt ở cực trị địa phương

Gradient Descent dựa vào thông tin cục bộ tại điểm hiện tại. Nếu thuật toán đi xuống một “hố” địa phương, nó có thể bị mắc kẹt ở đó và không tìm được cực trị toàn cục.

Lưu ý

Một biến cũng có thể có nhiều cực trị. Không phải chỉ bài toán nhiều biến mới có nhiều cực trị. Tuy nhiên, trong bài toán nhiều biến, bề mặt mất mát phức tạp hơn rất nhiều.

3.6.4 Ảnh hưởng trong huấn luyện ANN

Trong mạng nơ-ron, hàm mất mát theo hàng triệu tham số tạo thành một bề mặt rất phức tạp. Bề mặt này có thể có:

- nhiều cực trị địa phương;
- vùng yên ngựa;
- vùng phẳng;
- dốc rất lớn hoặc rất nhỏ.

Do đó, việc tối ưu không hề đơn giản. Không phải cứ chạy huấn luyện là chắc chắn tìm được nghiệm tốt nhất.

3.7 Exploration và Exploitation

3.7.1 Exploitation là gì?

Exploitation có thể hiểu là khai thác thông tin hiện có ở khu vực hiện tại để cải thiện lời giải.

Trong Gradient Descent, tại một điểm hiện tại, ta dùng gradient ở điểm đó để quyết định hướng đi tiếp theo. Đây là dạng khai thác thông tin cục bộ.

Thông tin cục bộ $\Rightarrow$ đi theo hướng giảm loss

3.7.2 Exploration là gì?

Exploration là thăm dò các khu vực khác của không gian tìm kiếm. Mục tiêu là tránh việc chỉ tập trung vào một vùng hiện tại và bỏ lỡ các nghiệm tốt hơn ở vùng khác.

Ví dụ minh họa

Có thể hình dung như việc tìm vàng:

- Exploitation: tập trung khai thác mỏ vàng hiện tại.
- Exploration: cử một số người đi tìm các mỏ vàng khác.

Nếu chỉ khai thác mỏ hiện tại, ta có thể bỏ lỡ một mỏ vàng lớn hơn ở nơi khác.

3.7.3 Cân bằng exploration và exploitation

Một thuật toán tối ưu tốt thường cần cân bằng:

khai thác lời giải hiện tại và tìm kiếm vùng mới

Nếu quá thiên về exploitation, thuật toán dễ mắc kẹt tại cực trị địa phương.

Nếu quá thiên về exploration, thuật toán có thể tìm kiếm lan man và hội tụ chậm.

Ghi nhớ

Trong tối ưu hóa, một trong những khó khăn lớn là cân bằng giữa việc khai thác nghiệm hiện tại và khám phá các vùng nghiệm mới.

3.8 Thuật toán di truyền và metaheuristic

3.8.1 Metaheuristic là gì?

Metaheuristic là nhóm thuật toán tối ưu cấp cao, thường được thiết kế để tìm nghiệm tốt trong những bài toán khó, đặc biệt khi:

- không có lời giải giải tích;
- không có đạo hàm;

- không gian tìm kiếm quá lớn;
- hàm mục tiêu có nhiều cực trị địa phương.

Các thuật toán metaheuristic thường không đảm bảo tìm được nghiệm tối ưu toàn cục, nhưng có thể tìm được nghiệm tốt trong thời gian chấp nhận được.

3.8.2 Thuật toán di truyền

Thuật toán di truyền, tiếng Anh là *genetic algorithm*, là một ví dụ của metaheuristic. Nó lấy cảm hứng từ tiến hóa tự nhiên.

Một quần thể gồm nhiều cá thể được tạo ra. Mỗi cá thể đại diện cho một lời giải. Qua nhiều thế hệ, các lời giải tốt được chọn, lai ghép và đột biến để tạo lời giải mới.

Các bước khái quát:

1. Khởi tạo một quần thể lời giải.
2. Đánh giá độ tốt của từng lời giải bằng hàm mục tiêu.
3. Chọn các lời giải tốt hơn.
4. Lai ghép và đột biến để tạo thế hệ mới.
5. Lặp lại cho đến khi đạt điều kiện dừng.

3.8.3 Liên hệ với ANN

Về nguyên tắc, thuật toán di truyền có thể được dùng để tối ưu trọng số của mạng nơ-ron, nhất là ở quy mô nhỏ.

Tuy nhiên, với mạng lớn có hàng triệu hoặc hàng trăm triệu tham số, thuật toán di truyền thường không hiệu quả bằng các phương pháp dựa trên gradient.

Lưu ý

Trong huấn luyện mạng nơ-ron hiện đại, các phương pháp dựa trên gradient vẫn là nền tảng chính. Metaheuristic có thể hữu ích trong một số bài toán đặc biệt hoặc quy mô nhỏ.

3.9 Momentum và ý tưởng vượt cực trị địa - local minimum

3.9.1 Hạn chế của Gradient Descent cơ bản

Gradient Descent cơ bản chỉ dựa vào gradient hiện tại. Vì vậy, khi gặp một cực trị địa phương hoặc vùng trũng nhỏ, thuật toán có thể bị mắc kẹt.

Ta có thể hình dung một hòn bi lăn xuống dốc. Nếu hòn bi không có quán tính, nó có thể dừng lại ở một hố nhỏ. Nhưng nếu có đủ động lượng, nó có thể vượt qua hố nhỏ và tiếp tục lăn xuống vùng thấp hơn.

3.9.2 Ý tưởng momentum

Momentum bổ sung yếu tố quán tính vào quá trình cập nhật. Thay vì chỉ dùng gradient hiện tại, thuật toán còn xét hướng di chuyển trước đó.

Một dạng công thức đơn giản:

$$v_t = \beta v_{t-1} + \eta \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_t$$

Trong đó:

- v_t là vận tốc cập nhật tại bước t ;
- β là hệ số momentum;
- η là learning rate;
- $\nabla_{\theta} J(\theta_t)$ là gradient hiện tại.

Ghi nhớ

Momentum giúp thuật toán có xu hướng duy trì hướng đi, nhờ đó có thể giảm dao động và đôi khi vượt qua các vùng cực trị địa phương nông.

3.9.3 Không đảm bảo tìm được cực trị toàn cục

Momentum và các kỹ thuật tối ưu hiện đại có thể giúp quá trình huấn luyện tốt hơn, nhưng không đảm bảo luôn tìm được cực trị toàn cục.

Kết quả còn phụ thuộc vào:

- hình dạng hàm mất mát;
- cách khởi tạo tham số;
- learning rate;
- kiến trúc mạng;
- dữ liệu;
- thuật toán tối ưu.

Cảnh báo

Không nên hiểu rằng dùng thuật toán tối ưu hiện đại là chắc chắn tìm được nghiệm tốt nhất. Trong thực tế, ta thường tìm nghiệm đủ tốt thay vì nghiệm tối ưu tuyệt đối.

3.10 Lựa chọn hàm mất mát

3.10.1 Các hàm mất mát phổ biến

Trong các bài ANN trước, ta đã gặp một số cách đo sai số:

1. Sai số theo chuẩn L_2 hoặc bình phương sai số.
2. Sai số dựa trên góc giữa hai vector.
3. Cross entropy.

Mỗi hàm mất mát có đặc điểm riêng và phù hợp với từng loại bài toán.

3.10.2 Mean Squared Error

Mean Squared Error, viết tắt là MSE, là sai số bình phương trung bình:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Với output dạng vector:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \|y_i - \hat{y}_i\|_2^2$$

MSE thường được dùng trong:

- bài toán hồi quy;
- dự đoán giá trị liên tục;
- một số bài toán thị giác máy tính đặc thù.

Ví dụ minh họa

Dự đoán giá cổ phiếu:

$$y = 105, \quad \hat{y} = 102$$

Sai số bình phương:

$$(105 - 102)^2 = 9$$

3.10.3 Cross entropy

Cross entropy thường được dùng trong bài toán phân loại.

Với K class:

$$H(p, q) = - \sum_{i=1}^K p_i \log(q_i)$$

Trong đó:

- p_i là phân phối đúng;
- q_i là phân phối dự đoán;
- K là số class.

Nếu p là one-hot vector và class đúng là c , công thức rút gọn:

$$H(p, q) = -\log(q_c)$$

Ví dụ minh họa

Ảnh thật là chó. Mạng dự đoán xác suất cho class chó là:

$$q_c = 0.9$$

Cross entropy:

$$-\log(0.9) \approx 0.105$$

Nếu mạng chỉ dự đoán xác suất chó là:

$$q_c = 0.1$$

Cross entropy:

$$-\log(0.1) \approx 2.303$$

Dự đoán càng tự tin đúng thì loss càng nhỏ. Dự đoán class đúng với xác suất thấp thì loss lớn.

3.10.4 Vì sao cross entropy thường tốt cho phân loại?

Trong bài toán phân loại, ta thường muốn mô hình gán xác suất cao cho class đúng. Cross entropy phạt mạnh khi xác suất của class đúng thấp.

Ví dụ, nếu label đúng là:

$$p = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

và mô hình dự đoán:

$$q = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

thì mô hình còn phân vân. Nếu mô hình dự đoán:

$$q = \begin{bmatrix} 0.99 \\ 0.01 \end{bmatrix}$$

thì mô hình rất chắc vào class đúng và loss sẽ nhỏ hơn nhiều.

Ghi nhớ

Cross entropy có xu hướng khuyến khích mô hình tiến gần hơn đến phân phối nhãn đúng, đặc biệt trong bài toán phân loại.

3.10.5 Không có hàm mất mát tốt nhất cho mọi bài toán

Dù cross entropy rất phổ biến trong phân loại, điều đó không có nghĩa là mọi nghiên cứu hoặc mọi mô hình đều dùng cross entropy.

Một số bài toán vẫn dùng MSE hoặc các biến thể của MSE. Ví dụ, trong các hệ thống phát hiện vật thể như YOLO, hàm mất mát thường là tổ hợp của nhiều thành phần, trong đó có các thành phần liên quan đến sai số bình phương cho tọa độ hộp bao.

Lưu ý

Chọn hàm mất mát phải dựa vào bản chất bài toán. Bài toán phân loại, hồi quy, phát hiện vật thể, phân đoạn ảnh hoặc sinh dữ liệu có thể cần các hàm loss khác nhau.

3.11 So sánh các khái niệm quan trọng

3.11.1 Backpropagation và Gradient Descent

Khái niệm	Vai trò	Trả lời câu hỏi
Backpropagation	Tính gradient của loss theo các tham số	Tham số nào ảnh hưởng đến loss như thế nào?
Gradient Descent	Dùng gradient để cập nhật tham số	Cần tăng hay giảm tham số bao nhiêu?

3.11.2 Local minimum và global minimum

Khái niệm	Ý nghĩa
Cực trị địa phương	Tốt nhất trong một vùng lân cận, nhưng có thể không phải tốt nhất toàn cục.
Cực trị toàn cục	Tốt nhất trên toàn bộ không gian tìm kiếm.

3.11.3 Exploration và exploitation

Khái niệm	Ý nghĩa
Exploitation	Khai thác thông tin hiện có để cải thiện nghiệm hiện tại.
Exploration	Tìm kiếm ở các vùng mới để tránh bỏ lỡ nghiệm tốt hơn.

3.12 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
Loss function	Hàm mất mát, đo sai khác giữa output dự đoán và label đúng.	Cross entropy, MSE.
Error	Sai số giữa kết quả mong muốn và kết quả hiện tại.	$e = y - \hat{y}$.
Optimization	Tối ưu hóa, quá trình tìm tham số để hàm mục tiêu đạt giá trị tốt nhất.	Tối thiểu hóa loss.
Objective function	Hàm mục tiêu cần tối ưu.	$J(\theta)$.
Parameter	Tham số mô hình được điều chỉnh trong huấn luyện.	Trọng số, bias.
Gradient Descent	Phương pháp suy giảm theo hướng ngược gradient để giảm hàm mục tiêu.	$x_{t+1} = x_t - \eta f'(x_t)$.
Derivative	Đạo hàm, cho biết độ dốc của hàm một biến.	$f'(x)$.
Gradient	Vector đạo hàm riêng của hàm nhiều biến.	$\nabla_{\theta} J$.
Partial derivative	Đạo hàm riêng theo một biến khi các biến khác xem như cố định.	$\frac{\partial J}{\partial w}$.
Learning rate	Hệ số quyết định độ lớn bước cập nhật.	$\eta = 0.001$.
Convergence	Hội tụ, trạng thái thuật toán tiến dần đến nghiệm ổn định.	Loss giảm dần.
Divergence	Phân kỳ, trạng thái thuật toán không tiến đến nghiệm ổn định.	Loss tăng hoặc dao động.
Local minimum	Cực tiểu địa phương, điểm thấp nhất trong một vùng lân cận.	Hố nhỏ trên đồ thị loss.
Global minimum	Cực tiểu toàn cục, điểm thấp nhất trên toàn miền.	Loss nhỏ nhất có thể.
Exploitation	Khai thác thông tin cục bộ hiện có.	Đi theo gradient hiện tại.
Exploration	Khám phá vùng tìm kiếm mới.	Tìm khu vực có nghiệm tốt hơn.
Metaheuristic	Nhóm thuật toán tối ưu tổng quát, thường dùng cho bài toán khó.	Thuật toán di truyền.
Genetic Algorithm	Thuật toán di truyền, mô phỏng chọn lọc tự nhiên để tìm nghiệm tốt.	Quần thể lời giải.
Momentum	Kỹ thuật thêm quán tính vào cập nhật gradient.	Giúp giảm dao động.
MSE	Mean Squared Error, sai số bình phương trung bình.	Dự đoán giá trị liên tục.
Cross entropy	Hàm mất mát phổ biến cho phân loại xác suất.	$-\sum p_i \log q_i$.
YOLO	Hệ thống phát hiện vật thể, thường dùng hàm loss gồm nhiều thành phần.	Nhận diện vật thể trong ảnh.

3.13 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

3.13.1 Các ý chính cần nhớ

1. Huấn luyện mạng nơ-ron là bài toán tối ưu hóa hàm mất mát.
2. Các tham số cần tối ưu là toàn bộ trọng số và bias trong mạng.
3. Gradient Descent cập nhật tham số theo hướng làm giảm loss.
4. Trong bài toán một biến, đạo hàm cho biết nên tăng hay giảm biến.
5. Trong bài toán nhiều biến, gradient thay thế vai trò của đạo hàm.
6. Learning rate quyết định độ lớn của mỗi bước cập nhật.
7. Learning rate quá nhỏ làm học chậm; quá lớn có thể gây dao động hoặc không hội tụ.
8. Hàm mất mát của ANN thường rất phức tạp và không có lời giải giải tích đơn giản.
9. Gradient Descent có thể mắc kẹt ở cực trị địa phương.
10. Momentum và các kỹ thuật tối ưu hiện đại giúp cải thiện khả năng hội tụ nhưng không đảm bảo tìm cực trị toàn cục.
11. Cross entropy phổ biến trong bài toán phân loại, nhưng không phải là lựa chọn duy nhất.
12. Chọn hàm loss phải dựa vào bản chất bài toán cụ thể.

3.13.2 Công thức quan trọng

Hàm mục tiêu trên tập dữ liệu

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f(x_i; \theta))$$

Bài toán tối ưu

$$\theta^* = \arg \min_{\theta} J(\theta)$$

Gradient Descent một biến

$$x_{t+1} = x_t - \eta f'(x_t)$$

Gradient Descent nhiều biến

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

Cập nhật một trọng số

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

MSE

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Cross entropy

$$H(p, q) = - \sum_{i=1}^K p_i \log(q_i)$$

3.14 Review Questions – Câu hỏi ôn tập

1. Vì sao huấn luyện mạng nơ-ron được xem là một bài toán tối ưu?
2. Trong ANN, các biến cần tối ưu là gì?
3. Hàm mất mát có vai trò gì trong quá trình huấn luyện?
4. Trong trường hợp lý tưởng, loss nên tiến về giá trị nào?
5. Gradient Descent dùng thông tin gì để cập nhật tham số?
6. Vì sao công thức Gradient Descent có dấu trừ?
7. Learning rate ảnh hưởng như thế nào đến tốc độ hội tụ?
8. Điều gì xảy ra nếu learning rate quá nhỏ?
9. Điều gì xảy ra nếu learning rate quá lớn?
10. Gradient khác đạo hàm thông thường như thế nào?
11. Cực trị địa phương khác gì cực trị toàn cục?
12. Vì sao Gradient Descent có thể mắc kẹt ở cực trị địa phương?
13. Exploration và exploitation khác nhau như thế nào?
14. Thuật toán di truyền thuộc nhóm thuật toán nào?

15. Momentum giúp ích gì trong tối ưu hóa?
16. Cross entropy thường phù hợp với bài toán nào?
17. MSE thường phù hợp với bài toán nào?
18. Vì sao không có một hàm mất mát tốt nhất cho mọi bài toán?

3.15 Practice Exercises – Bài tập vận dụng

Bài tập 1: Gradient Descent một biến

Cho:

$$f(x) = (x - 4)^2$$

$$x_0 = 0, \quad \eta = 0.1$$

1. Tính đạo hàm $f'(x)$.
2. Tính x_1 theo công thức Gradient Descent.
3. Tính x_2 .

Lời giải gợi ý

Đạo hàm:

$$f'(x) = 2(x - 4)$$

Bước 1:

$$f'(0) = 2(0 - 4) = -8$$

$$x_1 = 0 - 0.1(-8) = 0.8$$

Bước 2:

$$f'(0.8) = 2(0.8 - 4) = -6.4$$

$$x_2 = 0.8 - 0.1(-6.4) = 1.44$$

Bài tập 2: Cập nhật trọng số

Cho một trọng số:

$$w = 1.2$$

Gradient:

$$\frac{\partial J}{\partial w} = 0.3$$

Learning rate:

$$\eta = 0.05$$

Tính trọng số mới.

Lời giải gợi ý

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

$$w_{\text{new}} = 1.2 - 0.05 \cdot 0.3 = 1.2 - 0.015 = 1.185$$

Bài tập 3: So sánh learning rate

Giải thích ngắn gọn hiện tượng xảy ra trong hai trường hợp:

1. Learning rate rất nhỏ.
2. Learning rate rất lớn.

Lời giải gợi ý

1. Learning rate rất nhỏ làm mỗi bước cập nhật rất nhỏ, thuật toán học chậm và cần nhiều vòng lặp.
2. Learning rate rất lớn làm bước cập nhật quá dài, thuật toán có thể vượt qua cực tiểu, dao động hoặc không hội tụ.

Bài tập 4: Tính cross entropy

Một bài toán phân loại có 3 lớp. Label đúng là lớp thứ nhất:

$$p = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Mạng dự đoán:

$$q = \begin{bmatrix} 0.8 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Tính cross entropy.

Lời giải gợi ý

Vì class đúng là class thứ nhất:

$$H(p, q) = -\log(0.8)$$

$$H(p, q) \approx 0.223$$

Bài tập 5: Nhận diện loại bài toán

Với mỗi bài toán sau, hãy đề xuất hàm mất mát phù hợp hơn: MSE hoặc cross entropy.

1. Dự đoán giá nhà.
2. Phân loại ảnh chó, mèo, chim.
3. Dự đoán nhiệt độ ngày mai.
4. Phân loại email spam hoặc không spam.

Lời giải gợi ý

Bài toán	Loại bài toán	Hàm mất mát gợi ý
Dự đoán giá nhà	Hồi quy	MSE
Phân loại ảnh	Phân loại nhiều lớp	Cross entropy
Dự đoán nhiệt độ	Hồi quy	MSE
Phân loại email	Phân loại nhị phân	Binary cross entropy

Kết luận

ANN4 làm rõ bản chất tối ưu hóa của quá trình huấn luyện mạng nơ-ron. Khi mạng tạo ra output khác với label đúng, hàm mất mát được dùng để đo sai số. Mục tiêu của huấn luyện là điều chỉnh toàn bộ trọng số và bias sao cho hàm mất mát giảm dần.

Gradient Descent là phương pháp cơ bản để cập nhật tham số theo hướng làm giảm loss. Trong trường hợp một biến, ta dùng đạo hàm; trong trường hợp nhiều biến, ta dùng gradient. Learning rate quyết định độ lớn bước cập nhật và ảnh hưởng mạnh đến quá trình hội tụ.

Bên cạnh đó, bài học cũng giới thiệu các khó khăn trong tối ưu hóa như cực trị địa phương, sự cần thiết của exploration và exploitation, cũng như ý tưởng của momentum và thuật toán di truyền. Cuối cùng, việc lựa chọn hàm mất mát phải gắn với bản chất bài toán: cross entropy thường dùng cho phân loại, còn MSE thường dùng cho hồi quy hoặc một số bài toán đặc thù.

Chương 4

ANN6: Backpropagation Example – Ví dụ lan truyền ngược

Learning Objectives – Mục tiêu của tài liệu

Tài liệu này chỉ tập trung giải thích ví dụ tính toán trong video ANN6. Nội dung không trình bày lại lý thuyết tổng quát, mà đi theo đúng tinh thần của ví dụ:

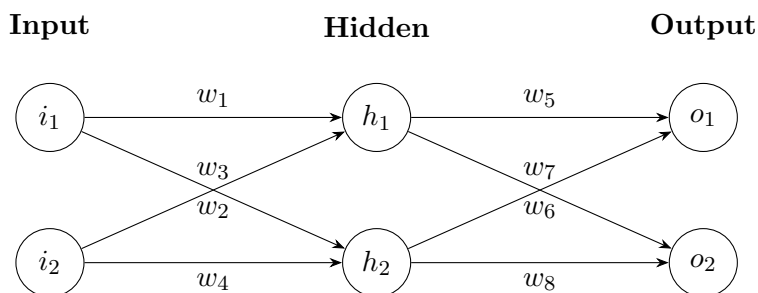
1. Tính xuôi để lấy output và sai số tổng.
2. Tính ngược để cập nhật các trọng số từ lớp output: w_5, w_6, w_7, w_8 .
3. Tính ngược tiếp để cập nhật các trọng số phía trước: w_1, w_2, w_3, w_4 .
4. Làm rõ điểm dễ sai nhất: khi tính đạo hàm theo w_1 , output của hidden neuron h_1 ảnh hưởng đồng thời đến cả o_1 và o_2 , nên phải cộng cả hai nhánh.

4.1 Example Description – Mô tả ví dụ

4.1.1 Cấu trúc mạng trong ví dụ

Ví dụ sử dụng một mạng nơ-ron rất nhỏ:

- 2 input: i_1, i_2 ;
- 2 neuron ẩn: h_1, h_2 ;
- 2 output: o_1, o_2 .



4.1.2 Dữ liệu ban đầu

Trong ví dụ, ta dùng các giá trị sau:

Dại lượng	Giá trị	Ý nghĩa
i_1	0.05	Input thứ nhất
i_2	0.10	Input thứ hai
t_1	0.01	Target mong muốn của output o_1
t_2	0.99	Target mong muốn của output o_2
b_h	0.35	Bias cho lớp hidden
b_o	0.60	Bias cho lớp output
η	0.5	Learning rate

Các trọng số ban đầu:

w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8
0.15	0.20	0.25	0.30	0.40	0.45	0.50	0.55

Hàm kích hoạt được dùng trong ví dụ là sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Đạo hàm sigmoid tại output của nó:

$$\sigma'(x) = \sigma(x) (1 - \sigma(x))$$

Trong khi tính, nếu đã biết output $a = \sigma(x)$, ta dùng nhanh:

$$\frac{da}{dx} = a(1 - a)$$

4.2 Forward Pass Example – Tính xuôi trong ví dụ

4.2.1 Tính output của hidden neuron h_1

Net input vào h_1 :

$$net_{h_1} = i_1 w_1 + i_2 w_2 + b_h$$

Thay số:

$$net_{h_1} = 0.05 \cdot 0.15 + 0.10 \cdot 0.20 + 0.35$$

$$net_{h_1} = 0.3775$$

Output của h_1 :

$$out_{h_1} = \sigma(0.3775)$$

$$out_{h_1} \approx 0.593269992$$

4.2.2 Tính output của hidden neuron h_2

Net input vào h_2 :

$$net_{h_2} = i_1 w_3 + i_2 w_4 + b_h$$

Thay số:

$$net_{h_2} = 0.05 \cdot 0.25 + 0.10 \cdot 0.30 + 0.35$$

$$net_{h_2} = 0.3925$$

Output của h_2 :

$$out_{h_2} = \sigma(0.3925)$$

$$out_{h_2} \approx 0.596884378$$

4.2.3 Tính output o_1

Net input vào o_1 :

$$net_{o_1} = out_{h_1} w_5 + out_{h_2} w_6 + b_o$$

Thay số:

$$net_{o_1} = 0.593269992 \cdot 0.40 + 0.596884378 \cdot 0.45 + 0.60$$

$$net_{o_1} \approx 1.105905967$$

Output:

$$out_{o_1} = \sigma(1.105905967)$$

$$out_{o_1} \approx 0.751365070$$

4.2.4 Tính output o_2

Net input vào o_2 :

$$net_{o_2} = out_{h_1} w_7 + out_{h_2} w_8 + b_o$$

Thay số:

$$net_{o_2} = 0.593269992 \cdot 0.50 + 0.596884378 \cdot 0.55 + 0.60$$

$$net_{o_2} \approx 1.224921404$$

Output:

$$out_{o_2} = \sigma(1.224921404)$$

$$out_{o_2} \approx 0.772928465$$

4.3 Error Calculation Example – Tính sai số của ví dụ

4.3.1 Sai số tại output o_1

Target của o_1 là:

$$t_1 = 0.01$$

Output hiện tại:

$$out_{o_1} = 0.751365070$$

Sai số tại o_1 :

$$E_{o_1} = \frac{1}{2}(t_1 - out_{o_1})^2$$

Thay số:

$$E_{o_1} = \frac{1}{2}(0.01 - 0.751365070)^2$$

$$E_{o_1} \approx 0.274811083$$

4.3.2 Sai số tại output o_2

Target của o_2 là:

$$t_2 = 0.99$$

Output hiện tại:

$$out_{o_2} = 0.772928465$$

Sai số tại o_2 :

$$E_{o_2} = \frac{1}{2}(t_2 - out_{o_2})^2$$

Thay số:

$$E_{o_2} = \frac{1}{2}(0.99 - 0.772928465)^2$$

$$E_{o_2} \approx 0.023560026$$

4.3.3 Sai số tổng

$$E_{total} = E_{o_1} + E_{o_2}$$

$$E_{total} = 0.274811083 + 0.023560026$$

$$E_{total} \approx 0.298371109$$

Làm tròn:

$$E_{total} \approx 0.2984$$

Ghi nhớ

Trong video, giá trị sai số được nhắc đến là khoảng 0.2984. Đây chính là sai số tổng sau bước tính xuôi đầu tiên.

4.4 Updating Output Layer Weights – Cập nhật các trọng số lớp output

4.4.1 Cập nhật w_5

Trọng số w_5 nối từ h_1 đến o_1 .

Ta cần tính:

$$\frac{\partial E_{total}}{\partial w_5}$$

Vì w_5 chỉ đi trực tiếp vào o_1 , nên:

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{o_1}}{\partial out_{o_1}} \cdot \frac{\partial out_{o_1}}{\partial net_{o_1}} \cdot \frac{\partial net_{o_1}}{\partial w_5}$$

Từng thành phần:

$$\frac{\partial E_{o_1}}{\partial out_{o_1}} = out_{o_1} - t_1$$

$$= 0.751365070 - 0.01 = 0.741365070$$

$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = out_{o_1}(1 - out_{o_1})$$

$$= 0.751365070(1 - 0.751365070) \approx 0.186815602$$

$$\frac{\partial net_{o_1}}{\partial w_5} = out_{h_1} = 0.593269992$$

Vậy:

$$\frac{\partial E_{total}}{\partial w_5} = 0.741365070 \cdot 0.186815602 \cdot 0.593269992$$

$$\frac{\partial E_{total}}{\partial w_5} \approx 0.082167041$$

Cập nhật:

$$w_5^{new} = w_5 - \eta \frac{\partial E_{total}}{\partial w_5}$$

$$w_5^{new} = 0.40 - 0.5 \cdot 0.082167041$$

$$w_5^{new} \approx 0.358916480$$

4.4.2 Cập nhật w_6, w_7, w_8

Cách làm tương tự như w_5 .

Với w_6

w_6 nối từ h_2 đến o_1 :

$$\frac{\partial E_{total}}{\partial w_6} = \frac{\partial E_{o_1}}{\partial out_{o_1}} \cdot \frac{\partial out_{o_1}}{\partial net_{o_1}} \cdot \frac{\partial net_{o_1}}{\partial w_6}$$

Trong đó:

$$\frac{\partial net_{o_1}}{\partial w_6} = out_{h_2}$$

$$\frac{\partial E_{total}}{\partial w_6} = 0.741365070 \cdot 0.186815602 \cdot 0.596884378$$

$$\frac{\partial E_{total}}{\partial w_6} \approx 0.082667628$$

$$w_6^{new} = 0.45 - 0.5 \cdot 0.082667628$$

$$w_6^{new} \approx 0.408666186$$

Với w_7

w_7 nối từ h_1 đến o_2 :

$$\frac{\partial E_{total}}{\partial w_7} = \frac{\partial E_{o_2}}{\partial out_{o_2}} \cdot \frac{\partial out_{o_2}}{\partial net_{o_2}} \cdot \frac{\partial net_{o_2}}{\partial w_7}$$

$$\frac{\partial E_{o_2}}{\partial out_{o_2}} = out_{o_2} - t_2 = 0.772928465 - 0.99$$

$$= -0.217071535$$

$$\frac{\partial out_{o_2}}{\partial net_{o_2}} = out_{o_2}(1 - out_{o_2})$$

$$= 0.772928465(1 - 0.772928465) \approx 0.175510052$$

$$\frac{\partial net_{o_2}}{\partial w_7} = out_{h_1} = 0.593269992$$

$$\frac{\partial E_{total}}{\partial w_7} = -0.217071535 \cdot 0.175510052 \cdot 0.593269992$$

$$\frac{\partial E_{total}}{\partial w_7} \approx -0.022602540$$

$$w_7^{new} = 0.50 - 0.5(-0.022602540)$$

$$w_7^{new} \approx 0.511301270$$

Với w_8

w_8 nối từ h_2 đến o_2 :

$$\frac{\partial net_{o_2}}{\partial w_8} = out_{h_2} = 0.596884378$$

$$\frac{\partial E_{total}}{\partial w_8} = -0.217071535 \cdot 0.175510052 \cdot 0.596884378$$

$$\frac{\partial E_{total}}{\partial w_8} \approx -0.022740242$$

$$w_8^{new} = 0.55 - 0.5(-0.022740242)$$

$$w_8^{new} \approx 0.561370121$$

4.4.3 Bảng kết quả cập nhật lớp output

Trọng số	Giá trị cũ	Gradient	Giá trị mới
w_5	0.400000000	0.082167041	0.358916480
w_6	0.450000000	0.082667628	0.408666186
w_7	0.500000000	-0.022602540	0.511301270
w_8	0.550000000	-0.022740242	0.561370121

4.5 Cập nhật bias ở lớp output

4.5.1 Bias ứng với output o_1

Với bias của o_1 :

$$\frac{\partial net_{o_1}}{\partial b_{o_1}} = 1$$

Do đó:

$$\begin{aligned}\frac{\partial E_{total}}{\partial b_{o_1}} &= \frac{\partial E_{o_1}}{\partial out_{o_1}} \cdot \frac{\partial out_{o_1}}{\partial net_{o_1}} \\ &= 0.741365070 \cdot 0.186815602\end{aligned}$$

$$\frac{\partial E_{total}}{\partial b_{o_1}} \approx 0.138498562$$

Nếu bias ban đầu là 0.60:

$$b_{o_1}^{new} = 0.60 - 0.5 \cdot 0.138498562$$

$$b_{o_1}^{new} \approx 0.530750719$$

4.5.2 Bias ứng với output o_2

$$\begin{aligned}\frac{\partial E_{total}}{\partial b_{o_2}} &= \frac{\partial E_{o_2}}{\partial out_{o_2}} \cdot \frac{\partial out_{o_2}}{\partial net_{o_2}} \\ &= -0.217071535 \cdot 0.175510052\end{aligned}$$

$$\frac{\partial E_{total}}{\partial b_{o_2}} \approx -0.038098237$$

$$b_{o_2}^{new} = 0.60 - 0.5(-0.038098237)$$

$$b_{o_2}^{new} \approx 0.619049118$$

Lưu ý

Nếu trong tài liệu gốc dùng chung một bias b_o cho cả hai output, cách cập nhật có thể được trình bày khác. Trong ví dụ học tập này, ta tách thành b_{o_1} và b_{o_2} để thấy rõ mỗi output có một nhánh sai số riêng.

4.6 Cập nhật các trọng số phía trước

4.6.1 Điểm quan trọng khi tính w_1

Trọng số w_1 nối từ input i_1 đến hidden neuron h_1 .

Khi w_1 thay đổi, chuỗi ảnh hưởng không chỉ là:

$$w_1 \rightarrow net_{h_1} \rightarrow out_{h_1} \rightarrow net_{o_1} \rightarrow out_{o_1} \rightarrow E$$

Chuỗi này chưa đủ.

Vì out_{h_1} đi đến cả o_1 và o_2 , nên w_1 ảnh hưởng đến sai số qua cả hai nhánh:

$$out_{h_1} \rightarrow o_1 \rightarrow E_{o_1}$$

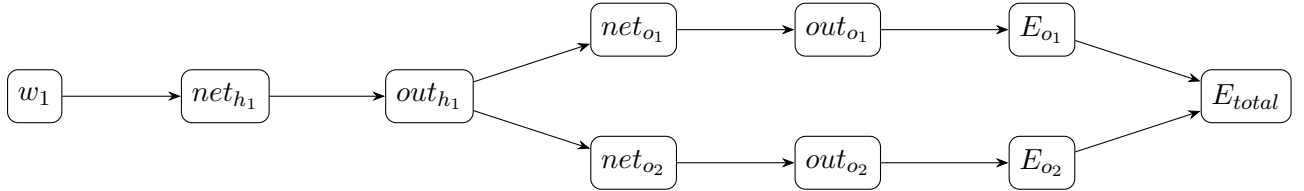
và:

$$out_{h_1} \rightarrow o_2 \rightarrow E_{o_2}$$

Ghi nhớ

Đây là điểm quan trọng nhất trong ví dụ ANN6: khi tính đạo hàm theo w_1 , phải cộng ảnh hưởng qua cả o_1 và o_2 . Nếu chỉ lấy nhánh qua o_1 , kết quả sẽ sai.

4.6.2 Sơ đồ đúng khi tính đạo hàm theo w_1



4.6.3 Tính $\frac{\partial E_{total}}{\partial out_{h_1}}$

Vì out_{h_1} ảnh hưởng đến cả o_1 và o_2 :

$$\frac{\partial E_{total}}{\partial out_{h_1}} = \frac{\partial E_{o_1}}{\partial out_{h_1}} + \frac{\partial E_{o_2}}{\partial out_{h_1}}$$

Nhánh qua o_1 :

$$\frac{\partial E_{o_1}}{\partial out_{h_1}} = \frac{\partial E_{o_1}}{\partial net_{o_1}} \cdot \frac{\partial net_{o_1}}{\partial out_{h_1}}$$

Ta đã có:

$$\frac{\partial E_{o_1}}{\partial net_{o_1}} = 0.138498562$$

và:

$$\frac{\partial net_{o_1}}{\partial out_{h_1}} = w_5 = 0.40$$

Do đó:

$$\frac{\partial E_{o_1}}{\partial out_{h_1}} = 0.138498562 \cdot 0.40$$

$$\frac{\partial E_{o_1}}{\partial out_{h_1}} \approx 0.055399425$$

Nhánh qua o_2 :

$$\frac{\partial E_{o_2}}{\partial out_{h_1}} = \frac{\partial E_{o_2}}{\partial net_{o_2}} \cdot \frac{\partial net_{o_2}}{\partial out_{h_1}}$$

Ta đã có:

$$\frac{\partial E_{o_2}}{\partial net_{o_2}} = -0.038098237$$

và:

$$\frac{\partial net_{o_2}}{\partial out_{h_1}} = w_7 = 0.50$$

Do đó:

$$\frac{\partial E_{o_2}}{\partial out_{h_1}} = -0.038098237 \cdot 0.50$$

$$\frac{\partial E_{o_2}}{\partial out_{h_1}} \approx -0.019049119$$

Cộng hai nhánh:

$$\frac{\partial E_{total}}{\partial out_{h_1}} = 0.055399425 + (-0.019049119)$$

$$\frac{\partial E_{total}}{\partial out_{h_1}} \approx 0.036350306$$

4.6.4 Tính gradient của w_1

Ta cần:

$$\frac{\partial E_{total}}{\partial w_1}$$

Theo chuỗi:

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h_1}} \cdot \frac{\partial out_{h_1}}{\partial net_{h_1}} \cdot \frac{\partial net_{h_1}}{\partial w_1}$$

Trong đó:

$$\frac{\partial E_{total}}{\partial out_{h_1}} \approx 0.036350306$$

$$\frac{\partial out_{h_1}}{\partial net_{h_1}} = out_{h_1}(1 - out_{h_1})$$

$$= 0.593269992(1 - 0.593269992)$$

$$\frac{\partial out_{h_1}}{\partial net_{h_1}} \approx 0.241300709$$

Và:

$$\frac{\partial net_{h_1}}{\partial w_1} = i_1 = 0.05$$

Vậy:

$$\frac{\partial E_{total}}{\partial w_1} = 0.036350306 \cdot 0.241300709 \cdot 0.05$$

$$\frac{\partial E_{total}}{\partial w_1} \approx 0.000438568$$

Cập nhật:

$$w_1^{new} = 0.15 - 0.5 \cdot 0.000438568$$

$$w_1^{new} \approx 0.149780716$$

4.6.5 Cập nhật w_2

w_2 cũng đi vào h_1 , nên phần đầu giống w_1 . Điểm khác là:

$$\frac{\partial net_{h_1}}{\partial w_2} = i_2 = 0.10$$

Do đó:

$$\frac{\partial E_{total}}{\partial w_2} = 0.036350306 \cdot 0.241300709 \cdot 0.10$$

$$\frac{\partial E_{total}}{\partial w_2} \approx 0.000877135$$

Cập nhật:

$$w_2^{new} = 0.20 - 0.5 \cdot 0.000877135$$

$$w_2^{new} \approx 0.199561432$$

4.7 Cập nhật w_3 và w_4

4.7.1 Tính ảnh hưởng qua hidden neuron h_2

Tương tự h_1 , output out_{h_2} cũng đi đến cả o_1 và o_2 .

Do đó:

$$\frac{\partial E_{total}}{\partial out_{h_2}} = \frac{\partial E_{o_1}}{\partial out_{h_2}} + \frac{\partial E_{o_2}}{\partial out_{h_2}}$$

Nhánh qua o_1 :

$$\frac{\partial E_{o_1}}{\partial out_{h_2}} = \frac{\partial E_{o_1}}{\partial net_{o_1}} \cdot \frac{\partial net_{o_1}}{\partial out_{h_2}}$$

$$= 0.138498562 \cdot w_6$$

$$= 0.138498562 \cdot 0.45$$

$$\approx 0.062324353$$

Nhánh qua o_2 :

$$\frac{\partial E_{o_2}}{\partial out_{h_2}} = \frac{\partial E_{o_2}}{\partial net_{o_2}} \cdot \frac{\partial net_{o_2}}{\partial out_{h_2}}$$

$$= -0.038098237 \cdot w_8$$

$$= -0.038098237 \cdot 0.55$$

$$\approx -0.020954030$$

Cộng lại:

$$\frac{\partial E_{total}}{\partial out_{h_2}} = 0.062324353 - 0.020954030$$

$$\frac{\partial E_{total}}{\partial out_{h_2}} \approx 0.041370323$$

4.7.2 Cập nhật w_3

$$\frac{\partial E_{total}}{\partial w_3} = \frac{\partial E_{total}}{\partial out_{h_2}} \cdot \frac{\partial out_{h_2}}{\partial net_{h_2}} \cdot \frac{\partial net_{h_2}}{\partial w_3}$$

Ta có:

$$\frac{\partial out_{h_2}}{\partial net_{h_2}} = out_{h_2}(1 - out_{h_2})$$

$$= 0.596884378(1 - 0.596884378)$$

$$\approx 0.240613417$$

Và:

$$\frac{\partial net_{h_2}}{\partial w_3} = i_1 = 0.05$$

Vậy:

$$\frac{\partial E_{total}}{\partial w_3} = 0.041370323 \cdot 0.240613417 \cdot 0.05$$

$$\frac{\partial E_{total}}{\partial w_3} \approx 0.000497713$$

Cập nhật:

$$w_3^{new} = 0.25 - 0.5 \cdot 0.000497713$$

$$w_3^{new} \approx 0.249751144$$

4.7.3 Cập nhật w_4

Với w_4 :

$$\frac{\partial net_{h_2}}{\partial w_4} = i_2 = 0.10$$

$$\frac{\partial E_{total}}{\partial w_4} = 0.041370323 \cdot 0.240613417 \cdot 0.10$$

$$\frac{\partial E_{total}}{\partial w_4} \approx 0.000995425$$

Cập nhật:

$$w_4^{new} = 0.30 - 0.5 \cdot 0.000995425$$

$$w_4^{new} \approx 0.299502287$$

4.7.4 Bảng kết quả cập nhật lớp hidden

Trọng số	Giá trị cũ	Gradient	Giá trị mới
w_1	0.1500000000	0.000438568	0.149780716
w_2	0.2000000000	0.000877135	0.199561432
w_3	0.2500000000	0.000497713	0.249751144
w_4	0.3000000000	0.000995425	0.299502287

4.8 Cập nhật bias ở lớp hidden

4.8.1 Bias ứng với h_1

Vì:

$$\frac{\partial net_{h_1}}{\partial b_{h_1}} = 1$$

nên:

$$\begin{aligned}\frac{\partial E_{total}}{\partial b_{h_1}} &= \frac{\partial E_{total}}{\partial out_{h_1}} \cdot \frac{\partial out_{h_1}}{\partial net_{h_1}} \\ &= 0.036350306 \cdot 0.241300709\end{aligned}$$

$$\frac{\partial E_{total}}{\partial b_{h_1}} \approx 0.008771355$$

Cập nhật:

$$b_{h_1}^{new} = 0.35 - 0.5 \cdot 0.008771355$$

$$b_{h_1}^{new} \approx 0.345614323$$

4.8.2 Bias ứng với h_2

$$\begin{aligned}\frac{\partial E_{total}}{\partial b_{h_2}} &= \frac{\partial E_{total}}{\partial out_{h_2}} \cdot \frac{\partial out_{h_2}}{\partial net_{h_2}} \\ &= 0.041370323 \cdot 0.240613417\end{aligned}$$

$$\frac{\partial E_{total}}{\partial b_{h_2}} \approx 0.009954255$$

Cập nhật:

$$b_{h_2}^{new} = 0.35 - 0.5 \cdot 0.009954255$$

$$b_{h_2}^{new} \approx 0.345022873$$

Lưu ý

Tương tự bias ở lớp output, nếu tài liệu gốc dùng chung một bias cho cả lớp hidden thì cách ghi có thể khác. Việc tách b_{h_1} và b_{h_2} giúp dễ theo dõi từng nhánh tính toán.

4.9 Tổng kết một lượt cập nhật

4.9.1 Bảng trọng số trước và sau cập nhật

Trọng số	Trước cập nhật	Sau cập nhật
w_1	0.150000000	0.149780716
w_2	0.200000000	0.199561432
w_3	0.250000000	0.249751144
w_4	0.300000000	0.299502287
w_5	0.400000000	0.358916480
w_6	0.450000000	0.408666186
w_7	0.500000000	0.511301270
w_8	0.550000000	0.561370121

4.9.2 Bảng bias trước và sau cập nhật

Bias	Trước cập nhật	Sau cập nhật
b_{o_1}	0.600000000	0.530750719
b_{o_2}	0.600000000	0.619049118
b_{h_1}	0.350000000	0.345614323
b_{h_2}	0.350000000	0.345022873

4.9.3 Ý nghĩa của một lượt cập nhật

Sau khi cập nhật xong tất cả trọng số và bias, mạng đã thay đổi một chút so với trạng thái ban đầu. Đây mới chỉ là một lượt cập nhật với một cặp dữ liệu.

Với cặp dữ liệu tiếp theo, ta lại làm:

tính xuôi $\rightarrow$ tính sai số $\rightarrow$ tính ngược $\rightarrow$ cập nhật trọng số

Quá trình này lặp đi lặp lại rất nhiều lần cho đến khi sai số giảm đến mức chấp nhận được.

4.10 Những lỗi dễ mắc trong ví dụ

4.10.1 Lỗi 1: Chỉ tính một nhánh khi cập nhật w_1

Sai lầm thường gặp là viết:

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{o_1}}{\partial out_{o_1}} \cdot \frac{\partial out_{o_1}}{\partial net_{o_1}} \cdot \frac{\partial net_{o_1}}{\partial out_{h_1}} \cdot \frac{\partial out_{h_1}}{\partial net_{h_1}} \cdot \frac{\partial net_{h_1}}{\partial w_1}$$

Biểu thức này chỉ đi qua nhánh o_1 , nên thiếu nhánh o_2 .

Biểu thức đúng phải là:

$$\frac{\partial E_{total}}{\partial w_1} = \left(\frac{\partial E_{o_1}}{\partial out_{h_1}} + \frac{\partial E_{o_2}}{\partial out_{h_1}} \right) \cdot \frac{\partial out_{h_1}}{\partial net_{h_1}} \cdot \frac{\partial net_{h_1}}{\partial w_1}$$

4.10.2 Lỗi 2: Nhầm dấu khi cập nhật

Công thức cập nhật là:

$$w^{new} = w - \eta \frac{\partial E}{\partial w}$$

Nếu gradient dương, trọng số giảm.

Nếu gradient âm, trọng số tăng.

Ví dụ:

$$\frac{\partial E}{\partial w_7} \approx -0.022602540$$

Do đó:

$$w_7^{new} = 0.50 - 0.5(-0.022602540)$$

$$w_7^{new} > 0.50$$

Vì gradient âm nên w_7 tăng.

4.10.3 Lỗi 3: Dùng nhầm output của neuron

Khi tính gradient của w_5 :

$$\frac{\partial net_{o_1}}{\partial w_5} = out_{h_1}$$

không phải out_{h_2} .

Khi tính gradient của w_6 :

$$\frac{\partial net_{o_1}}{\partial w_6} = out_{h_2}$$

không phải out_{h_1} .

Lưu ý

Mỗi trọng số nằm trên một cạnh cụ thể. Đạo hàm của net theo trọng số đó bằng đúng output của neuron ở đầu vào của cạnh đó.

4.11 Bài tập tự kiểm tra theo ví dụ

Bài tập 1: Kiểm tra sai số tổng

Dùng các giá trị:

$$out_{o_1} = 0.751365070, \quad t_1 = 0.01$$

$$out_{o_2} = 0.772928465, \quad t_2 = 0.99$$

Hãy tính:

$$E_{total} = \frac{1}{2}(t_1 - out_{o_1})^2 + \frac{1}{2}(t_2 - out_{o_2})^2$$

Đáp án

$$E_{total} \approx 0.298371109$$

$$E_{total} \approx 0.2984$$

Bài tập 2: Tính lại w_5^{new}

Cho:

$$\frac{\partial E}{\partial w_5} = 0.082167041$$

$$w_5 = 0.40, \quad \eta = 0.5$$

Tính w_5^{new} .

Đáp án

$$w_5^{new} = 0.40 - 0.5 \cdot 0.082167041$$

$$w_5^{new} \approx 0.358916480$$

Bài tập 3: Vì sao w_7 tăng?

Cho:

$$\frac{\partial E}{\partial w_7} \approx -0.022602540$$

$$w_7 = 0.50, \quad \eta = 0.5$$

Tính w_7^{new} và giải thích vì sao nó tăng.

Đáp án

$$w_7^{new} = 0.50 - 0.5(-0.022602540)$$

$$w_7^{new} = 0.50 + 0.011301270$$

$$w_7^{new} \approx 0.511301270$$

Vì gradient âm nên khi trừ gradient, giá trị trọng số tăng.

Bài tập 4: Viết đúng biểu thức cho w_1

Điền vào chỗ trống:

$$\frac{\partial E_{total}}{\partial w_1} = (\text{_____} + \text{_____}) \cdot \frac{\partial out_{h_1}}{\partial net_{h_1}} \cdot \frac{\partial net_{h_1}}{\partial w_1}$$

Đáp án

$$\frac{\partial E_{total}}{\partial w_1} = \left(\frac{\partial E_{o_1}}{\partial out_{h_1}} + \frac{\partial E_{o_2}}{\partial out_{h_1}} \right) \cdot \frac{\partial out_{h_1}}{\partial net_{h_1}} \cdot \frac{\partial net_{h_1}}{\partial w_1}$$

Kết luận

Ví dụ ANN6 minh họa cách tính lan truyền ngược bằng tay trên một mạng nhỏ. Sau khi tính xuôi, ta thu được sai số tổng khoảng:

$$E_{total} \approx 0.2984$$

Sau đó, ta tính ngược từ lớp output để cập nhật w_5, w_6, w_7, w_8 , rồi tiếp tục đi về lớp trước để cập nhật w_1, w_2, w_3, w_4 .

Điểm quan trọng nhất của ví dụ là khi một neuron ở lớp hidden nối đến nhiều output, sai số truyền về neuron đó là tổng ảnh hưởng từ tất cả các nhánh phía sau. Cụ thể, khi tính w_1 , không được chỉ xét nhánh từ h_1 đến o_1 , mà phải cộng cả nhánh từ h_1 đến o_2 .

$$out_{h_1} \rightarrow o_1 \rightarrow E_{o_1}$$

$$out_{h_1} \rightarrow o_2 \rightarrow E_{o_2}$$

Vì vậy:

$$\frac{\partial E_{total}}{\partial out_{h_1}} = \frac{\partial E_{o_1}}{\partial out_{h_1}} + \frac{\partial E_{o_2}}{\partial out_{h_1}}$$

Đây là phần “ruột” quan trọng của ví dụ, giúp hiểu thật sự điều gì xảy ra bên trong quá trình backpropagation thay vì chỉ dùng thư viện có sẵn.

Chương 5

ANN7: Training Issues – Vấn đề huấn luyện

Learning Objectives – Mục tiêu bài học

Bài ANN7 tập trung vào một số vấn đề thực tế khi huấn luyện mạng nơ-ron. Sau khi học xong, người học cần nắm được:

1. Vì sao về mặt lý thuyết ta có thể dùng backpropagation và gradient descent để huấn luyện mạng, nhưng thực tế việc huấn luyện không phải lúc nào cũng dễ.
2. Hiểu hiện tượng *vanishing gradient*: gradient quá nhỏ làm trọng số cập nhật rất chậm hoặc gần như không cập nhật.
3. Biết vì sao các mạng sâu dùng sigmoid dễ gặp vanishing gradient.
4. Hiểu vì sao ReLU và các biến thể của nó được dùng phổ biến hơn sigmoid trong nhiều mạng sâu hiện đại.
5. Phân biệt được *underfitting*, *good fitting* và *overfitting*.
6. Hiểu overfitting qua hai dạng bài toán: phân loại và hồi quy.
7. Hiểu vai trò của nhiều dữ liệu, sai số đo lường và khả năng tổng quát hóa trên dữ liệu mới.
8. Phân biệt bài toán phân loại và bài toán hồi quy.

5.1 From Theory to Practice – Từ lý thuyết huấn luyện đến vấn đề thực tế

5.1.1 Nhắc lại quá trình huấn luyện

Trong các bài trước, ta đã học quy trình huấn luyện mạng nơ-ron:

Input → Tính xuôi → Output dự đoán → Tính loss → Lan truyền ngược → Cập nhật trọng số

Về mặt logic, nếu có dữ liệu, có hàm mất mát và có thuật toán tối ưu, ta có thể kỳ vọng rằng mô hình sẽ dần hội tụ.

Tuy nhiên, thực tế phức tạp hơn. Có những trường hợp mạng học rất chậm, không học được, hoặc học quá sát dữ liệu huấn luyện đến mức làm việc kém trên dữ liệu mới.

Ghi nhớ

Một mô hình học tốt không chỉ là mô hình có loss nhỏ trên dữ liệu huấn luyện. Điều quan trọng hơn là mô hình phải làm việc tốt trên dữ liệu mới chưa từng thấy.

5.1.2 Ba vấn đề chính trong bài ANN7

Bài học tập trung vào ba vấn đề quan trọng:

1. **Vanishing gradient**: gradient biến mất hoặc gần bằng 0.
2. **Underfitting**: mô hình quá đơn giản, chưa học được xu hướng dữ liệu.
3. **Overfitting**: mô hình quá khớp với dữ liệu huấn luyện, làm mất khả năng tổng quát hóa.

5.2 Vanishing Gradient – Vanishing Gradient

5.2.1 Vanishing gradient là gì?

Trong backpropagation, gradient được dùng để cập nhật trọng số:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial E}{\partial w}$$

Trong đó:

- w là trọng số;
- η là learning rate;
- $\frac{\partial E}{\partial w}$ là gradient của hàm mất mát theo trọng số w .

Nếu gradient rất nhỏ:

$$\frac{\partial E}{\partial w} \approx 0$$

thì bước cập nhật:

$$\eta \frac{\partial E}{\partial w}$$

cũng rất nhỏ. Khi đó trọng số gần như không thay đổi.

Ghi nhớ

Vanishing gradient là hiện tượng gradient trở nên quá nhỏ trong quá trình lan truyền ngược, khiến các trọng số ở những lớp phía trước cập nhật rất chậm hoặc gần như không cập nhật.

5.2.2 Ví dụ số đơn giản

Giả sử:

$$w = 0.5, \quad \eta = 0.1$$

Nếu gradient bình thường:

$$\frac{\partial E}{\partial w} = 0.8$$

thì:

$$w_{\text{new}} = 0.5 - 0.1 \cdot 0.8 = 0.42$$

Trọng số thay đổi đáng kể.

Nhưng nếu gradient rất nhỏ:

$$\frac{\partial E}{\partial w} = 0.000001$$

thì:

$$w_{\text{new}} = 0.5 - 0.1 \cdot 0.000001$$

$$w_{\text{new}} = 0.4999999$$

Trọng số gần như không thay đổi.

Ví dụ minh họa

Nếu một mạng có hàng triệu trọng số, và nhiều trọng số ở các lớp đầu chỉ cập nhật với lượng cực nhỏ như vậy, quá trình huấn luyện có thể trở nên rất chậm. Mạng có thể cần rất nhiều thời gian mà vẫn không học được tốt.

5.2.3 Vì sao gradient có thể biến mất?

Trong backpropagation, gradient thường là tích của nhiều đạo hàm thành phần.

Ví dụ đơn giản:

$$\frac{\partial E}{\partial w_1} = a_1 \cdot a_2 \cdot a_3 \cdot \dots \cdot a_n$$

Nếu nhiều thành phần trong đó nhỏ hơn 1, tích của chúng sẽ càng ngày càng nhỏ.

$$0.5^5 = 0.03125$$

$$0.5^{10} = 0.0009765625$$

$$0.5^{20} \approx 0.0000009537$$

Khi mạng càng sâu, chuỗi nhân trong backpropagation càng dài. Vì vậy, gradient truyền về các lớp đầu có thể trở nên cực kỳ nhỏ.

Ghi nhớ

Mạng càng sâu thì gradient càng phải đi qua nhiều lớp. Nếu các đạo hàm trung gian nhỏ, gradient ở các lớp đầu có thể bị “triệt tiêu”.

5.3 Vanishing Gradient & Sigmoid – Vanishing Gradient và hàm sigmoid

5.3.1 Hàm sigmoid

Hàm sigmoid được định nghĩa:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

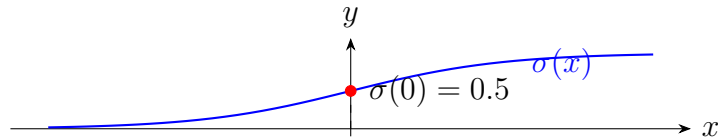
Đạo hàm của sigmoid có dạng:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Giá trị lớn nhất của đạo hàm sigmoid là:

$$\sigma'(0) = 0.25$$

Điều này có nghĩa là đạo hàm sigmoid không bao giờ lớn hơn 0.25.



5.3.2 Vấn đề của sigmoid trong mạng sâu

Vì đạo hàm sigmoid thường nhỏ, khi nhân nhiều đạo hàm sigmoid qua nhiều lớp, gradient có thể giảm rất nhanh.

Ví dụ, nếu mỗi lớp tạo ra một hệ số đạo hàm khoảng 0.25, sau 10 lớp:

$$0.25^{10} = 0.00000095367431640625$$

Đây là một số rất nhỏ.

Sau 20 lớp:

$$0.25^{20} \approx 9.09 \times 10^{-13}$$

Gradient lúc này gần như biến mất.

Cảnh báo

Đây là một lý do quan trọng khiến sigmoid không còn được dùng phổ biến ở các lớp ẩn trong nhiều mạng sâu hiện đại.

5.3.3 Sigmoid bị bão hòa

Khi x rất lớn hoặc rất nhỏ, sigmoid gần như nằm ngang.

$$x \gg 0 \Rightarrow \sigma(x) \approx 1$$

$$x \ll 0 \Rightarrow \sigma(x) \approx 0$$

Ở hai vùng này, đạo hàm rất nhỏ:

$$\sigma'(x) \approx 0$$

Khi đạo hàm gần 0, gradient truyền qua neuron đó cũng rất nhỏ.

Ví dụ minh họa

Giả sử một neuron sigmoid nhận $x = 10$.

$$\sigma(10) \approx 0.99995$$

Đạo hàm:

$$\sigma'(10) = \sigma(10)(1 - \sigma(10))$$

$$\approx 0.99995(1 - 0.99995) \approx 0.00005$$

Đạo hàm này rất nhỏ. Nếu nhiều neuron đều rơi vào vùng như vậy, gradient sẽ yếu đi rất nhanh.

5.4 ReLU: Mitigating Vanishing Gradient – ReLU và ý tưởng giảm vanishing gradient

5.4.1 Hàm ReLU

ReLU, viết đầy đủ là *Rectified Linear Unit*, được định nghĩa:

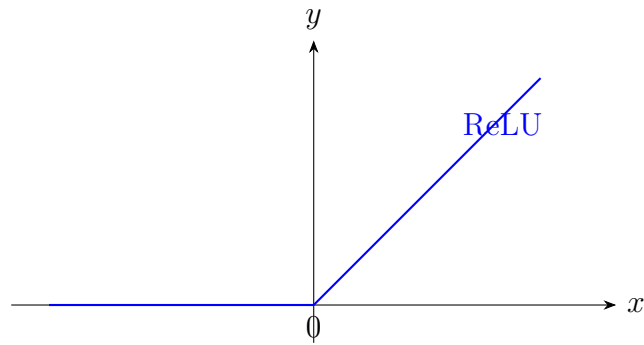
$$\text{ReLU}(x) = \max(0, x)$$

Tức là:

$$\text{ReLU}(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

Đạo hàm của ReLU:

$$\text{ReLU}'(x) = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases}$$



5.4.2 Vì sao ReLU giúp giảm vanishing gradient?

Với $x > 0$, đạo hàm của ReLU bằng 1.

$$\text{ReLU}'(x) = 1$$

Khi lan truyền ngược qua nhiều lớp, nhân với 1 không làm gradient nhỏ đi như khi nhân với các số nhỏ hơn 1.

Ví dụ minh họa

So sánh hai chuỗi nhân:

$$0.25^{10} \approx 0.0000009537$$

nhưng:

$$1^{10} = 1$$

Vì vậy, ở vùng ReLU hoạt động, gradient có thể truyền ngược tốt hơn nhiều so với sigmoid.

5.4.3 Hạn chế của ReLU

ReLU không hoàn hảo. Với $x < 0$, đạo hàm của ReLU bằng 0. Khi một neuron rơi vào vùng âm và luôn cho output 0, nó có thể khó được cập nhật.

Hiện tượng này thường được gọi là *đying ReLU*.

Để khắc phục, nhiều biến thể của ReLU được đề xuất, ví dụ:

- Leaky ReLU;
- Parametric ReLU;
- ELU;
- SELU.

Lưu ý

Ý chính trong bài học là: các hàm kích hoạt hiện đại được phát triển một phần để giảm vấn đề vanishing gradient và giúp mạng sâu huấn luyện hiệu quả hơn.

5.5 Underfitting và Overfitting

5.5.1 Bài toán fitting là gì?

Khi huấn luyện mô hình, ta muốn mô hình học được quan hệ giữa input và output từ dữ liệu huấn luyện.

Nếu mô hình học quá ít, nó không nắm được xu hướng dữ liệu. Đây là underfitting.

Nếu mô hình học quá sát từng điểm dữ liệu huấn luyện, kể cả nhiễu và điểm bất thường, nó có thể làm việc kém trên dữ liệu mới. Đây là overfitting.

Trường hợp mong muốn là mô hình học được xu hướng chung của dữ liệu.

Ghi nhớ

Mục tiêu không phải là nhớ chính xác mọi điểm dữ liệu huấn luyện, mà là học được quy luật tổng quát để áp dụng cho dữ liệu mới.

5.5.2 Ba trạng thái thường gặp

Trạng thái	Mô tả	Hậu quả
Underfitting	Mô hình quá đơn giản, chưa khớp dữ liệu	Sai nhiều trên cả train và test
Good fitting	Mô hình học được xu hướng chung	Làm việc tốt trên dữ liệu mới
Overfitting	Mô hình quá khớp dữ liệu huấn luyện	Train tốt nhưng test kém

5.6 Ví dụ phân loại 2D

5.6.1 Mô tả ví dụ

Giả sử ta có dữ liệu 2D, ví dụ:

$$x_1 = \text{chiều cao}, \quad x_2 = \text{cân nặng}$$

Mỗi điểm dữ liệu thuộc một trong hai lớp:

Lớp A hoặc Lớp B

Ví dụ:

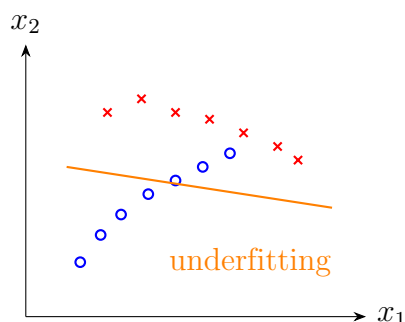
- người thuộc nhóm A và nhóm B;
- người thừa cân và không thừa cân;
- người mắc bệnh và không mắc bệnh.

Mô hình cần tìm đường ranh giới để phân chia hai lớp.

5.6.2 Underfitting trong phân loại

Underfitting xảy ra khi đường phân lớp quá đơn giản so với dữ liệu.

Ví dụ, dữ liệu có xu hướng cong, nhưng mô hình chỉ dùng một đường thẳng đơn giản. Khi đó, nhiều điểm bị phân loại sai.

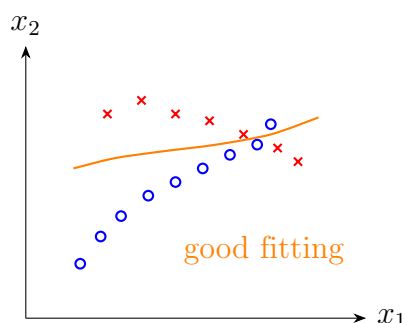


Ví dụ minh họa

Nếu dữ liệu thật có ranh giới cong, nhưng ta dùng mô hình tuyến tính quá đơn giản, mô hình sẽ không học được cấu trúc dữ liệu. Kết quả là nhiều điểm ở cả hai lớp bị phân loại sai.

5.6.3 fitting trong phân loại

Good fitting là khi mô hình học được xu hướng chung của dữ liệu. Nó có thể chấp nhận một vài điểm sai do nhiễu hoặc điểm bất thường, nhưng đường phân lớp vẫn hợp lý.



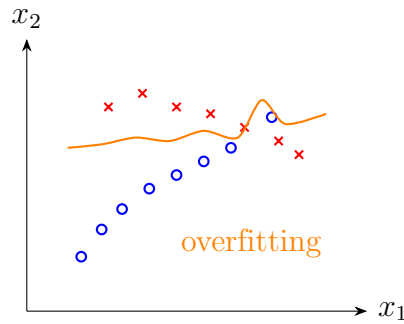
Ghi nhớ

Một mô hình tốt không nhất thiết phải phân loại đúng 100% dữ liệu huấn luyện. Nó cần học được xu hướng chung để dự đoán tốt trên dữ liệu mới.

5.6.4 Overfitting trong phân loại

Overfitting xảy ra khi mô hình cố gắng phân loại đúng hoàn hảo mọi điểm trong tập huấn luyện, kể cả điểm nhiễu hoặc điểm bất thường.

Đường phân lớp khi đó có thể trở nên rất ngoằn ngoèo, phức tạp và phi tự nhiên.



Cảnh báo

Nếu mô hình quá khớp với dữ liệu huấn luyện, nó có thể đạt sai số gần 0 trên tập huấn luyện nhưng lại dự đoán sai trên dữ liệu thực tế.

5.6.5 Vì sao phân loại hoàn hảo chưa chắc là tốt?

Giả sử mô hình phân loại hoàn hảo mọi điểm huấn luyện. Điều này nghe có vẻ tốt, nhưng chưa chắc đúng.

Lý do là bài toán thực tế không chỉ yêu cầu xử lý các điểm đã có. Ta muốn dùng mô hình cho dữ liệu mới trong tương lai.

Nếu đường phân lớp quá phức tạp chỉ để ôm sát dữ liệu huấn luyện, nó có thể đánh mất xu hướng chung.

Ví dụ minh họa

Giả sử mô hình dùng để phân loại người mắc bệnh và không mắc bệnh. Nếu trong dữ liệu huấn luyện có một điểm nhiễu, mô hình overfit có thể bẻ cong đường phân lớp chỉ để phân loại đúng điểm đó.

Khi gặp một người mới có đặc trưng gần khu vực đó, mô hình có thể dự đoán sai nghiêm trọng. Ví dụ, người không thuộc nhóm nguy cơ lại bị dự đoán là có bệnh.

5.7 Nhiều dữ liệu và sai số đo lường

5.7.1 Dữ liệu đo được không hoàn hảo

Trong thực tế, dữ liệu không bao giờ hoàn hảo tuyệt đối. Dữ liệu có thể bị ảnh hưởng bởi:

- sai số cảm biến;
- điều kiện môi trường;
- cách đo;
- sai số do con người nhập liệu;
- nhiễu ngẫu nhiên;
- điểm bất thường.

5.7.2 Ví dụ đo chiều dài

Giả sử ta cần đo chiều dài một viên gạch bằng thước hoặc cảm biến laser.

Con số đo được không chắc là chiều dài thật tuyệt đối của vật thể. Nó chỉ là giá trị đo lường.

Giá trị đo có thể bị ảnh hưởng bởi:

- độ chính xác của thiết bị;
- góc đặt cảm biến;
- bề mặt phản xạ;
- nhiệt độ và độ ẩm;
- lỗi ghi chép hoặc nhập dữ liệu.

Ghi nhớ

Dữ liệu huấn luyện thường chứa nhiễu. Vì vậy, mô hình tốt cần học xu hướng chung thay vì cố gắng khớp hoàn hảo từng điểm dữ liệu.

5.7.3 Liên hệ với overfitting

Nếu mô hình quá phức tạp, nó có thể học luôn cả nhiễu trong dữ liệu.

Khi đó, mô hình không chỉ học quy luật thật mà còn học cả sai số đo, điểm bất thường và lỗi nhập liệu.

Đây là nguyên nhân quan trọng dẫn đến overfitting.

5.8 Ví dụ hồi quy

5.8.1 Bài toán hồi quy là gì?

Hồi quy, tiếng Anh là *regression*, là bài toán dự đoán một giá trị liên tục.

Ví dụ:

- dự đoán giá cổ phiếu ngày mai là bao nhiêu;
- dự đoán nhiệt độ ngày mai;
- dự đoán sản lượng điện;
- dự đoán giá nhà;
- dự đoán chi phí sản xuất.

Khác với phân loại, output của hồi quy không phải là một lớp rời rạc, mà là một con số liên tục.

5.8.2 Dữ liệu hồi quy có nhiễu

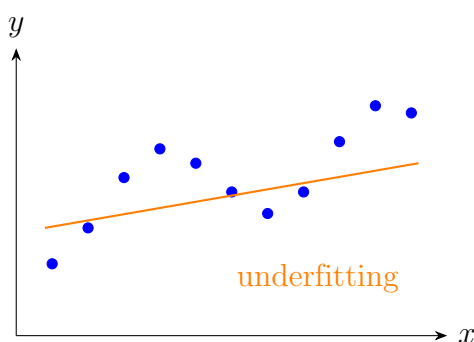
Giả sử dữ liệu thật đi theo một đường cong nào đó. Nhưng do nhiễu, các điểm dữ liệu đo được không nằm chính xác trên đường cong thật.

Mô hình cần tìm một đường xấp xỉ phù hợp với xu hướng chung của dữ liệu.

5.8.3 Underfitting trong hồi quy

Underfitting xảy ra khi mô hình quá đơn giản.

Ví dụ, dữ liệu có dạng cong hoặc dạng sóng, nhưng ta dùng một đường thẳng để xấp xỉ.

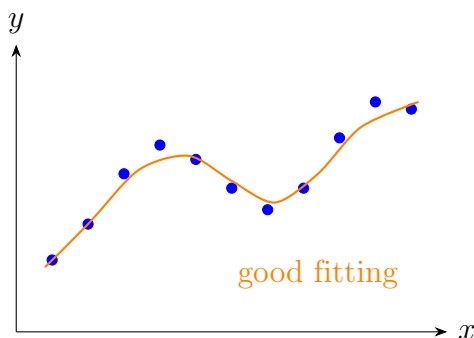


Ví dụ minh họa

Nếu dữ liệu nhiệt độ theo thời gian có xu hướng tăng giảm theo mùa, nhưng mô hình chỉ dùng một đường thẳng, mô hình có thể không bắt được dao động thật của dữ liệu.

5.8.4 Good fitting trong hồi quy

Good fitting là khi mô hình xấp xỉ được xu hướng chính của dữ liệu, dù không đi qua mọi điểm.



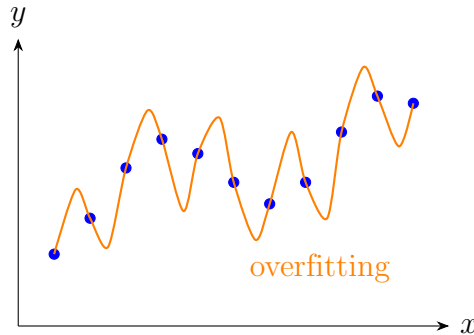
Ghi nhớ

Trong hồi quy, mô hình tốt không nhất thiết đi qua tất cả điểm dữ liệu. Nó cần mô tả được xu hướng tổng quát phía sau dữ liệu nhiễu.

5.8.5 Overfitting trong hồi quy

Overfitting xảy ra khi mô hình quá phức tạp và cố gắng đi qua gần như tất cả các điểm dữ liệu.

Ví dụ, dùng đa thức bậc rất cao để fit một tập điểm nhiễu.



Cảnh báo

Một đường cong đi qua tất cả điểm huấn luyện có thể có sai số huấn luyện bằng 0, nhưng vẫn là mô hình xấu nếu nó không phản ánh xu hướng thật của dữ liệu.

5.8.6 Ví dụ đa thức bậc cao

Giả sử dữ liệu thật có thể được mô tả tốt bằng một hàm bậc 4. Nhưng nếu ta dùng đa thức bậc 15, mô hình có thể uốn lượn rất mạnh để đi qua từng điểm dữ liệu.

Khi đó:

$$\text{Training error} \approx 0$$

nhưng:

$$\text{Test error có thể rất lớn}$$

Vì mô hình đã học cả nhiễu, không chỉ học quy luật.

5.9 Phân loại và hồi quy

5.9.1 Bài toán phân loại

Phân loại, tiếng Anh là *classification*, là bài toán dự đoán một nhãn rời rạc.

Ví dụ:

- ảnh là chó hay mèo;
- email là spam hay không spam;
- người có mắc COVID hay không;
- giá cổ phiếu ngày mai tăng, giảm hay không đổi;
- vật thể là xe hơi, máy bay, người hay con vật.

Output thường là một class:

$$y \in \{C_1, C_2, \dots, C_K\}$$

5.9.2 Bài toán hồi quy

Hồi quy là bài toán dự đoán giá trị liên tục.

Ví dụ:

- giá cổ phiếu ngày mai bằng bao nhiêu;
- giá nhà là bao nhiêu;
- sản lượng điện tháng tới là bao nhiêu;
- nhiệt độ ngày mai là bao nhiêu.

Output thường là một số thực:

$$y \in \mathbb{R}$$

5.9.3 So sánh classification và regression

Tiêu chí	Classification	Regression
Loại output	Nhãn rời rạc	Giá trị liên tục
Ví dụ output	Chó, mèo, xe hơi	25.3 độ C, 1.2 tỷ đồng
Ví dụ bài toán	Email spam hay không	Giá nhà bao nhiêu
Sai số thường dùng	Cross entropy	MSE, MAE

Ví dụ minh họa

Cùng là giá cổ phiếu, có thể có hai bài toán khác nhau:

- Dự đoán ngày mai cổ phiếu tăng hay giảm: classification.
- Dự đoán ngày mai cổ phiếu có giá chính xác bao nhiêu: regression.

5.10 Tổng hợp: Khi nào mô hình học tốt?

5.10.1 Mô hình học tốt cần cân bằng

Một mô hình học tốt cần cân bằng giữa hai cực:

quá đơn giản $\longleftrightarrow$ quá phức tạp

Nếu quá đơn giản, mô hình underfit.

Nếu quá phức tạp, mô hình dễ overfit.

Trường hợp tốt là mô hình đủ linh hoạt để học xu hướng dữ liệu, nhưng không quá phức tạp đến mức học cả nhiễu.

5.10.2 Dấu hiệu nhận biết

Trường hợp	Dữ liệu huấn luyện	Dữ liệu kiểm tra
Underfitting	Sai số cao	Sai số cao
Good fitting	Sai số thấp vừa phải	Sai số thấp
Overfitting	Sai số rất thấp	Sai số cao

5.10.3 Liên hệ với loss bằng 0

Trong lý thuyết, ta thường muốn loss tiến về 0. Nhưng trong thực tế, loss huấn luyện bằng 0 không phải lúc nào cũng là mục tiêu tốt.

Nếu dữ liệu có nhiễu, việc cố gắng đưa loss huấn luyện về 0 có thể làm mô hình học luôn cả nhiễu.

Ghi nhớ

Loss thấp trên training set là cần thiết, nhưng chưa đủ. Mô hình phải có khả năng tổng quát hóa tốt trên dữ liệu mới.

5.11 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
Vanishing gradient	Gradient trở nên rất nhỏ khi lan truyền ngược, khiến trọng số cập nhật rất chậm.	Gradient $\approx 10^{-9}$.
Gradient	Đạo hàm của loss theo tham số, dùng để cập nhật trọng số.	$\frac{\partial E}{\partial w}$.
Backpropagation	Quá trình lan truyền ngược sai số để tính gradient.	Tính gradient từ output về input.
Sigmoid	Hàm kích hoạt có output từ 0 đến 1.	$\sigma(x) = \frac{1}{1+e^{-x}}$.
ReLU	Hàm kích hoạt bằng 0 nếu $x \leq 0$, bằng x nếu $x > 0$.	$\max(0, x)$.
Dying ReLU	Hiện tượng neuron ReLU rơi vào vùng âm và gần như không hoạt động.	Output luôn bằng 0.
Underfitting	Mô hình quá đơn giản, chưa học được xu hướng dữ liệu.	Dùng đường thẳng cho dữ liệu cong.
Overfitting	Mô hình quá khớp dữ liệu huấn luyện, học cả nhiễu.	Đường cong đi qua mọi điểm train.
Good fitting	Mô hình học được xu hướng chung và làm việc tốt trên dữ liệu mới.	Đường cong xấp xỉ hợp lý.
Noise	Nhiều trong dữ liệu do đo lường, môi trường hoặc lỗi nhập liệu.	Cảm biến laser sai số.
Generalization	Khả năng tổng quát hóa, tức làm việc tốt trên dữ liệu mới.	Test error thấp.

Thuật ngữ	Giải thích	Ví dụ
Training data	Dữ liệu dùng để huấn luyện mô hình.	500 điểm dữ liệu đã có.
Test data	Dữ liệu dùng để kiểm tra khả năng tổng quát hóa.	Dữ liệu mới chưa học.
Classification	Bài toán phân loại vào các class rời rạc.	Chó hoặc mèo.
Regression	Bài toán dự đoán giá trị liên tục.	Dự đoán giá nhà.
Decision boundary	Đường hoặc mặt phân chia giữa các class.	Đường đỏ phân lớp.
MSE	Mean Squared Error, sai số bình phương trung bình.	Dùng trong hồi quy.
Cross entropy	Hàm mất mát phổ biến cho phân loại.	Phân loại nhiều lớp.

5.12 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

5.12.1 Các ý chính cần nhớ

1. Backpropagation về lý thuyết giúp tính gradient và cập nhật trọng số, nhưng thực tế huấn luyện có thể gặp nhiều vấn đề.
2. Vanishing gradient xảy ra khi gradient quá nhỏ, làm trọng số cập nhật rất chậm.
3. Mạng càng sâu thì gradient càng dễ bị nhỏ đi nếu phải nhân nhiều đạo hàm nhỏ.
4. Sigmoid dễ gây vanishing gradient vì đạo hàm của nó nhỏ, đặc biệt ở vùng bão hòa.
5. ReLU giúp giảm vanishing gradient ở vùng $x > 0$ vì đạo hàm bằng 1.
6. Underfitting là mô hình chưa học được xu hướng dữ liệu.
7. Overfitting là mô hình học quá sát dữ liệu huấn luyện, kể cả nhiễu.
8. Mô hình tốt là mô hình học được xu hướng chung và làm việc tốt trên dữ liệu mới.
9. Dữ liệu thực tế luôn có nhiễu, sai số đo lường và điểm bất thường.
10. Classification dự đoán nhãn rời rạc; regression dự đoán giá trị liên tục.

5.12.2 Công thức quan trọng

Cập nhật trọng số

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial E}{\partial w}$$

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Đạo hàm sigmoid

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Đạo hàm ReLU

$$\text{ReLU}'(x) = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases}$$

5.13 Review Questions – Câu hỏi ôn tập

1. Vanishing gradient là gì?
2. Khi gradient gần bằng 0, trọng số được cập nhật như thế nào?
3. Vì sao mạng sâu dễ gặp vanishing gradient hơn mạng nông?
4. Vì sao sigmoid có thể gây vanishing gradient?
5. Đạo hàm lớn nhất của sigmoid là bao nhiêu?
6. ReLU có công thức như thế nào?
7. Vì sao ReLU giúp gradient truyền tốt hơn ở vùng $x > 0$?
8. Underfitting là gì?
9. Overfitting là gì?
10. Vì sao mô hình có training loss bằng 0 chưa chắc là mô hình tốt?
11. Vì sao dữ liệu thực tế thường có nhiễu?
12. Trong bài toán phân loại 2D, đường phân lớp tốt có nhất thiết phân loại đúng mọi điểm huấn luyện không?
13. Trong hồi quy, vì sao đường cong đi qua mọi điểm dữ liệu có thể là mô hình xấu?
14. Classification khác regression như thế nào?
15. Cho ví dụ một bài toán classification và một bài toán regression.

5.14 Practice Exercises – Bài tập vận dụng

Bài tập 1: Kiểm tra vanishing gradient

Giả sử gradient qua mỗi lớp bị nhân với hệ số 0.25. Tính giá trị gradient còn lại sau 5 lớp và sau 10 lớp nếu gradient ban đầu là 1.

Lời giải gợi ý

Sau 5 lớp:

$$1 \cdot 0.25^5 = 0.0009765625$$

Sau 10 lớp:

$$1 \cdot 0.25^{10} \approx 0.0000009537$$

Gradient giảm rất nhanh khi số lớp tăng.

Bài tập 2: Tính đạo hàm sigmoid

Cho:

$$\sigma(x) = 0.9$$

Tính:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Lời giải gợi ý

$$\sigma'(x) = 0.9(1 - 0.9)$$

$$\sigma'(x) = 0.9 \cdot 0.1 = 0.09$$

Bài tập 3: Nhận diện underfitting và overfitting

Hãy xác định trường hợp nào là underfitting, good fitting hoặc overfitting:

1. Mô hình sai nhiều trên cả dữ liệu huấn luyện và dữ liệu kiểm tra.
2. Mô hình đúng gần như tuyệt đối trên dữ liệu huấn luyện nhưng sai nhiều trên dữ liệu kiểm tra.
3. Mô hình có sai số huấn luyện vừa phải và sai số kiểm tra thấp.

Lời giải gợi ý

1. Underfitting.
2. Overfitting.
3. Good fitting.

Bài tập 4: Phân biệt classification và regression

Xác định mỗi bài toán sau là classification hay regression:

1. Dự đoán giá nhà.
2. Phân loại email spam hoặc không spam.
3. Dự đoán sản lượng điện tháng tới.
4. Nhận dạng ảnh là chó, mèo hay xe hơi.
5. Dự đoán cổ phiếu ngày mai tăng, giảm hay không đổi.
6. Dự đoán giá cổ phiếu ngày mai bằng bao nhiêu.

Lời giải gợi ý

Bài toán	Loại bài toán
Dự đoán giá nhà	Regression
Email spam hoặc không spam	Classification
Dự đoán sản lượng điện	Regression
Nhận dạng ảnh chó, mèo, xe hơi	Classification
Cổ phiếu tăng, giảm hay không đổi	Classification
Giá cổ phiếu bằng bao nhiêu	Regression

Kết luận

ANN7 nhấn mạnh rằng huấn luyện mạng nơ-ron không chỉ là áp dụng backpropagation và gradient descent một cách máy móc. Trong thực tế, mạng có thể gặp các vấn đề như vanishing gradient, underfitting và overfitting.

Vanishing gradient làm cho các trọng số ở những lớp đầu cập nhật rất chậm, đặc biệt trong các mạng sâu dùng sigmoid. Đây là một trong những lý do các hàm kích hoạt như ReLU và các biến thể của ReLU trở nên phổ biến.

Underfitting xảy ra khi mô hình quá đơn giản, chưa học được xu hướng dữ liệu. Overfitting xảy ra khi mô hình quá phức tạp, học quá sát dữ liệu huấn luyện và học cả nhiễu. Mô hình tốt là mô hình không nhất thiết đúng hoàn hảo trên dữ liệu huấn luyện, nhưng phải học được xu hướng chung và tổng quát hóa tốt trên dữ liệu mới.

Chương 6

ANN8: Overfitting & Underfitting – Quá khớp và dưới khớp

Learning Objectives – Mục tiêu bài học

Bài ANN8 tiếp tục nội dung về underfitting và overfitting, nhưng tập trung hơn vào nguyên nhân và các kỹ thuật xử lý. Sau khi học xong, người học cần nắm được:

1. Nhắc lại ý nghĩa của underfitting và overfitting trong bài toán phân loại và hồi quy.
2. Hiểu nguyên nhân gây underfitting: mô hình quá đơn giản hoặc huấn luyện chưa đủ.
3. Hiểu nguyên nhân gây overfitting: mô hình quá phức tạp, dữ liệu ít hoặc dữ liệu thiếu đa dạng.
4. Biết một số hướng xử lý underfitting.
5. Biết một số hướng xử lý overfitting: đơn giản hóa mô hình, bỏ bớt đặc trưng, tăng dữ liệu, data augmentation, early stopping và dropout.
6. Hiểu ý nghĩa của train/validation/test set và nguyên tắc sử dụng từng tập.
7. Hiểu vì sao dùng sai tập test có thể làm sai lệch kết quả nghiên cứu.
8. Nắm được trực giác của dropout: không cho mạng phụ thuộc quá mạnh vào một số neuron cụ thể.

6.1 Underfitting & Overfitting Recap – Ôn lại Underfitting và Overfitting

6.1.1 Underfitting

Underfitting là tình huống mô hình chưa khớp được với dữ liệu. Nói cách khác, mô hình chưa học được xu hướng tổng quát của dữ liệu.

Thường gặp khi:

- mô hình quá đơn giản;

- số tham số quá ít;
- kiến trúc mạng quá yếu;
- thời gian huấn luyện chưa đủ;
- đặc trưng đầu vào chưa đủ thông tin.

Ghi nhớ

Underfitting nghĩa là mô hình chưa đủ khả năng học bài toán. Dù có tiếp tục dùng mô hình quá đơn giản, sai số vẫn có thể không giảm đến mức mong muốn.

6.1.2 Overfitting

Overfitting là tình huống mô hình quá khớp với dữ liệu huấn luyện. Khi đó, sai số trên tập huấn luyện có thể rất thấp, thậm chí gần bằng 0, nhưng mô hình lại làm việc kém trên dữ liệu mới.

Thường gặp khi:

- mô hình quá phức tạp so với bài toán;
- dữ liệu huấn luyện quá ít;
- dữ liệu thiếu đa dạng;
- mô hình học cả nhiễu và chi tiết bất thường;
- quá trình huấn luyện kéo dài quá lâu.

Cảnh báo

Overfitting nguy hiểm hơn underfitting ở chỗ: trên tập huấn luyện, mô hình có vẻ rất tốt vì loss thấp. Nếu không kiểm tra đúng cách, ta có thể nhầm rằng mô hình đã làm việc tốt.

6.1.3 So sánh ngắn gọn

Trường hợp	Biểu hiện	Nguyên nhân thường gặp
Underfitting	Sai nhiều trên cả train và test	Mô hình quá đơn giản hoặc train chưa đủ
Good fitting	Học được xu hướng chung, test tốt	Mô hình phù hợp với độ phức tạp bài toán
Overfitting	Train rất tốt, test kém	Mô hình quá phức tạp hoặc dữ liệu ít/thiếu đa dạng

6.2 Underfitting & Overfitting: Classification – Ví dụ Underfitting và Overfitting trong phân loại

6.2.1 Mô tả bài toán phân loại

Giả sử ta có dữ liệu 2D, ví dụ:

$$x_1 = \text{chiều cao}, \quad x_2 = \text{cân nặng}$$

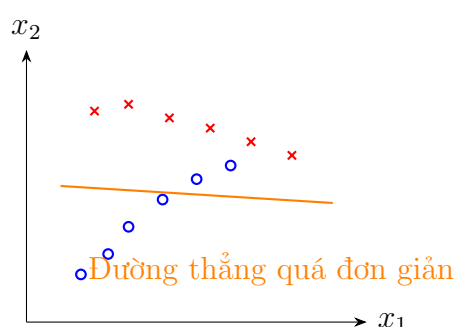
Mỗi điểm dữ liệu thuộc một trong hai class:

$$C_1 \quad \text{hoặc} \quad C_2$$

Mục tiêu là tìm một đường ranh giới để phân biệt hai class.

6.2.2 Underfitting trong phân loại

Nếu ranh giới thật cần là một đường cong, nhưng ta chỉ dùng một đường thẳng, mô hình có thể không phân chia tốt hai class.

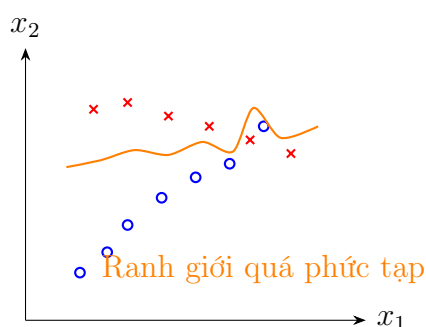


Ví dụ minh họa

Nếu dữ liệu có xu hướng cong nhưng mô hình chỉ có khả năng tạo đường thẳng, ta xoay hoặc dịch đường thẳng thế nào cũng khó phân loại tốt. Đây là ví dụ điển hình của underfitting.

6.2.3 Overfitting trong phân loại

Ngược lại, nếu mô hình quá mạnh, nó có thể tạo ra một đường ranh giới rất ngoằn ngoèo để phân loại đúng mọi điểm huấn luyện, kể cả các điểm nhiễu.



Ghi nhớ

Overfitting trong phân loại thường biểu hiện bằng ranh giới quyết định quá phức tạp, cố gắng “né” từng điểm dữ liệu thay vì học xu hướng chung.

6.3 Underfitting & Overfitting: Regression – Ví dụ Underfitting và Overfitting trong hồi quy

6.3.1 Mô tả bài toán hồi quy

Hồi quy là bài toán dự đoán giá trị liên tục.

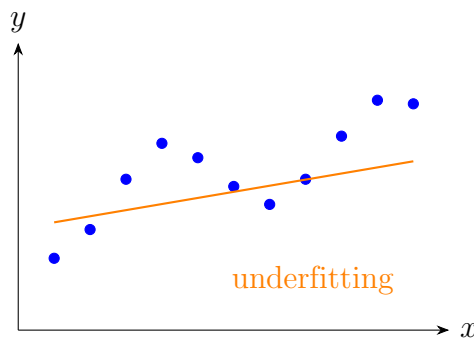
Ví dụ:

- sản lượng điện tiêu thụ trong ngày;
- giá nhà;
- nhiệt độ ngày mai;
- giá cổ phiếu ngày mai.

Ta có một tập điểm dữ liệu và cần tìm một mô hình khớp với xu hướng chung của dữ liệu.

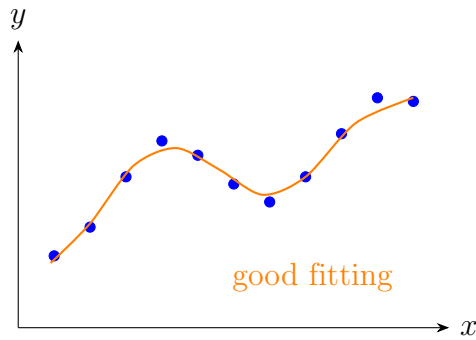
6.3.2 Underfitting trong hồi quy

Nếu dữ liệu có dạng cong nhưng ta dùng đường thẳng, mô hình không đủ khả năng mô tả dữ liệu.



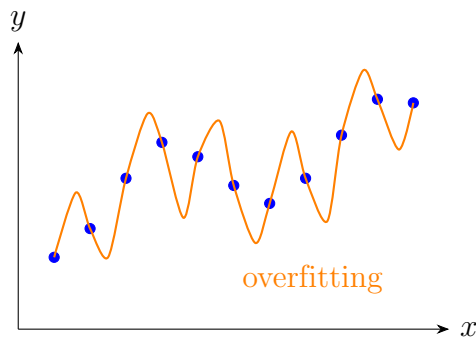
6.3.3 Good fitting trong hồi quy

Mô hình tốt không cần đi qua toàn bộ điểm dữ liệu. Nó cần xấp xỉ được xu hướng tổng quát.



6.3.4 Overfitting trong hồi quy

Nếu dùng đa thức bậc rất cao, mô hình có thể uốn lượn để đi qua gần như toàn bộ điểm dữ liệu. Sai số train có thể bằng 0, nhưng mô hình mất xu hướng tổng quát.



Cảnh báo

Trong hồi quy, việc tăng bậc đa thức quá cao có thể làm mô hình đi qua tất cả điểm training, nhưng mô hình đó chưa chắc dự đoán tốt cho điểm dữ liệu mới.

6.4 Causes of Underfitting – Nguyên nhân gây Underfitting

6.4.1 Mô hình quá đơn giản

Nguyên nhân chính của underfitting là mô hình không đủ khả năng biểu diễn bài toán. Ví dụ:

- cần đường cong nhưng lại dùng đường thẳng;
- cần mạng sâu nhưng lại dùng mạng quá nhỏ;
- cần mô hình có nhiều tham số nhưng lại dùng mô hình quá ít tham số.

Ví dụ minh họa

Nếu yêu cầu một mạng nơ-ron rất nhỏ nhận dạng ảnh chó, mèo, xe hơi, máy bay từ ảnh thực tế, mạng đó có thể không đủ khả năng học bài toán dù huấn luyện rất lâu.

6.4.2 Huấn luyện chưa đủ

Một nguyên nhân khác là mô hình đủ mạnh nhưng chưa được huấn luyện đủ lâu.

Ví dụ, một bài toán cần huấn luyện khoảng một tuần mới đạt kết quả tốt, nhưng ta dừng sau 2–3 giờ. Khi đó mô hình có thể vẫn chưa học xong và còn underfit.

Ghi nhớ

Underfitting có thể do mô hình yếu, nhưng cũng có thể do mô hình chưa được huấn luyện đủ.

6.4.3 Cách xử lý underfitting

Một số cách xử lý:

1. Dùng mô hình mạnh hơn.
2. Tăng số lớp hoặc số neuron.
3. Dùng kiến trúc phù hợp hơn với bài toán.
4. Huấn luyện lâu hơn.
5. Bổ sung đặc trưng đầu vào có ý nghĩa.

Ví dụ minh họa

Với bài toán phân loại ảnh 1000 class, một mạng nhỏ tự thiết kế đơn giản có thể không đủ. Cần dùng các kiến trúc mạnh hơn, ví dụ các mạng tích chập sâu như AlexNet hoặc các kiến trúc hiện đại hơn.

6.5 Causes of Overfitting – Nguyên nhân gây Overfitting

6.5.1 Mô hình quá phức tạp

Nếu mô hình quá mạnh so với bài toán, nó có thể học cả những chi tiết nhỏ, nhiễu hoặc điểm bất thường trong tập huấn luyện.

Ví dụ:

- bài toán chỉ cần đa thức bậc 4 nhưng ta dùng đa thức bậc 15;
- bài toán đơn giản nhưng dùng mạng quá sâu;
- số tham số lớn trong khi dữ liệu ít.

6.5.2 Dữ liệu quá ít

Nếu dữ liệu ít, mô hình dễ ghi nhớ dữ liệu huấn luyện thay vì học quy luật tổng quát.

Ví dụ minh họa

Một bài toán phân loại ảnh có 1000 class nhưng chỉ có khoảng 1000 ảnh cho mỗi class. Với bài toán ảnh phức tạp, số lượng này có thể vẫn còn ít, đặc biệt khi ảnh có nhiều biến thể về góc chụp, ánh sáng, vị trí đối tượng và nền ảnh.

6.5.3 Dữ liệu thiếu đa dạng

Không chỉ số lượng, tính đa dạng của dữ liệu cũng rất quan trọng.

Nếu tất cả ảnh chó trong tập huấn luyện đều có con chó nằm chính giữa ảnh, mô hình có thể học nhầm rằng “chó phải nằm ở giữa ảnh”. Khi test với ảnh chó lệch sang bên trái hoặc bên phải, mô hình có thể dự đoán sai.

Ghi nhớ

Dữ liệu đủ nhiều chưa chắc đã đủ tốt. Dữ liệu còn cần đủ đa dạng để mô hình học được quy luật tổng quát.

6.6 Giải pháp cho Overfitting: Đơn giản hóa mô hình

6.6.1 Giảm độ phức tạp mô hình

Nếu mô hình quá mạnh, một hướng xử lý là làm mô hình đơn giản hơn.

Ví dụ:

- giảm số lớp;
- giảm số neuron;
- giảm số tham số;
- chọn kiến trúc phù hợp hơn với độ khó của bài toán.

6.6.2 Loại bỏ bớt đặc trưng

Nếu input có quá nhiều đặc trưng, mô hình có thể học các quan hệ phụ không cần thiết. Khi đó có thể cân nhắc loại bỏ một số đặc trưng.

Ví dụ minh họa

Trong bài toán dự đoán sức khỏe, dữ liệu có thể gồm chiều cao, cân nặng, huyết áp, nhịp tim, độ tuổi, đường huyết và nhiều chỉ số khác. Nếu đưa quá nhiều đặc trưng không liên quan, mô hình có thể học các chi tiết linh tinh và overfit.

Lưu ý

Không nên bỏ đặc trưng một cách tùy tiện. Cần dựa vào hiểu biết bài toán, phân tích dữ liệu hoặc thực nghiệm để quyết định đặc trưng nào nên giữ, đặc trưng nào nên bỏ.

6.7 Giải pháp cho Overfitting: Tăng dữ liệu

6.7.1 Tại sao cần thêm dữ liệu?

Một nguyên nhân phổ biến gây overfitting là thiếu dữ liệu. Khi dữ liệu quá ít, mô hình dễ học thuộc tập huấn luyện.

Cách trực tiếp nhất là thu thập thêm dữ liệu mới, thật, độc lập và đa dạng.

6.7.2 Ví dụ ImageNet

Trong bài học, giảng viên nhắc đến một bài toán nhận dạng ảnh quy mô lớn với khoảng:

1000 class

và khoảng:

1.2 triệu ảnh

Nghe có vẻ rất nhiều, nhưng nếu chia đều cho 1000 class thì trung bình mỗi class chỉ có khoảng:

$$\frac{1,200,000}{1000} = 1200 \text{ ảnh/class}$$

Với bài toán thị giác máy tính phức tạp, con số này vẫn có thể chưa đủ.

Ghi nhớ

Trong deep learning, dữ liệu ảnh thường cần số lượng rất lớn và độ đa dạng cao. Một vài nghìn ảnh cho mỗi class có thể vẫn chưa đủ nếu bài toán phức tạp.

6.7.3 Ví dụ nhận diện khuôn mặt

Bài toán nhận diện khuôn mặt còn có thể cần lượng dữ liệu lớn hơn rất nhiều. Có những hệ thống được huấn luyện với hàng chục triệu hoặc hàng trăm triệu ảnh khuôn mặt.

Điều này giúp mô hình học được nhiều biến thể:

- góc mặt;
- ánh sáng;
- tuổi tác;

- biểu cảm;
- kiểu tóc;
- chất lượng ảnh;
- môi trường chụp.

6.8 Data Augmentation

6.8.1 Data augmentation là gì?

Data augmentation là kỹ thuật làm tăng số lượng và độ đa dạng của dữ liệu huấn luyện bằng cách biến đổi dữ liệu gốc.

Với ảnh, ta có thể tạo thêm ảnh mới từ ảnh ban đầu bằng các phép biến đổi như:

- cắt ảnh;
- tịnh tiến vùng nhìn;
- lật đối xứng;
- xoay ảnh;
- thay đổi độ sáng;
- thay đổi màu sắc;
- thay đổi kênh màu.

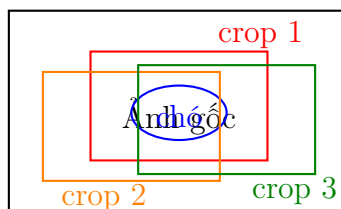
Ghi nhớ

Mục tiêu của data augmentation không phải là tạo ảnh kỳ quặc, mà là tạo thêm các tình huống hợp lý có thể xảy ra trong thực tế.

6.8.2 Cắt ảnh và dịch cửa sổ

Giả sử ta có một ảnh con chó. Nếu trong mọi ảnh huấn luyện, con chó luôn nằm chính giữa, mô hình có thể học sai rằng con chó phải nằm ở giữa ảnh.

Ta có thể cắt các vùng khác nhau của ảnh để tạo thêm nhiều ảnh huấn luyện.



Ví dụ minh họa

Từ một ảnh gốc, ta có thể lấy nhiều vùng crop khác nhau. Nhờ đó, mô hình học rằng đối tượng có thể xuất hiện ở nhiều vị trí khác nhau, không nhất thiết nằm ở trung tâm.

6.8.3 Lật đối xứng

Nếu tất cả ảnh voi trong tập huấn luyện đều quay mặt sang phải, mô hình có thể không nhận ra voi quay mặt sang trái. Khi đó, ta có thể dùng phép lật đối xứng.

ảnh voi quay phải $\rightarrow$ ảnh voi quay trái

Ví dụ minh họa

Với ảnh động vật, lật ngang thường hợp lý. Một con voi quay mặt sang trái hay sang phải vẫn là con voi.

6.8.4 Xoay ảnh

Với một số đối tượng, ta có thể xoay ảnh một góc nhỏ để tăng độ đa dạng. Ví dụ:

$$-30^\circ \leq \theta \leq 30^\circ$$

Tuy nhiên, không phải bài toán nào cũng cho phép xoay tùy ý.

Lưu ý

Cần bảo đảm ảnh sau khi augmentation vẫn có ý nghĩa. Ví dụ, xoay ảnh con voi 180 độ làm voi chống chân lên trời có thể không phù hợp nếu bài toán thực tế không bao giờ gặp tình huống đó.

6.8.5 Thay đổi màu sắc và độ sáng

Nếu tất cả ảnh hoa hồng trong tập huấn luyện có cùng màu sắc và độ sáng, mô hình có thể phụ thuộc quá nhiều vào màu đó.

Ta có thể thay đổi:

- độ sáng;
- độ tương phản;
- sắc độ màu;
- các kênh màu RGB.

Ví dụ minh họa

Hoa hồng có thể xuất hiện dưới nhiều điều kiện ánh sáng khác nhau. Nếu chỉ học ảnh hoa hồng có màu đỏ tươi và ánh sáng chuẩn, mô hình có thể nhận sai hoa hồng trong ảnh tối hơn, sáng hơn hoặc có màu lệch.

6.8.6 Augmentation phải phù hợp với bài toán

Không phải phép biến đổi nào cũng hợp lý.

Đối tượng	Biến đổi thường hợp lý	Biến đổi cần cẩn thận
Con chó	Crop, lật ngang, đổi sáng	Xoay 180 độ
Con voi	Crop, lật ngang, xoay nhẹ	Lộn ngược hoàn toàn
Cây búa	Xoay nhiều góc có thể hợp lý	Tùy bài toán cụ thể
Ngôi nhà	Đổi sáng, crop, xoay nhẹ	Lật ngược đầu
Hoa hồng	Đổi sáng, màu, crop	Biến màu quá giả tạo

Cảnh báo

Data augmentation không phải là biến đổi dữ liệu vô tội vạ. Nếu tạo ra ảnh quá giả hoặc không có khả năng xảy ra trong thực tế, mô hình có thể học sai.

6.8.7 Tác dụng về số lượng dữ liệu

Trong transcript, giảng viên nêu ví dụ: nếu một ảnh có thể biến thành 2048 ảnh khác bằng các kỹ thuật augmentation, thì tập dữ liệu tăng rất mạnh.

Nếu có:

1.2 triệu ảnh

và mỗi ảnh tạo thành:

2048 ảnh

thì tổng số ảnh xấp xỉ:

$$1,200,000 \times 2048 = 2,457,600,000$$

Tức khoảng:

2.46 tỷ ảnh

Ghi nhớ

Data augmentation vừa tăng số lượng ảnh, vừa tăng độ đa dạng của dữ liệu. Đây là một kỹ thuật rất quan trọng để giảm overfitting trong bài toán ảnh.

6.9 Train, Validation và Test Set

6.9.1 Vì sao phải chia dữ liệu?

Nếu chỉ huấn luyện và đánh giá trên cùng một tập dữ liệu, ta không biết mô hình có thực sự tổng quát hóa tốt hay chỉ đang học thuộc dữ liệu.

Do đó, dữ liệu thường được chia thành nhiều tập.

6.9.2 Chia thành hai tập

Cách đơn giản:

- training set: dùng để huấn luyện;
- test set: dùng để đánh giá sau cùng.

Ví dụ:

80% training 20% test

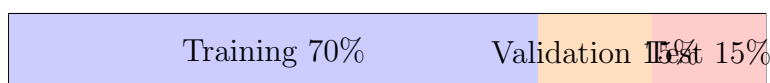
6.9.3 Chia thành ba tập

Trong thực tế, thường nên chia thành ba tập:

- training set;
- validation set;
- test set.

Ví dụ:

70% training 15% validation 15% test



6.9.4 Vai trò của training set

Training set là tập được dùng để cập nhật tham số của mô hình.

Training set → tính loss → backpropagation → cập nhật trọng số

6.9.5 Vai trò của validation set

Validation set không dùng để cập nhật trọng số trực tiếp. Nó được dùng để đánh giá mô hình trong quá trình huấn luyện.

Ta dùng validation set để:

- theo dõi mô hình có overfit không;
- chọn mô hình tốt;
- điều chỉnh hyperparameter;
- quyết định thời điểm dừng huấn luyện.

Ghi nhớ

Validation set được dùng trong quá trình huấn luyện để đánh giá và lựa chọn mô hình, nhưng không được dùng để cập nhật trọng số như training set.

6.9.6 Vai trò của test set

Test set là tập dữ liệu chỉ dùng để đánh giá cuối cùng.

Cảnh báo

Test set không được dùng trong quá trình huấn luyện, không được dùng để chỉnh mô hình, không được dùng để chọn hyperparameter. Test set chỉ được dùng khi ta đã quyết định mô hình cuối cùng.

6.9.7 Ví dụ chia dữ liệu

Giả sử có 1,000 ảnh.

Có thể chia:

700 ảnh training

150 ảnh validation

150 ảnh test

Trong quá trình làm mô hình:

- dùng 700 ảnh training để học;
- dùng 150 ảnh validation để theo dõi và điều chỉnh;
- cất riêng 150 ảnh test, chỉ dùng cuối cùng.

6.10 Nguyên tắc sử dụng Test Set

6.10.1 Không được dùng test set nhiều lần để chỉnh mô hình

Một lỗi nghiêm trọng là:

1. huấn luyện mô hình;
2. đem test set ra đánh giá;
3. thấy kết quả chưa tốt;
4. quay lại chỉnh mô hình;
5. lại đem cùng test set ra đánh giá;
6. lặp lại nhiều lần.

Cách làm này làm mô hình hoặc quá trình lựa chọn mô hình dần dần khớp với test set.

Cảnh báo

Nếu dùng test set nhiều lần để quyết định chỉnh mô hình, test set không còn là tập kiểm tra độc lập nữa. Kết quả báo cáo sau đó có thể đánh giá sai năng lực thật của mô hình.

6.10.2 Vì sao đây là lỗi nghiêm trọng?

Test set có vai trò mô phỏng dữ liệu mới trong thực tế. Nếu ta liên tục nhìn vào test set để chỉnh mô hình, ta đã làm mất tính độc lập của nó.

Khi đó, kết quả trên test set không còn phản ánh khả năng tổng quát hóa thật.

6.10.3 Liên hệ với nghiên cứu khoa học

Trong nghiên cứu khoa học, nếu báo cáo kết quả trên một test set đã bị dùng nhiều lần để chỉnh mô hình, kết quả có thể gây hiểu lầm rằng mô hình tốt hơn thực tế.

Điều này có thể bị xem là sai nguyên tắc nghiên cứu, thậm chí là gian lận nếu cố ý.

Ghi nhớ

Test set là bài kiểm tra cuối cùng. Nếu đã dùng test set để chỉnh mô hình, cần có một test set độc lập mới để đánh giá cuối cùng.

6.10.4 Nếu lỡ dùng test set thì làm sao?

Nếu đã lỡ dùng test set trong quá trình phát triển mô hình, cách xử lý đúng là:

- thu thập thêm dữ liệu mới để làm test set độc lập;
- hoặc chia lại dữ liệu và tạo một test set mới chưa từng dùng;
- báo cáo rõ quy trình đánh giá.

6.11 Cross-validation

6.11.1 Ý tưởng

Khi dữ liệu ít, việc tách riêng một test set cố định có thể làm lãng phí dữ liệu. Một hướng xử lý là kiểm tra chéo, tiếng Anh là *cross-validation*.

Ý tưởng chung:

- chia dữ liệu thành nhiều phần;
- mỗi lần dùng một phần khác nhau làm tập kiểm tra;
- huấn luyện và đánh giá nhiều lần;
- tổng hợp kết quả.

6.11.2 Ví dụ K-fold cross-validation

Giả sử chia dữ liệu thành 5 phần:

$$D_1, D_2, D_3, D_4, D_5$$

Ta thực hiện 5 lần:

Lần	Dùng để train	Dùng để kiểm tra
1	D_2, D_3, D_4, D_5	D_1
2	D_1, D_3, D_4, D_5	D_2
3	D_1, D_2, D_4, D_5	D_3
4	D_1, D_2, D_3, D_5	D_4
5	D_1, D_2, D_3, D_4	D_5

Sau đó lấy trung bình kết quả đánh giá.

Lưu ý

Trong transcript, cross-validation chỉ được nhắc như một hướng liên quan khi cần chia lại và đánh giá dữ liệu nhiều lần. Nội dung chi tiết về cross-validation có thể học sâu hơn ở phần machine learning.

6.12 Early Stopping – Early Stopping

6.12.1 Early stopping là gì?

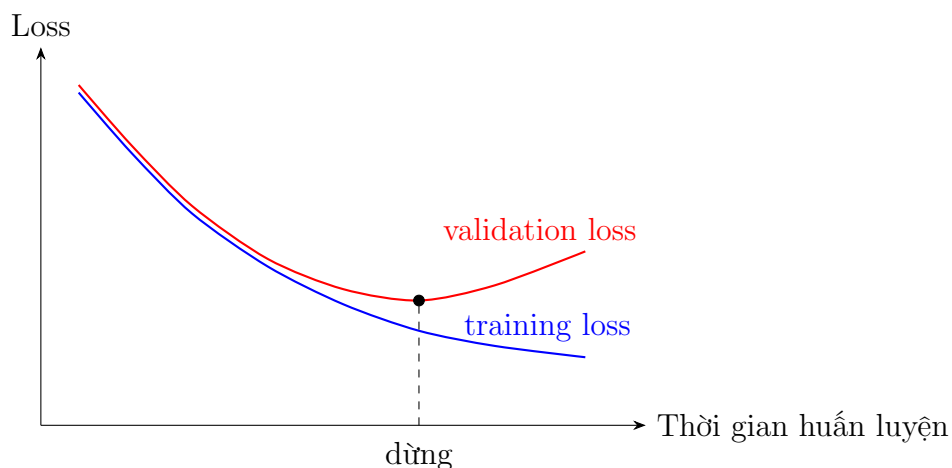
Early stopping, hay dừng sớm, là kỹ thuật dừng quá trình huấn luyện khi mô hình bắt đầu có dấu hiệu overfitting.

Ta theo dõi loss trên:

- training set;
- validation set.

Thông thường, training loss sẽ giảm dần. Nhưng nếu validation loss giảm đến một mức nào đó rồi bắt đầu tăng trở lại, đó là dấu hiệu mô hình đang overfit.

6.12.2 Minh họa



6.12.3 Ý nghĩa

Giả sử ban đầu dự định huấn luyện 100,000 bước. Nhưng sau 70,000 bước, validation loss bắt đầu tăng, trong khi training loss vẫn giảm. Khi đó nên dừng huấn luyện và giữ mô hình tại thời điểm validation loss tốt nhất.

Ghi nhớ

Early stopping giúp tránh tình trạng mô hình tiếp tục học quá sát training set sau khi đã đạt khả năng tổng quát hóa tốt nhất trên validation set.

6.12.4 Không phải cứ early stopping là mô hình tốt

Dừng sớm chỉ giúp giảm overfitting. Nó không đảm bảo mô hình cuối cùng chắc chắn tốt.

Nếu mô hình quá yếu, dữ liệu sai, hoặc bài toán chưa được thiết kế đúng, early stopping vẫn không cứu được mô hình.

6.13 Dropout – Dropout

6.13.1 Dropout là gì?

Dropout là kỹ thuật trong quá trình huấn luyện, tại mỗi thời điểm, ta ngẫu nhiên loại bỏ một số neuron khỏi mạng với một xác suất nào đó.

Ví dụ, nếu xác suất giữ lại neuron là:

$$p = 0.5$$

thì trung bình khoảng một nửa số neuron được giữ lại, một nửa bị tạm thời bỏ qua trong bước huấn luyện đó.

Ghi nhớ

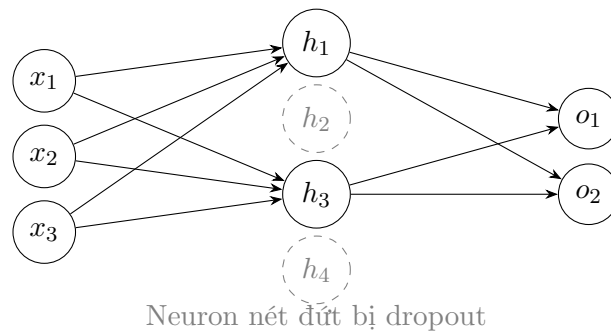
Dropout chỉ dùng trong quá trình training. Khi test hoặc sử dụng mô hình thật, ta dùng toàn bộ mạng.

6.13.2 Một thời điểm trong dropout là gì?

Trong bài học, một “thời điểm” có thể hiểu là một lần đưa một mẫu dữ liệu vào mạng. Ví dụ:

- đưa ảnh thứ nhất vào, dropout một số neuron, tính xuôi, tính ngược, cập nhật;
- đưa ảnh thứ hai vào, dropout một nhóm neuron khác, tính xuôi, tính ngược, cập nhật;
- tiếp tục như vậy cho các mẫu tiếp theo.

6.13.3 Minh họa dropout



6.13.4 Vì sao dropout giúp giảm overfitting?

Có hai cách hiểu trực quan.

6.13.5 Góc nhìn ensemble

Nếu mỗi lần dropout tạo ra một cấu hình mạng khác nhau, thì trong quá trình huấn luyện, ta giống như đang huấn luyện rất nhiều mạng con khác nhau.

Khi test, ta dùng toàn bộ mạng, có thể xem như sự kết hợp của nhiều mạng con đã được huấn luyện.

Ví dụ minh họa

Thay vì huấn luyện một mạng duy nhất, dropout khiến mô hình trải qua rất nhiều phiên bản mạng con. Điều này gần giống tư tưởng ensemble: nhiều mô hình cùng đóng góp để tạo ra kết quả ổn định hơn.

6.13.6 Góc nhìn giảm phụ thuộc

Nếu không có dropout, một neuron có thể phụ thuộc quá mạnh vào một vài neuron khác. Khi dropout ngẫu nhiên loại bỏ neuron, mạng buộc phải học cách sử dụng thông tin từ nhiều nguồn khác nhau.

Điều này giúp mạng mạnh mẽ hơn và ít phụ thuộc vào một đường truyền thông tin cụ thể.

Ghi nhớ

Dropout buộc các neuron phải “học cách làm việc với nhiều đồng đội khác nhau”, thay vì phụ thuộc quá mức vào một vài neuron cố định.

6.13.7 Dropout và robust model

Một mô hình robust là mô hình vẫn hoạt động tốt trong nhiều tình huống khác nhau.

Dropout giúp tăng tính robust vì trong quá trình train, mạng thường xuyên phải làm việc trong điều kiện thiếu một số neuron. Nhờ đó, mạng học được biểu diễn phân tán hơn và ít nhạy với từng neuron riêng lẻ.

6.13.8 Vì sao không dùng dropout khi test?

Trong training, mỗi lần chỉ một phần neuron hoạt động. Nhưng khi test, ta muốn dùng toàn bộ năng lực của mạng đã học.

Vì vậy:

Training: dùng dropout

Testing: không dùng dropout

6.13.9 Vấn đề scale tín hiệu

Giả sử trong training, mỗi neuron được giữ lại với xác suất:

$$p = 0.5$$

Khi đó, trung bình một neuron ở lớp sau chỉ nhận được khoảng một nửa số tín hiệu so với khi dùng toàn bộ mạng.

Nếu khi test ta bật toàn bộ neuron mà không điều chỉnh gì, tổng tín hiệu đầu vào có thể tăng lên so với lúc training.

Theo cách trình bày trong bài học, để làm cho mức tín hiệu giữa training và testing tương thích hơn, output của neuron có thể được nhân với hệ số p khi dùng toàn bộ mạng.

$$out_{\text{test}} = p \cdot out$$

Ví dụ nếu:

$$p = 0.5$$

thì khi test:

$$out_{\text{test}} = 0.5 \cdot out$$

Lưu ý

Trong nhiều thư viện hiện đại, người ta thường dùng biến thể gọi là inverted dropout: scale ngay trong lúc training, nên khi test không cần nhân thêm. Ý chính cần nhớ trong bài này là phải giữ mức tín hiệu giữa training và testing nhất quán.

6.13.10 Xác suất dropout

Giá trị p là một hyperparameter. Ví dụ phổ biến:

$$p = 0.5$$

Nhưng không có một giá trị tốt nhất cho mọi bài toán. Trong thực tế, người ta thường dùng các giá trị đã được nghiên cứu trước hoặc thử nghiệm để chọn giá trị phù hợp.

Lưu ý

Không nên hiểu rằng $p = 0.5$ luôn là tốt nhất. Nó chỉ là một lựa chọn phổ biến trong nhiều tình huống.

6.14 Tổng hợp các kỹ thuật xử lý Overfitting

6.14.1 Các kỹ thuật chính

Kỹ thuật	Ý nghĩa
Đơn giản hóa mô hình	Giảm số lớp, số neuron hoặc độ phức tạp để tránh học nhiều
Feature selection	Bỏ bớt đặc trưng không cần thiết hoặc gây nhiễu
Thêm dữ liệu	Thu thập thêm dữ liệu thật, độc lập và đa dạng
Data augmentation	Tạo thêm dữ liệu hợp lý từ dữ liệu gốc
Train/validation/test split	Đánh giá đúng khả năng tổng quát hóa
Early stopping	Dừng khi validation loss bắt đầu xấu đi
Dropout	Ngẫu nhiên loại neuron trong training để giảm phụ thuộc và tăng robust
Cross-validation	Đánh giá ổn định hơn khi dữ liệu ít

6.14.2 Kỹ thuật nào quan trọng nhất?

Không có một kỹ thuật duy nhất giải quyết mọi trường hợp. Trong thực tế thường phải kết hợp nhiều kỹ thuật.

Ví dụ với bài toán ảnh:

- dùng mô hình đủ mạnh;

- dùng data augmentation;
- chia train/validation/test đúng;
- theo dõi validation loss;
- dùng early stopping;
- dùng dropout hoặc các kỹ thuật regularization khác.

6.15 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
Underfitting	Mô hình chưa khớp dữ liệu, chưa học được xu hướng tổng quát.	Dùng đường thẳng cho dữ liệu cong.
Overfitting	Mô hình quá khớp training data, học cả nhiễu.	Đa thức bậc cao đi qua mọi điểm.
Training set	Tập dữ liệu dùng để cập nhật tham số mô hình.	70% dữ liệu.
Validation set	Tập dùng để đánh giá trong quá trình huấn luyện và chọn mô hình.	15% dữ liệu.
Test set	Tập chỉ dùng để đánh giá cuối cùng.	15% dữ liệu còn lại.
Data augmentation	Tạo thêm dữ liệu từ dữ liệu gốc bằng biến đổi hợp lý.	Crop, flip, rotate ảnh.
Crop	Cắt một vùng trong ảnh gốc để tạo ảnh mới.	Cắt vùng chứa con chó.
Reflection/Flip	Lật ảnh theo chiều ngang hoặc chiều dọc.	Lật ảnh voi quay phải thành quay trái.
Rotation	Xoay ảnh một góc nào đó.	Xoay ảnh búa 30 độ.
Color augmentation	Thay đổi màu sắc, độ sáng hoặc kênh màu.	Làm ảnh hoa sáng hơn.
Early stopping	Dừng sớm khi validation loss bắt đầu tăng.	Dừng sau 5 ngày thay vì 7 ngày.
Dropout	Ngẫu nhiên loại bỏ một số neuron trong quá trình training.	Giữ lại neuron với xác suất 0.5.
Ensemble	Kết hợp nhiều mô hình để có kết quả ổn định hơn.	Trung bình kết quả nhiều mạng.
Robust	Mạnh mẽ, ít bị ảnh hưởng bởi thay đổi nhỏ hoặc thiếu một phần thông tin.	Mạng không phụ thuộc một neuron.
Hyperparameter	Tham số do người thiết kế chọn trước hoặc điều chỉnh.	Dropout rate, số lớp.
Cross-validation	Kiểm tra chéo, chia dữ liệu nhiều lần để đánh giá ổn định hơn.	5-fold cross-validation.

Thuật ngữ	Giải thích	Ví dụ
Feature selection	Chọn hoặc loại bỏ đặc trưng đầu vào.	Bỏ chỉ số không liên quan.
Generalization	Khả năng tổng quát hóa trên dữ liệu mới.	Test accuracy cao.

6.16 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

6.16.1 Các ý chính cần nhớ

1. Underfitting xảy ra khi mô hình quá đơn giản hoặc chưa được huấn luyện đủ.
2. Overfitting xảy ra khi mô hình quá phức tạp, dữ liệu ít hoặc thiếu đa dạng.
3. Underfitting dễ nhận ra hơn vì loss thường còn cao.
4. Overfitting nguy hiểm hơn vì training loss thấp khiến ta dễ nhầm rằng mô hình tốt.
5. Để giảm underfitting, có thể dùng mô hình mạnh hơn hoặc huấn luyện lâu hơn.
6. Để giảm overfitting, có thể đơn giản hóa mô hình, bỏ bớt đặc trưng, thêm dữ liệu hoặc dùng data augmentation.
7. Data augmentation giúp tăng cả số lượng và độ đa dạng dữ liệu.
8. Augmentation phải tạo ra dữ liệu hợp lý với bài toán thực tế.
9. Dữ liệu nên được chia thành training, validation và test set.
10. Test set chỉ được dùng để đánh giá cuối cùng.
11. Dùng test set nhiều lần để chỉnh mô hình là sai nguyên tắc.
12. Early stopping dùng validation loss để dừng trước khi mô hình overfit.
13. Dropout ngẫu nhiên loại neuron trong training để giảm phụ thuộc và tăng khả năng tổng quát hóa.
14. Dropout chỉ dùng trong training, không dùng trong testing.

6.16.2 Công thức và con số cần nhớ

Tỷ lệ chia dữ liệu ví dụ

70% training 15% validation 15% test

Số ảnh trung bình mỗi class

$$\frac{1,200,000}{1000} = 1200 \text{ ảnh/class}$$

Data augmentation 2048 lần

$$1,200,000 \times 2048 = 2,457,600,000$$

$$\approx 2.46 \text{ tỷ ảnh}$$

Dropout scale theo xác suất giữ neuron

$$out_{\text{test}} = p \cdot out$$

6.17 Review Questions – Câu hỏi ôn tập

1. Underfitting là gì?
2. Overfitting là gì?
3. Vì sao overfitting nguy hiểm hơn underfitting trong quá trình đánh giá mô hình?
4. Nêu hai nguyên nhân gây underfitting.
5. Nêu ba nguyên nhân gây overfitting.
6. Vì sao dữ liệu nhiều nhưng thiếu đa dạng vẫn có thể gây overfitting?
7. Data augmentation là gì?
8. Nêu ba kỹ thuật data augmentation cho ảnh.
9. Vì sao không nên xoay ảnh một cách tùy tiện?
10. Training set dùng để làm gì?
11. Validation set dùng để làm gì?
12. Test set dùng để làm gì?
13. Vì sao không được dùng test set nhiều lần để chỉnh mô hình?
14. Early stopping hoạt động dựa trên dấu hiệu nào?
15. Dropout là gì?
16. Dropout được dùng trong training hay testing?
17. Vì sao dropout có thể giúp giảm overfitting?
18. Xác suất giữ neuron $p = 0.5$ có phải luôn là tốt nhất không?

6.18 Practice Exercises – Bài tập vận dụng

Bài tập 1: Nhận diện lỗi mô hình

Xác định trường hợp sau là underfitting hay overfitting:

1. Training loss cao, validation loss cao.
2. Training loss rất thấp, validation loss cao.
3. Mô hình dùng đường thẳng cho dữ liệu có dạng cong.
4. Mô hình dùng đa thức bậc rất cao và đi qua mọi điểm training.

Lời giải gợi ý

1. Underfitting.
2. Overfitting.
3. Underfitting.
4. Overfitting.

Bài tập 2: Chia dữ liệu

Bạn có 10,000 ảnh. Hãy chia theo tỷ lệ:

70% training, 15% validation, 15% test

Tính số ảnh trong mỗi tập.

Lời giải gợi ý

Training:

$$10,000 \times 0.70 = 7,000$$

Validation:

$$10,000 \times 0.15 = 1,500$$

Test:

$$10,000 \times 0.15 = 1,500$$

Bài tập 3: Data augmentation

Một tập dữ liệu có 5,000 ảnh. Nếu mỗi ảnh được augmentation thành 100 ảnh, tổng số ảnh sau augmentation là bao nhiêu?

Lời giải gợi ý

$$5,000 \times 100 = 500,000$$

Vậy sau augmentation có 500,000 ảnh.

Bài tập 4: Chọn augmentation phù hợp

Với từng bài toán, hãy cho biết phép augmentation nào hợp lý hơn:

1. Nhận dạng cây búa trong ảnh.
2. Nhận dạng ngôi nhà bình thường.
3. Nhận dạng hoa hồng trong nhiều điều kiện ánh sáng.
4. Nhận dạng con voi trong ảnh tự nhiên.

Lời giải gợi ý

1. Có thể xoay ảnh nhiều góc nếu cây búa có thể xuất hiện ở nhiều hướng.
2. Có thể crop, đổi sáng, xoay nhẹ; không nên lật ngược nhà 180 độ nếu thực tế không có nhà lộn ngược.
3. Nên đổi sáng, màu sắc, tương phản.
4. Có thể crop, lật ngang, xoay nhẹ; không nên lật ngược voi nếu bài toán không có tình huống đó.

Bài tập 5: Dropout

Một layer có 100 neuron. Nếu dùng dropout với xác suất giữ lại:

$$p = 0.5$$

Trung bình mỗi lần training có bao nhiêu neuron được giữ lại?

Lời giải gợi ý

$$100 \times 0.5 = 50$$

Trung bình có 50 neuron được giữ lại.

Kết luận

ANN8 giải thích sâu hơn nguyên nhân và cách xử lý underfitting, overfitting. Underfitting thường xảy ra khi mô hình quá đơn giản hoặc chưa được huấn luyện đủ. Overfitting thường xảy ra khi mô hình quá phức tạp, dữ liệu quá ít hoặc thiếu đa dạng.

Để xử lý underfitting, ta có thể dùng mô hình mạnh hơn hoặc huấn luyện lâu hơn. Để xử lý overfitting, ta có nhiều kỹ thuật như đơn giản hóa mô hình, bỏ bớt đặc trưng, tăng dữ liệu, data augmentation, chia đúng train/validation/test, early stopping và dropout.

Một điểm rất quan trọng của bài học là nguyên tắc sử dụng test set. Test set chỉ dùng để đánh giá cuối cùng, không dùng để chỉnh mô hình. Nếu dùng test set nhiều lần trong quá trình phát triển, kết quả đánh giá cuối cùng sẽ không còn đáng tin cậy.

Cuối cùng, dropout được trình bày như một kỹ thuật rất phổ biến để giảm overfitting. Nó ngẫu nhiên loại bỏ neuron trong quá trình training, buộc mạng học biểu diễn mạnh mẽ hơn và ít phụ thuộc vào một số neuron cụ thể.

Chương 7

ANN9: Optimization Algorithms – Thuật toán tối ưu

Learning Objectives – Mục tiêu bài học

Bài ANN9 quay lại phần Gradient Descent và làm rõ ba phương pháp cập nhật tham số phổ biến khi huấn luyện mạng nơ-ron:

1. Stochastic Gradient Descent.
2. Batch Gradient Descent.
3. Mini-batch Gradient Descent.

Sau khi học xong, người học cần nắm được:

1. Sự khác nhau giữa cập nhật theo từng mẫu, cập nhật theo toàn bộ dữ liệu và cập nhật theo từng nhóm nhỏ dữ liệu.
2. Vì sao Stochastic Gradient Descent có đường đi nhiễu nhưng có thể tránh cực trị địa phương tốt hơn.
3. Vì sao Batch Gradient Descent ổn định hơn nhưng có thể dễ mắc kẹt ở cực trị địa phương.
4. Vì sao Mini-batch Gradient Descent là giải pháp trung gian và được dùng rất phổ biến.
5. Hiểu các khái niệm: sample, data point, feature, batch, batch size, iteration và epoch.
6. Biết cách tính số lần cập nhật trong một epoch tùy theo batch size.
7. Hiểu yêu cầu bài tập: tự lập trình và so sánh ba phương pháp Gradient Descent trên hàm không lồi hai biến.

7.1 Gradient Descent Recap – Nhắc lại Gradient Descent trong huấn luyện ANN

7.1.1 Mục tiêu của huấn luyện

Trong mạng nơ-ron, ta có hàm mất mát:

$$L(W)$$

Trong đó W đại diện cho toàn bộ tham số của mạng, bao gồm:

- trọng số;
- bias.

Mục tiêu của huấn luyện là tìm bộ tham số W sao cho loss nhỏ nhất:

$$W^* = \arg \min_W L(W)$$

Để cập nhật tham số, ta cần gradient:

$$\frac{\partial L}{\partial W}$$

hoặc viết tổng quát hơn:

$$\nabla_W L(W)$$

Công thức cập nhật cơ bản:

$$W_{\text{new}} = W_{\text{old}} - \eta \nabla_W L(W_{\text{old}})$$

Trong đó:

- η là learning rate;
- $\nabla_W L(W)$ là gradient của loss theo tham số;
- dấu trừ thể hiện việc đi theo hướng làm loss giảm.

Ghi nhớ

Backpropagation giúp tính gradient. Gradient Descent dùng gradient đó để cập nhật trọng số và bias.

7.1.2 Một vấn đề cần làm rõ

Trong các bài trước, ta thường nói đơn giản rằng:

đưa dữ liệu vào $\rightarrow$ tính loss $\rightarrow$ tính gradient $\rightarrow$ cập nhật

Nhưng trong thực tế, ta có rất nhiều dữ liệu. Câu hỏi là:

Mỗi lần cập nhật, ta dùng bao nhiêu mẫu dữ liệu?

Có ba lựa chọn chính:

1. Dùng một mẫu dữ liệu.
2. Dùng toàn bộ tập dữ liệu.
3. Dùng một nhóm nhỏ dữ liệu.

Ba lựa chọn này tạo ra ba phương pháp:

1. Stochastic Gradient Descent.
2. Batch Gradient Descent.
3. Mini-batch Gradient Descent.

7.2 Stochastic Gradient Descent (SGD) – Stochastic Gradient Descent

7.2.1 Ý tưởng

Stochastic Gradient Descent, viết tắt là SGD, là phương pháp cập nhật tham số sau mỗi mẫu dữ liệu.

Giả sử tập dữ liệu gồm:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

Với SGD, ta làm như sau:

1. Đưa một mẫu (x_i, y_i) vào mạng.
2. Tính output dự đoán.
3. Tính loss trên mẫu đó.
4. Tính gradient dựa trên mẫu đó.
5. Cập nhật tham số ngay.

Công thức:

$$W_{t+1} = W_t - \eta \nabla_W L_i(W_t)$$

Trong đó L_i là loss trên mẫu dữ liệu thứ i .

Ghi nhớ

SGD cập nhật sau từng sample. Vì vậy, nếu tập dữ liệu có N sample thì trong một epoch, SGD cập nhật N lần.

7.2.2 Ví dụ trực quan

Giả sử ta có các ảnh:

ảnh chó, ảnh mèo, ảnh xe hơi, ảnh máy bay

SGD hoạt động như sau:

1. Đưa ảnh chó vào, tính sai số, cập nhật mạng.
2. Đưa ảnh mèo vào, tính sai số, cập nhật mạng.
3. Đưa ảnh xe hơi vào, tính sai số, cập nhật mạng.
4. Đưa ảnh máy bay vào, tính sai số, cập nhật mạng.

Mỗi ảnh tạo ra một lần cập nhật.

7.2.3 SGD và online learning

Vì SGD cập nhật theo từng mẫu, nó có thể dùng trong tình huống online learning.

Online learning là tình huống mô hình vừa được sử dụng, vừa tiếp tục được cập nhật khi có dữ liệu mới.

Ví dụ minh họa

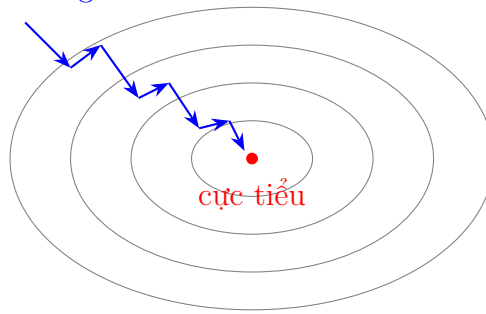
Một hệ thống nhận diện ảnh đang hoạt động. Khi thu thập thêm một ảnh mới đã có nhãn, ta có thể đưa ảnh đó vào để cập nhật mô hình ngay. Đây là kiểu cập nhật phù hợp với tình huống online learning.

7.2.4 Đặc điểm: đường đi nhiễu

Do mỗi lần chỉ dùng một mẫu dữ liệu, gradient của SGD thường rất nhiễu. Một mẫu riêng lẻ chưa chắc đại diện cho toàn bộ tập dữ liệu.

Vì vậy đường đi của SGD khi tìm cực tiểu thường không mượt. Nó có thể nhảy qua nhảy lại.

SGD: đường đi nhiều



7.2.5 Ưu điểm của SGD

- Cập nhật nhanh sau từng mẫu.
- Có thể dùng cho online learning.
- Không cần đưa toàn bộ dữ liệu vào bộ nhớ cùng lúc.
- Do có nhiễu, đôi khi có thể thoát khỏi cực trị địa phương tốt hơn.

7.2.6 Nhược điểm của SGD

- Đường đi không ổn định.
- Gradient của từng mẫu có thể lệch nhiều so với gradient thật của toàn bộ dữ liệu.
- Có thể dao động quanh điểm cực tiểu.
- Khó tận dụng tối đa khả năng tính toán song song của phần cứng hiện đại.

Lưu ý

SGD không “xấu”. Nó chỉ nhiễu. Chính tính nhiễu này vừa là nhược điểm trong việc đi đến cực tiểu nhanh, vừa là ưu điểm trong việc tránh bị kẹt ở một số cực trị địa phương.

7.3 Batch Gradient Descent – Batch Gradient Descent

7.3.1 Ý tưởng

Batch Gradient Descent là phương pháp dùng toàn bộ tập dữ liệu để tính gradient rồi mới cập nhật một lần.

Với tập dữ liệu:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

Ta tính loss trung bình:

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(W)$$

Gradient toàn bộ tập dữ liệu là:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(W)$$

Sau đó cập nhật:

$$W_{t+1} = W_t - \eta \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(W_t)$$

Ghi nhớ

Batch Gradient Descent không cập nhật sau từng mẫu. Nó phải đi qua toàn bộ dữ liệu, tính gradient trung bình, rồi mới cập nhật một lần.

7.3.2 Ví dụ

Giả sử có 10,000 ảnh.

Với Batch Gradient Descent:

1. Đưa cả 10,000 ảnh qua mô hình.
2. Tính loss và gradient cho từng ảnh.
3. Lấy trung bình các gradient.
4. Cập nhật trọng số một lần.

7.3.3 Vì sao lấy trung bình gradient?

Hàm loss tổng thể thường được định nghĩa là trung bình loss trên toàn bộ dữ liệu:

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(W)$$

Do đó, gradient của loss tổng thể là:

$$\nabla_W L(W) = \nabla_W \left(\frac{1}{N} \sum_{i=1}^N L_i(W) \right)$$

Vì đạo hàm của tổng bằng tổng đạo hàm:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(W)$$

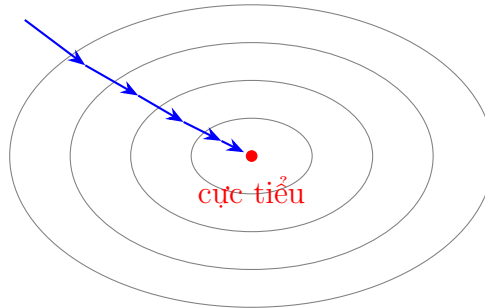
Như vậy, việc lấy trung bình gradient không chỉ là do “không biết mẫu nào quan trọng hơn”. Bản chất toán học là ta đang tối ưu loss trung bình trên toàn bộ tập dữ liệu.

7.3.4 Đặc điểm: ổn định hơn SGD

Vì dùng toàn bộ dữ liệu, gradient của Batch Gradient Descent gần với hướng giảm thật của hàm loss tổng thể hơn.

Đường đi thường mượt hơn, ít nhiễu hơn.

Batch: đường đi ổn định hơn



7.3.5 Ưu điểm của Batch Gradient Descent

- Hướng cập nhật ổn định hơn.
- Thể hiện đúng hơn hướng giảm của loss tổng thể.
- Ít nhiễu hơn SGD.

7.3.6 Nhược điểm của Batch Gradient Descent

- Không phù hợp với online learning.
- Cần xử lý toàn bộ dữ liệu trước khi cập nhật.
- Với dữ liệu rất lớn, một lần cập nhật có thể rất tốn thời gian và bộ nhớ.
- Do quá ổn định, nếu rơi vào cực trị địa phương thì có thể khó thoát ra.

Lưu ý

Batch Gradient Descent rất hiệu quả trong việc đi theo hướng dốc nhất của toàn bộ hàm loss, nhưng chính sự ổn định này có thể làm nó dễ mắc kẹt ở cực trị địa phương hơn so với phương pháp nhiễu như SGD.

7.4 Contour Lines & Local Minima – Đường đồng mức và cực trị địa phương

7.4.1 Đường đồng mức là gì?

Trong bài học, giảng viên nhắc đến hình có nhiều vòng tròn hoặc đường cong bao quanh một điểm. Đó là các đường đồng mức, tiếng Anh là *contour lines*.

Với hàm hai biến:

$$f(x, y)$$

đường đồng mức là tập các điểm có cùng giá trị hàm:

$$f(x, y) = c$$

Trong đó c là một hằng số.

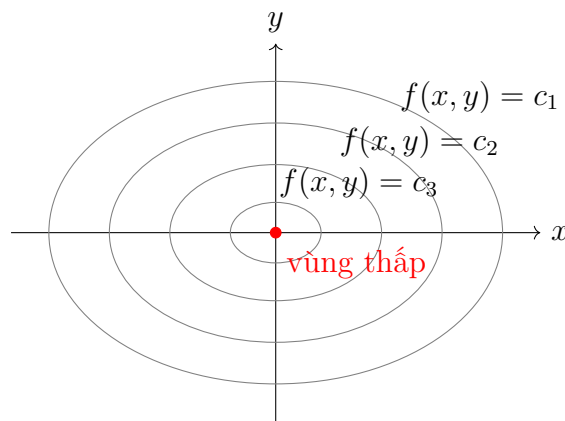
Ghi nhớ

Các điểm nằm trên cùng một đường đồng mức có cùng giá trị hàm. Nhưng hai đường đồng mức khác nhau thường tương ứng với hai giá trị hàm khác nhau.

7.4.2 Không gian hai biến và giá trị hàm

Nếu ta có hai biến x và y , miền tìm kiếm là không gian 2D.

Giá trị $f(x, y)$ có thể xem như chiều cao tại điểm (x, y) . Vì vậy, khi vẽ contour, ta biểu diễn bề mặt 3D bằng các đường trên mặt phẳng 2D.



7.4.3 Cực trị địa phương

Nếu hàm loss không lồi, nó có thể có nhiều cực trị địa phương.

Cực trị địa phương là điểm thấp nhất trong một vùng nhỏ, nhưng không nhất thiết là thấp nhất toàn bộ.

$$\text{local minimum} \neq \text{global minimum}$$

Cảnh báo

Trong tối ưu hóa mạng nơ-ron, một khó khăn lớn là thuật toán có thể mắc kẹt ở cực trị địa phương hoặc vùng nghiệm không tốt.

7.4.4 Vì sao SGD có thể tránh cực trị địa phương?

SGD dùng gradient từ từng mẫu dữ liệu. Vì mỗi mẫu chỉ phản ánh một phần dữ liệu, hướng cập nhật có nhiễu.

Nhiều này làm đường đi không mượt, nhưng đôi khi nó giúp mô hình “nhảy” ra khỏi một vùng cực trị địa phương nông.

Ví dụ minh họa

Hình dung ta đang đi xuống một thung lũng nhỏ. Nếu đường đi quá ổn định, ta có thể dừng lại trong thung lũng đó. Nếu đường đi có dao động, ta có thể bị đẩy ra khỏi thung lũng nhỏ và tiếp tục đi đến vùng thấp hơn.

7.5 Mini-batch Gradient Descent

7.5.1 Ý tưởng

Mini-batch Gradient Descent là phương pháp trung gian giữa SGD và Batch Gradient Descent.

Thay vì dùng:

- một mẫu duy nhất như SGD;
- toàn bộ dữ liệu như Batch Gradient Descent;

ta dùng một nhóm nhỏ dữ liệu gọi là mini-batch.

Giả sử một mini-batch có B mẫu:

$$\mathcal{B} = \{(x_1, y_1), (x_2, y_2), \dots, (x_B, y_B)\}$$

Gradient trên mini-batch:

$$\nabla_W L_{\mathcal{B}}(W) = \frac{1}{B} \sum_{i=1}^B \nabla_W L_i(W)$$

Cập nhật:

$$W_{t+1} = W_t - \eta \frac{1}{B} \sum_{i=1}^B \nabla_W L_i(W_t)$$

7.5.2 Batch size

Batch size là số lượng mẫu trong một mini-batch.

Ký hiệu:

$$B = \text{batch size}$$

Nếu:

$$B = 1$$

thì mini-batch trở thành SGD.

Nếu:

$$B = N$$

thì mini-batch trở thành Batch Gradient Descent.
Thông thường, batch size có thể nằm trong khoảng:

$$8 \leq B \leq 256$$

hoặc lớn hơn, tùy vào phần cứng, bộ nhớ GPU và bài toán cụ thể.

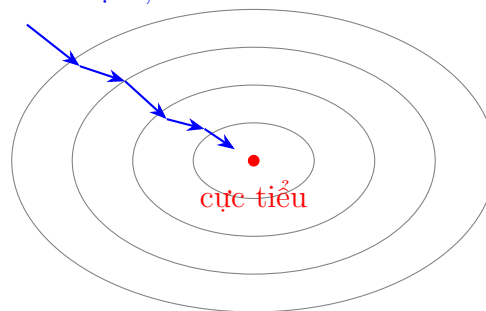
Ghi nhớ

Mini-batch Gradient Descent là giải pháp cân bằng giữa độ nhiễu của SGD và độ ổn định của Batch Gradient Descent.

7.5.3 Đường đi của mini-batch

Mini-batch ít nhiễu hơn SGD vì nó lấy trung bình gradient trên nhiều mẫu. Nhưng nó vẫn còn một lượng nhiễu nhất định, nên có thể tránh cực trị địa phương tốt hơn Batch Gradient Descent.

Mini-batch: vừa ổn định, vừa còn nhiễu



7.5.4 Vì sao mini-batch được dùng phổ biến?

Mini-batch Gradient Descent có nhiều ưu điểm thực tế:

- cập nhật thường xuyên hơn Batch Gradient Descent;
- ít nhiễu hơn SGD;
- tận dụng tốt tính toán song song trên GPU;
- không cần đưa toàn bộ dữ liệu vào bộ nhớ cùng lúc;
- thường hội tụ hiệu quả trong thực tế.

Ghi nhớ

Trong huấn luyện deep learning hiện đại, mini-batch gần như là lựa chọn mặc định.

7.6 So sánh ba phương pháp Gradient Descent

7.6.1 Bảng so sánh

Tiêu chí	SGD	Batch GD	Mini-batch GD
Số mẫu mỗi lần cập nhật	1	Toàn bộ dữ liệu	Một nhóm nhỏ
Batch size	$B = 1$	$B = N$	$1 < B < N$
Độ nhiễu	Cao	Thấp	Trung bình
Độ ổn định	Thấp	Cao	Vừa phải
Khả năng online learning	Có	Không thuận lợi	Có thể, tùy cách dùng
Khả năng thoát local minimum	Tốt hơn do nhiễu	Kém hơn	Khá tốt
Tận dụng GPU	Không tối ưu	Có thể nặng bộ nhớ	Tốt
Mức phổ biến hiện nay	Ít dùng nguyên bản	Ít dùng với dữ liệu lớn	Rất phổ biến

7.6.2 Ba thái cực và giải pháp trung gian

SGD và Batch Gradient Descent là hai thái cực:

$$B = 1 \quad \text{và} \quad B = N$$

Mini-batch nằm giữa:

$$1 < B < N$$

Ví dụ minh họa

Có thể hình dung như việc học và chơi. Chỉ học mà không chơi thì không cân bằng; chỉ chơi mà không học cũng không ổn. Cách tốt thường nằm ở giữa. Mini-batch cũng vậy: nó cân bằng giữa tính nhiễu của SGD và tính ổn định của Batch Gradient Descent.

7.7 Các thuật ngữ quan trọng

7.7.1 Sample, data point và example

Một mẫu dữ liệu có thể được gọi bằng nhiều tên:

- sample;
- example;
- data point;
- instance.

Ví dụ trong bài toán dự đoán sức khỏe, một sample có thể là thông tin của một người:

$$x = \begin{bmatrix} \text{chiều cao} \\ \text{cân nặng} \\ \text{huyết áp} \\ \text{nhịp tim} \\ \text{tuổi} \end{bmatrix}$$

7.7.2 Feature

Feature là một đặc trưng hoặc thuộc tính của mẫu dữ liệu.
Ví dụ:

- chiều cao;
- cân nặng;
- nhóm máu;
- huyết áp;
- độ tuổi;
- giá trị pixel trong ảnh.

Nếu một người được mô tả bằng chiều cao, cân nặng và huyết áp, thì ta có 3 feature.

7.7.3 Batch

Batch là một nhóm mẫu dữ liệu được xử lý cùng nhau trong một lần cập nhật hoặc một lần tính toán.

Nếu batch có 32 mẫu:

$$B = 32$$

thì batch size bằng 32.

7.7.4 Batch size

Batch size là số sample trong một batch.

$$\text{batch size} = B$$

Các trường hợp đặc biệt:

$$B = 1 \Rightarrow \text{SGD}$$

$$B = N \Rightarrow \text{Batch Gradient Descent}$$

$$1 < B < N \Rightarrow \text{Mini-batch Gradient Descent}$$

7.7.5 Iteration

Iteration là một lần cập nhật tham số.

Nếu dùng mini-batch, mỗi mini-batch thường tạo ra một iteration.

$$1 \text{ mini-batch} \rightarrow 1 \text{ lần cập nhật} \rightarrow 1 \text{ iteration}$$

7.7.6 Epoch

Một epoch là một lần mô hình đi qua toàn bộ tập dữ liệu huấn luyện.

Nếu tập dữ liệu có N mẫu, thì một epoch nghĩa là tất cả N mẫu đã được dùng một lần để huấn luyện.

$$1 \text{ epoch} = \text{đi qua toàn bộ training set một lần}$$

Ghi nhớ

Epoch không đồng nghĩa với một lần cập nhật. Trong một epoch có thể có rất nhiều lần cập nhật, tùy batch size.

7.8 Cách tính số iteration trong một epoch

7.8.1 Công thức tổng quát

Giả sử:

$$N = \text{số mẫu dữ liệu}$$

$$B = \text{batch size}$$

Số iteration trong một epoch xấp xỉ:

$$\text{iterations per epoch} = \left\lceil \frac{N}{B} \right\rceil$$

Nếu N chia hết cho B :

$$\text{iterations per epoch} = \frac{N}{B}$$

7.8.2 Trường hợp SGD

Với SGD:

$$B = 1$$

Do đó:

$$\text{iterations per epoch} = \frac{N}{1} = N$$

Nếu có 2.4 tỷ ảnh:

$$N = 2,400,000,000$$

thì SGD cần:

$$2,400,000,000$$

lần cập nhật để hoàn thành một epoch.

7.8.3 Trường hợp Batch Gradient Descent

Với Batch Gradient Descent:

$$B = N$$

Do đó:

$$\text{iterations per epoch} = \frac{N}{N} = 1$$

Tức là mỗi epoch chỉ cập nhật một lần.

7.8.4 Trường hợp Mini-batch Gradient Descent

Giả sử:

$$N = 2,400,000,000$$

và:

$$B = 1,000,000$$

Số iteration trong một epoch:

$$\frac{2,400,000,000}{1,000,000} = 2400$$

Như vậy, trong một epoch, ta có 2400 lần cập nhật.

Ví dụ minh họa

Nếu dùng mini-batch size 1 triệu ảnh, mỗi lần đưa 1 triệu ảnh vào, cập nhật một lần. Sau 2400 mini-batch như vậy, toàn bộ 2.4 tỷ ảnh đã được dùng một lượt. Khi đó kết thúc một epoch.

7.8.5 Nhiều epoch

Thông thường, một epoch là chưa đủ để mô hình hội tụ.

Ta thường cần:

$$10, 50, 100, 1000$$

epoch hoặc nhiều hơn, tùy bài toán.

Ghi nhớ

Sau một epoch, mô hình mới chỉ nhìn thấy toàn bộ dữ liệu một lần. Với bài toán phức tạp, cần lặp lại nhiều epoch để mô hình học tốt hơn.

7.9 Tại sao Batch GD là hướng dốc nhất còn SGD thì nhiều?

7.9.1 Hàm loss tổng thể

Giả sử loss tổng thể là:

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(W)$$

Gradient thật của loss tổng thể là:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(W)$$

Đây là hướng tăng nhanh nhất của loss tổng thể. Do đó:

$$-\nabla_W L(W)$$

là hướng giảm nhanh nhất của loss tổng thể.

7.9.2 Batch Gradient Descent

Batch Gradient Descent dùng đúng:

$$\nabla_W L(W)$$

nên nó đi theo hướng giảm dốc nhất của toàn bộ hàm loss.

$$W_{t+1} = W_t - \eta \nabla_W L(W_t)$$

7.9.3 Stochastic Gradient Descent

SGD chỉ dùng một mẫu:

$$\nabla_W L_i(W)$$

Gradient này là gradient của một mẫu, không phải gradient của toàn bộ tập dữ liệu. Vì vậy, hướng cập nhật:

$$-\nabla_W L_i(W)$$

chỉ là hướng giảm loss của mẫu thứ i , không nhất thiết là hướng giảm nhanh nhất của loss tổng thể.

Lưu ý

SGD nhiễu vì mỗi sample có thể kéo tham số theo một hướng hơi khác nhau. Nhưng khi xét trung bình qua nhiều mẫu, các hướng này có xu hướng xấp xỉ gradient tổng thể.

7.9.4 Mini-batch Gradient Descent

Mini-batch dùng trung bình gradient của một nhóm nhỏ:

$$\frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_W L_i(W)$$

Nó gần với gradient tổng thể hơn SGD, nhưng vẫn chưa chính xác bằng Batch Gradient Descent.

Do đó:

SGD: nhiễu nhiều

Mini-batch: nhiễu vừa

Batch: ít nhiễu nhất

7.10 Bài tập được giao trong bài ANN9

7.10.1 Nội dung bài tập

Trong bài học, giảng viên giao bài tập yêu cầu người học tự lập trình để hiểu rõ “ruột” của ba phương pháp Gradient Descent.

Yêu cầu chính:

1. Tìm ít nhất hai hàm không lồi.
2. Hàm phải là hàm hai biến.
3. Giá trị hàm là một số vô hướng.
4. Hàm nên có nhiều cực trị địa phương để dễ minh họa.
5. Áp dụng ba phương pháp:
 - (a) Stochastic Gradient Descent.
 - (b) Batch Gradient Descent.
 - (c) Mini-batch Gradient Descent.
6. Với mỗi phương pháp, khảo sát ít nhất 5 điểm khởi tạo khác nhau.
7. Tự điều chỉnh learning rate và các tham số liên quan.
8. Vẽ hoặc ghi lại đường đi của thuật toán.
9. So sánh khả năng hội tụ và khả năng mắc kẹt ở cực trị địa phương.

7.10.2 Gợi ý hàm không lồi

Có thể dùng các hàm hai biến sau để minh họa.

7.10.3 Hàm Himmelblau

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

Hàm này có nhiều cực tiểu địa phương và thường được dùng để minh họa tối ưu hóa.

7.10.4 Hàm Rastrigin hai biến

$$f(x, y) = 20 + x^2 - 10 \cos(2\pi x) + y^2 - 10 \cos(2\pi y)$$

Hàm này có nhiều vùng lồi lõm và nhiều cực trị địa phương.

Lưu ý

Khi tự lập trình, cần tính đạo hàm riêng theo x và y , sau đó cập nhật theo công thức Gradient Descent.

7.10.5 Gradient tổng quát cho hàm hai biến

Với hàm:

$$f(x, y)$$

gradient là:

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Cập nhật:

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial f}{\partial x}(x_t, y_t) \\ \frac{\partial f}{\partial y}(x_t, y_t) \end{bmatrix}$$

7.10.6 Ý nghĩa của việc tự lập trình

Nếu chỉ dùng thư viện có sẵn, người học có thể không hiểu chính xác:

- gradient được tính như thế nào;
- batch size ảnh hưởng ra sao;
- vì sao SGD nhiều;
- vì sao Batch GD ổn định;
- vì sao Mini-batch là giải pháp trung gian;

- learning rate làm thay đổi đường đi như thế nào.

Ghi nhớ

Tự lập trình bài toán nhỏ giúp hiểu bản chất của thuật toán tốt hơn nhiều so với chỉ gọi một hàm có sẵn trong thư viện.

7.11 Pseudocode cho ba phương pháp

7.11.1 Pseudocode SGD

```
Khởi tạo W ngẫu nhiên
For epoch = 1 to num_epochs:
    Xáo trộn dữ liệu
    For từng sample (x_i, y_i):
        Tính loss L_i(W)
        Tính gradient grad_i
        W = W - learning_rate * grad_i
```

7.11.2 Pseudocode Batch Gradient Descent

```
Khởi tạo W ngẫu nhiên
For epoch = 1 to num_epochs:
    grad_total = 0
    For từng sample (x_i, y_i):
        Tính gradient grad_i
        grad_total = grad_total + grad_i
    grad_average = grad_total / N
    W = W - learning_rate * grad_average
```

7.11.3 Pseudocode Mini-batch Gradient Descent

```
Khởi tạo W ngẫu nhiên
For epoch = 1 to num_epochs:
    Xáo trộn dữ liệu
    Chia dữ liệu thành các mini-batch
    For từng mini-batch B:
        grad_batch = 0
        For từng sample trong B:
            Tính gradient grad_i
            grad_batch = grad_batch + grad_i
        grad_average = grad_batch / batch_size
        W = W - learning_rate * grad_average
```

Lưu ý

Trong các thư viện deep learning, việc tính gradient thường được tự động hóa bằng automatic differentiation. Tuy nhiên, về bản chất, các bước phía trên vẫn là nền tảng.

7.12 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
Gradient Descent	Phương pháp cập nhật tham số theo hướng ngược gradient để giảm loss.	$W = W - \eta \nabla L$.
Stochastic Gradient Descent	Cập nhật tham số sau từng sample.	Batch size bằng 1.
Batch Gradient Descent	Dùng toàn bộ dữ liệu để tính gradient rồi cập nhật một lần.	Batch size bằng N .
Mini-batch Gradient Descent	Dùng một nhóm nhỏ dữ liệu để tính gradient rồi cập nhật.	Batch size 32.
Gradient Loss function	Vector đạo hàm của loss theo tham số. Hàm mất mát cần tối thiểu hóa.	$\nabla_W L$. Cross entropy, MSE.
Weight	Trọng số trong mạng nơ-ron.	w_1, w_2 .
Bias	Tham số cộng thêm vào tổng có trọng số.	b .
Sample	Một mẫu dữ liệu.	Một ảnh con chó.
Example	Cách gọi khác của sample.	Một dòng dữ liệu.
Data point	Một điểm dữ liệu.	Một người với chiều cao, cân nặng.
Feature	Đặc trưng mô tả sample.	Chiều cao, huyết áp.
Batch	Một nhóm sample được xử lý cùng nhau.	32 ảnh.
Batch size	Số sample trong một batch.	$B = 64$.
Iteration	Một lần cập nhật tham số.	Một mini-batch tạo một iteration.
Epoch	Một lượt đi qua toàn bộ training set.	50 epoch.
Online learning	Vừa sử dụng mô hình vừa cập nhật khi có dữ liệu mới.	Cập nhật sau mỗi ảnh mới.
Contour line	Đường đồng mức, gồm các điểm có cùng giá trị hàm.	$f(x, y) = c$.
Local minimum	Cực tiểu địa phương.	Hố nhỏ trên bề mặt loss.
Global minimum	Cực tiểu toàn cục.	Điểm có loss nhỏ nhất toàn miền.
Non-convex function	Hàm không lồi, có thể có nhiều cực trị địa phương.	Rastrigin.

Thuật ngữ	Giải thích	Ví dụ
Learning rate	Hệ số quyết định bước cập nhật lớn hay nhỏ.	$\eta = 0.001$.

7.13 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

7.13.1 Các ý chính cần nhớ

1. Gradient Descent dùng gradient để cập nhật trọng số và bias.
2. SGD cập nhật sau từng sample.
3. Batch Gradient Descent cập nhật sau khi dùng toàn bộ dữ liệu.
4. Mini-batch Gradient Descent cập nhật sau từng nhóm nhỏ dữ liệu.
5. SGD có đường đi nhiễu nhưng có thể tránh cực trị địa phương tốt hơn.
6. Batch Gradient Descent ổn định hơn nhưng có thể mắc kẹt ở cực trị địa phương.
7. Mini-batch cân bằng giữa SGD và Batch Gradient Descent.
8. Batch size bằng 1 tương ứng với SGD.
9. Batch size bằng toàn bộ dữ liệu tương ứng với Batch Gradient Descent.
10. Một epoch là một lần đi qua toàn bộ tập dữ liệu.
11. Một iteration là một lần cập nhật tham số.
12. Trong một epoch có thể có nhiều iteration.
13. Với N mẫu và batch size B , số iteration trong một epoch là $\lceil N/B \rceil$.
14. Hiện nay, mini-batch được dùng rất phổ biến trong huấn luyện deep learning.

7.13.2 Công thức quan trọng

Loss trung bình

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(W)$$

Gradient toàn bộ dữ liệu

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(W)$$

SGD

$$W_{t+1} = W_t - \eta \nabla_W L_i(W_t)$$

Batch Gradient Descent

$$W_{t+1} = W_t - \eta \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(W_t)$$

Mini-batch Gradient Descent

$$W_{t+1} = W_t - \eta \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_W L_i(W_t)$$

Số iteration trong một epoch

$$\text{iterations per epoch} = \left\lceil \frac{N}{B} \right\rceil$$

7.14 Review Questions – Câu hỏi ôn tập

1. Stochastic Gradient Descent là gì?
2. Batch Gradient Descent là gì?
3. Mini-batch Gradient Descent là gì?
4. Batch size là gì?
5. Nếu batch size bằng 1 thì phương pháp trở thành gì?
6. Nếu batch size bằng toàn bộ số mẫu dữ liệu thì phương pháp trở thành gì?
7. Vì sao SGD có đường đi nhiễu?
8. Vì sao Batch Gradient Descent ổn định hơn SGD?
9. Vì sao SGD có thể tránh cực trị địa phương tốt hơn?
10. Vì sao Batch Gradient Descent có thể dễ mắc kẹt ở cực trị địa phương?
11. Vì sao Mini-batch Gradient Descent được dùng phổ biến?
12. Đường đồng mức là gì?
13. Một epoch là gì?
14. Một iteration là gì?

15. Với 10,000 mẫu và batch size 100, một epoch có bao nhiêu iteration?
16. Với 2.4 tỷ mẫu và batch size 1 triệu, một epoch có bao nhiêu iteration?
17. Vì sao cần huấn luyện nhiều epoch?
18. Vì sao khi dùng Batch hoặc Mini-batch ta lấy trung bình gradient?

7.15 Practice Exercises – Bài tập vận dụng

Bài tập 1: Tính số iteration

Một tập dữ liệu có:

$$N = 50,000$$

mẫu. Tính số iteration trong một epoch cho các trường hợp:

1. SGD.
2. Batch Gradient Descent.
3. Mini-batch với $B = 100$.
4. Mini-batch với $B = 250$.

Lời giải gợi ý

SGD:

$$B = 1$$

$$\frac{50,000}{1} = 50,000$$

Batch Gradient Descent:

$$B = N = 50,000$$

$$\frac{50,000}{50,000} = 1$$

Mini-batch $B = 100$:

$$\frac{50,000}{100} = 500$$

Mini-batch $B = 250$:

$$\frac{50,000}{250} = 200$$

Bài tập 2: Nhận diện phương pháp

Hãy xác định phương pháp tương ứng:

1. Mỗi lần đưa một ảnh vào rồi cập nhật ngay.
2. Đưa toàn bộ dữ liệu vào, lấy trung bình gradient rồi cập nhật.
3. Mỗi lần đưa 64 ảnh vào rồi cập nhật.

Lời giải gợi ý

1. Stochastic Gradient Descent.
2. Batch Gradient Descent.
3. Mini-batch Gradient Descent.

Bài tập 3: So sánh ưu nhược điểm

Điền vào bảng sau:

Phương pháp	Ưu điểm	Nhược điểm
SGD		
Batch GD		
Mini-batch GD		

Lời giải gợi ý

Phương pháp	Ưu điểm	Nhược điểm
SGD	Cập nhật nhanh, có thể tránh local minimum	Nhiều, dao động mạnh
Batch GD	Ổn định, đúng hướng gradient tổng thể	Tốn bộ nhớ, dễ kẹt local minimum
Mini-batch GD	Cân bằng, tận dụng GPU tốt	Cần chọn batch size phù hợp

Bài tập 4: Bài tập lập trình theo yêu cầu

Hãy tự chọn ít nhất hai hàm không lồi hai biến. Với mỗi hàm:

1. Tính đạo hàm riêng theo x và y .
2. Lập trình ba phương pháp:
 - (a) SGD.
 - (b) Batch Gradient Descent.
 - (c) Mini-batch Gradient Descent.

3. Khảo sát ít nhất 5 điểm khởi tạo.
4. Thử nhiều learning rate khác nhau.
5. Vẽ đường đi của nghiệm trên contour plot.
6. Nhận xét phương pháp nào ổn định hơn, phương pháp nào dễ thoát local minimum hơn.

Gợi ý hàm

$$f_1(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

$$f_2(x, y) = 20 + x^2 - 10 \cos(2\pi x) + y^2 - 10 \cos(2\pi y)$$

Kết luận

ANN9 làm rõ ba cách cập nhật tham số trong Gradient Descent. Stochastic Gradient Descent cập nhật sau từng mẫu dữ liệu, nên đường đi có nhiều nhiễu nhưng có khả năng tránh cực trị địa phương tốt hơn. Batch Gradient Descent dùng toàn bộ dữ liệu để tính gradient trung bình, nên ổn định hơn và đi theo hướng giảm tốt hơn của loss tổng thể, nhưng có thể tốn kém và dễ bị kẹt trong cực trị địa phương. Mini-batch Gradient Descent là giải pháp trung gian, dùng một nhóm nhỏ dữ liệu cho mỗi lần cập nhật.

Trong thực tế huấn luyện deep learning, mini-batch được dùng rất phổ biến vì nó cân bằng giữa hiệu quả tính toán, độ ổn định và khả năng tổng quát. Bài học cũng nhấn mạnh các thuật ngữ quan trọng như sample, feature, batch size, iteration và epoch. Đặc biệt, cần phân biệt rõ: một epoch là một lượt đi qua toàn bộ dữ liệu, còn một iteration là một lần cập nhật tham số.

Chương 8

Softmax Function – Hàm Softmax

Learning Objectives – Mục tiêu bài học

Bài học này tập trung vào hàm *softmax*, một hàm thường được sử dụng ở lớp output của mạng nơ-ron trong bài toán phân loại nhiều lớp.

Sau khi học xong, người học cần nắm được:

1. Vì sao bài toán phân loại nhiều lớp cần output dạng vector xác suất.
2. Phân biệt bộ phân loại thông thường và bộ phân loại dựa trên xác suất.
3. Hiểu cách mã hóa nhãn bằng one-hot vector.
4. Hiểu vai trò của vector z , còn gọi là logits, trước khi đưa qua softmax.
5. Biết công thức softmax và cách tính từng phần tử.
6. Hiểu vì sao softmax biến một vector số thực bất kỳ thành một phân bố xác suất.
7. Hiểu softmax liên hệ với argmax như thế nào.
8. Làm được ví dụ tính softmax với 4 class.

8.1 Softmax in Classification – Bối cảnh: Softmax trong bài toán phân loại

8.1.1 Bài toán phân loại ảnh

Trong bài toán phân loại ảnh, input là một tấm ảnh và output là nhãn của ảnh đó. Ví dụ:

Ảnh đầu vào $\longrightarrow$ chó, mèo, điều, xe hơi, ...

Với bài toán có nhiều class, hệ thống phải xác định ảnh thuộc class nào.

Ví dụ minh họa

Trong một bài toán phân loại ảnh có 1000 class, mỗi ảnh đầu vào phải được gán vào một trong 1000 lớp. Nếu ảnh là một con chó, hệ thống cần dự đoán đúng class tương ứng với chó.

8.1.2 Ví dụ hệ thống phân loại ảnh lớn

Một hệ thống phân loại ảnh có thể nhận input là ảnh màu kích thước:

$$224 \times 224$$

Sau khi đi qua nhiều lớp xử lý, hệ thống tạo ra 1000 output tương ứng với 1000 class.

$$\text{Ảnh } 224 \times 224 \longrightarrow \text{mạng nơ-ron} \longrightarrow 1000 \text{ output}$$

Ghi nhớ

Với bài toán phân loại 1000 class, lớp output thường có 1000 phần tử. Mỗi phần tử tương ứng với một class.

8.2 Two Types of Classifiers – Hai kiểu bộ phân loại

8.2.1 Bộ phân loại thông thường

Bộ phân loại thông thường là một hàm nhận input x và trả về trực tiếp một nhãn dự đoán:

$$\hat{y} = f(x)$$

Trong đó:

- x là input;
- $\hat{y}$ là nhãn dự đoán;
- $\hat{y}$ thuộc tập các class có thể có.

Nếu có 1000 class:

$$\hat{y} \in \{1, 2, 3, \dots, 1000\}$$

Ví dụ minh họa

Nếu input là ảnh con mèo, bộ phân loại có thể trả về:

$$\hat{y} = 2$$

Trong đó số 2 được quy ước là class “mèo”.

8.2.2 Bộ phân loại dựa trên xác suất

Bộ phân loại dựa trên xác suất không chỉ trả về một nhãn duy nhất. Thay vào đó, nó trả về xác suất tương ứng với từng class.

Ta có thể viết:

$$P(y|x)$$

Nghĩa là xác suất class y xảy ra khi biết input x .

Nếu có K class, output là một vector:

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix}$$

Trong đó:

$$y_i = P(\text{class } i|x)$$

Các phần tử phải thỏa:

$$0 \leq y_i \leq 1$$

và:

$$\sum_{i=1}^K y_i = 1$$

Ghi nhớ

Bộ phân loại xác suất cho ta biết mô hình tin mỗi class với mức độ bao nhiêu, thay vì chỉ trả về một nhãn duy nhất.

8.2.3 Ví dụ với 4 class

Giả sử bài toán có 4 class:

{chó, mèo, điều, xe hơi}

Một output xác suất có thể là:

$$y = \begin{bmatrix} 0.20 \\ 0.35 \\ 0.05 \\ 0.40 \end{bmatrix}$$

Ý nghĩa:

- xác suất là chó: 20%;
- xác suất là mèo: 35%;

- xác suất là điều: 5%;
- xác suất là xe hơi: 40%.

Dự đoán cuối cùng là class có xác suất lớn nhất:

$$\hat{y} = \arg \max_i y_i$$

Trong ví dụ này:

$$\hat{y} = \text{xe hơi}$$

vì 0.40 là giá trị lớn nhất.

8.3 One-hot Encoding – One-hot Encoding

8.3.1 Vì sao cần mã hóa nhãn?

Trong học có giám sát, mỗi ảnh đi kèm với một label đúng. Ví dụ:

ảnh con chó $\rightarrow$ chó

Để mạng nơ-ron có thể tính toán, label dạng chữ thường được mã hóa thành vector số.

Một cách mã hóa phổ biến là one-hot encoding.

8.3.2 One-hot vector

One-hot vector là vector có đúng một phần tử bằng 1, các phần tử còn lại bằng 0. Với 4 class:

{chó, mèo, điều, xe hơi}

ta có thể mã hóa:

$$\text{chó} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{mèo} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{điều} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\text{xe hơi} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Ghi nhớ

Nếu bài toán có K class, one-hot vector sẽ có K phần tử.

8.3.3 Bảng one-hot encoding

Class	One-hot vector	Ý nghĩa
Chó	$[1, 0, 0, 0]^T$	Class thứ nhất
Mèo	$[0, 1, 0, 0]^T$	Class thứ hai
Điêu	$[0, 0, 1, 0]^T$	Class thứ ba
Xe hơi	$[0, 0, 0, 1]^T$	Class thứ tư

8.4 Raw Network Output Issue – Vấn đề ở output thô của mạng

8.4.1 Vector logits

Trước khi đi qua softmax, mạng nơ-ron thường tạo ra một vector số thực:

$$z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_K \end{bmatrix}$$

Vector này thường được gọi là *logits*.

Các giá trị trong logits có thể là bất kỳ số thực nào:

$$z_i \in \mathbb{R}$$

Chúng có thể âm, dương, lớn hơn 1 hoặc nhỏ hơn 0.

8.4.2 Ví dụ output thô

Giả sử mạng cho output thô:

$$z = \begin{bmatrix} 0.8 \\ 1.4 \\ -0.8 \\ 0.2 \end{bmatrix}$$

Vector này chưa phải là xác suất vì:

- có phần tử âm: -0.8 ;
- tổng các phần tử không bằng 1;
- các phần tử không nằm trong khoảng $[0, 1]$.

Tổng:

$$0.8 + 1.4 - 0.8 + 0.2 = 1.6$$

Nhưng phân bố xác suất cần tổng bằng 1 và mọi phần tử không âm.

Cảnh báo

Không thể xem trực tiếp logits là xác suất. Cần một hàm biến đổi logits thành vector xác suất. Hàm đó chính là softmax.

8.5 Softmax Function – Hàm Softmax

8.5.1 Công thức tổng quát

Với vector logits:

$$z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_K \end{bmatrix}$$

Hàm softmax tạo ra vector:

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix}$$

Trong đó:

$$y_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Hay viết đầy đủ:

$$y_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_K}}$$

$$y_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + \dots + e^{z_K}}$$

...

$$y_K = \frac{e^{z_K}}{e^{z_1} + e^{z_2} + \dots + e^{z_K}}$$

Ghi nhớ

Mẫu số của softmax giống nhau cho tất cả các phần tử. Nó là tổng của e^{z_i} trên toàn bộ các class.

8.5.2 Vì sao softmax tạo ra xác suất?

Softmax có hai tính chất quan trọng.
Thứ nhất, mọi phần tử đều dương:

$$e^{z_i} > 0$$

nên:

$$y_i > 0$$

Thứ hai, tổng các phần tử bằng 1:

$$\sum_{i=1}^K y_i = \sum_{i=1}^K \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Vì mẫu số là hằng số chung:

$$\sum_{i=1}^K y_i = \frac{\sum_{i=1}^K e^{z_i}}{\sum_{j=1}^K e^{z_j}} = 1$$

Do đó, output của softmax có thể được xem như một phân bố xác suất trên các class.

Ghi nhớ

Softmax biến vector số thực bất kỳ thành vector xác suất: các phần tử dương và tổng bằng 1.

8.6 Softmax Calculation Example – Ví dụ tính Softmax

8.6.1 Dữ liệu ví dụ

Giả sử bài toán có 4 class:

{chó, mèo, điều, xe hơi}

Mạng tạo ra logits:

$$z = \begin{bmatrix} 0.8 \\ 1.4 \\ -0.8 \\ 0.2 \end{bmatrix}$$

Ta cần tính:

$$y = \text{softmax}(z)$$

8.6.2 Bước 1: Tính các giá trị mũ

$$e^{z_1} = e^{0.8} \approx 2.2255$$

$$e^{z_2} = e^{1.4} \approx 4.0552$$

$$e^{z_3} = e^{-0.8} \approx 0.4493$$

$$e^{z_4} = e^{0.2} \approx 1.2214$$

8.6.3 Bước 2: Tính mẫu số chung

$$S = e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}$$

$$S = 2.2255 + 4.0552 + 0.4493 + 1.2214$$

$$S \approx 7.9514$$

8.6.4 Bước 3: Tính từng xác suất

$$y_1 = \frac{e^{z_1}}{S} = \frac{2.2255}{7.9514} \approx 0.280$$

$$y_2 = \frac{e^{z_2}}{S} = \frac{4.0552}{7.9514} \approx 0.510$$

$$y_3 = \frac{e^{z_3}}{S} = \frac{0.4493}{7.9514} \approx 0.057$$

$$y_4 = \frac{e^{z_4}}{S} = \frac{1.2214}{7.9514} \approx 0.154$$

Vậy:

$$y = \begin{bmatrix} 0.280 \\ 0.510 \\ 0.057 \\ 0.154 \end{bmatrix}$$

Làm tròn theo transcript:

$$y \approx \begin{bmatrix} 0.28 \\ 0.51 \\ 0.06 \\ 0.15 \end{bmatrix}$$

8.6.5 Diễn giải kết quả

Output có thể hiểu là:

- class 1: khoảng 28%;
- class 2: khoảng 51%;
- class 3: khoảng 6%;
- class 4: khoảng 15%.

Class có xác suất cao nhất là class thứ 2.

$$\arg \max_i y_i = 2$$

Do đó, mô hình dự đoán class thứ 2.

Ví dụ minh họa

Nếu class thứ 2 là “mèo”, thì mô hình dự đoán ảnh thuộc class mèo, với xác suất khoảng 51%.

8.7 Softmax & Argmax – Softmax và Argmax

8.7.1 Hàm max

Với vector:

$$z = \begin{bmatrix} 0.8 \\ 1.4 \\ -0.8 \\ 0.2 \end{bmatrix}$$

Hàm max trả về giá trị lớn nhất:

$$\max(z) = 1.4$$

Giá trị lớn nhất nằm ở vị trí thứ 2.

8.7.2 Hàm argmax

Hàm argmax không trả về giá trị lớn nhất, mà trả về vị trí của giá trị lớn nhất.

$$\arg \max(z) = 2$$

Vì phần tử lớn nhất của z là:

$$z_2 = 1.4$$

Ghi nhớ

Max trả về giá trị lớn nhất. Argmax trả về vị trí hoặc chỉ số của giá trị lớn nhất.

8.7.3 Argmax dưới dạng one-hot

Nếu argmax là 2, ta có thể biểu diễn kết quả bằng one-hot vector:

$$\text{onehot}(\arg \max(z)) = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Đây là vector class thứ 2.

8.7.4 Softmax xấp xỉ argmax theo nghĩa mềm

Với cùng vector:

$$z = \begin{bmatrix} 0.8 \\ 1.4 \\ -0.8 \\ 0.2 \end{bmatrix}$$

Argmax one-hot là:

$$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Softmax là:

$$\begin{bmatrix} 0.28 \\ 0.51 \\ 0.06 \\ 0.15 \end{bmatrix}$$

Ta thấy softmax vẫn cho phần tử thứ 2 lớn nhất, nhưng không ép các phần tử khác về 0 tuyệt đối.

Ghi nhớ

Softmax có thể được hiểu như một phiên bản “mềm” của argmax. Nó vẫn nhấn mạnh phần tử lớn nhất, nhưng giữ lại thông tin xác suất cho các class khác.

8.8 Why Softmax? – Vì sao gọi là Softmax?

8.8.1 Ý nghĩa tên gọi

Tên *softmax* gồm hai phần:

- *soft*: mềm, trơn, mượt;
- *max*: liên quan đến việc chọn giá trị lớn nhất.

Ý tưởng trực quan là softmax tạo ra một phiên bản mềm của phép chọn max hoặc argmax.

8.8.2 Max cứng và softmax mềm

Hàm max hoặc argmax là lựa chọn cứng:

$$\arg \max(z)$$

Nó chỉ chọn một class duy nhất.

Trong khi đó, softmax cho ra một phân bố xác suất:

$$\text{softmax}(z)$$

Nó không chỉ nói class nào lớn nhất, mà còn cho biết mức độ tự tin tương đối giữa các class.

Ví dụ minh họa

Nếu softmax cho:

$$[0.49, 0.48, 0.02, 0.01]$$

thì class 1 là lớn nhất, nhưng class 2 cũng rất gần. Mô hình chưa thật sự chắc chắn.

Nếu softmax cho:

$$[0.95, 0.03, 0.01, 0.01]$$

thì mô hình rất tự tin vào class 1.

8.8.3 Lưu ý về tên gọi

Trong video, giảng viên nhấn mạnh rằng tên *softmax* có thể gây hiểu nhầm nếu ta nghĩ nó là xấp xỉ trơn của hàm max theo nghĩa trả về giá trị lớn nhất.

Thực tế, trong bài toán phân loại, softmax gần với ý tưởng xấp xỉ mềm của argmax hơn, vì nó tạo ra một vector phân bố xác suất trên các class.

Lưu ý

Softmax không trả về một số giống như hàm max. Softmax trả về một vector xác suất có cùng số chiều với vector input.

8.9 Key Softmax Properties – Tính chất quan trọng của Softmax

8.9.1 Tính chất 1: Output luôn dương

Vì:

$$e^{z_i} > 0$$

nên:

$$y_i > 0$$

Điều này phù hợp với xác suất.

8.9.2 Tính chất 2: Tổng output bằng 1

Như đã chứng minh:

$$\sum_{i=1}^K y_i = 1$$

Vì vậy softmax tạo ra một phân bố xác suất hợp lệ.

8.9.3 Tính chất 3: Giữ thứ tự tương đối

Nếu:

$$z_i > z_j$$

thì:

$$e^{z_i} > e^{z_j}$$

Do đó:

$$y_i > y_j$$

Softmax giữ thứ tự của các logits.

Ví dụ minh họa

Trong ví dụ:

$$z_2 = 1.4$$

là lớn nhất, nên:

$$y_2 = 0.51$$

cũng là xác suất lớn nhất.

8.9.4 Tính chất 4: Khuếch đại sự khác biệt

Do dùng hàm mũ, softmax có xu hướng khuếch đại sự khác biệt giữa các logits. Nếu một logit lớn hơn các logit còn lại đáng kể, xác suất của nó sẽ tăng mạnh.

Ví dụ minh họa

Nếu:

$$z = \begin{bmatrix} 1 \\ 2 \\ 10 \end{bmatrix}$$

thì e^{10} lớn hơn rất nhiều so với e^1 và e^2 . Do đó xác suất của class thứ 3 sẽ gần 1.

8.10 Softmax at Output Layer – Softmax ở lớp output của mạng nơ-ron

8.10.1 Quy trình trong bài toán phân loại

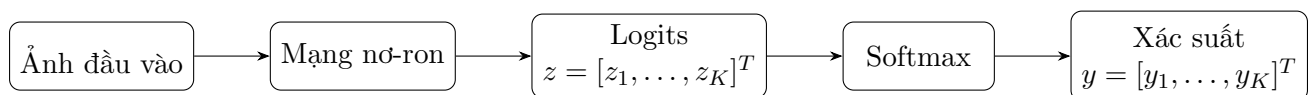
Trong bài toán phân loại nhiều lớp, quy trình thường là:

$$\text{Input} \rightarrow \text{Mạng nơ-ron} \rightarrow z \rightarrow \text{softmax} \rightarrow y$$

Trong đó:

- z là logits;
- y là vector xác suất;
- class dự đoán là phần tử có xác suất lớn nhất.

8.10.2 Sơ đồ minh họa



8.10.3 Dự đoán class cuối cùng

Sau khi có:

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix}$$

ta chọn class:

$$\hat{c} = \arg \max_i y_i$$

Ghi nhớ

Softmax cho ta xác suất từng class. Argmax trên output softmax cho ta class dự đoán cuối cùng.

8.11 Softmax & Cross Entropy – Softmax và Cross Entropy

8.11.1 Vì sao softmax thường đi cùng cross entropy?

Trong bài toán phân loại nhiều lớp, softmax thường được dùng ở output để tạo xác suất. Sau đó, ta dùng cross entropy để đo sai số giữa phân phối dự đoán và nhãn one-hot.

Nếu label đúng là:

$$t = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

và softmax output là:

$$y = \begin{bmatrix} 0.28 \\ 0.51 \\ 0.06 \\ 0.15 \end{bmatrix}$$

thì cross entropy:

$$L(t, y) = - \sum_{i=1}^K t_i \log(y_i)$$

Vì chỉ có $t_2 = 1$, nên:

$$L(t, y) = -\log(y_2)$$

$$L(t, y) = -\log(0.51)$$

$$L(t, y) \approx 0.673$$

Ghi nhớ

Với one-hot label, cross entropy chỉ lấy log của xác suất mà mô hình gán cho class đúng.

8.11.2 Ý nghĩa

Nếu mô hình gán xác suất cao cho class đúng, loss nhỏ.

Nếu mô hình gán xác suất thấp cho class đúng, loss lớn.

Ví dụ:

$$y_2 = 0.90 \Rightarrow -\log(0.90) \approx 0.105$$

$$y_2 = 0.10 \Rightarrow -\log(0.10) \approx 2.303$$

Lưu ý

Trong nhiều thư viện deep learning, người ta thường kết hợp softmax và cross entropy trong một hàm loss duy nhất để tính toán ổn định hơn.

8.12 Softmax Computation Notes – Lưu ý khi tính Softmax

8.12.1 Vấn đề số quá lớn

Softmax dùng e^{z_i} . Nếu z_i rất lớn, e^{z_i} có thể vượt quá khả năng biểu diễn của máy tính.

Ví dụ:

$$e^{1000}$$

là một số cực lớn.

8.12.2 Cách ổn định số học

Một kỹ thuật phổ biến là trừ tất cả logits cho giá trị lớn nhất trước khi tính softmax. Gọi:

$$m = \max_i z_i$$

Ta tính:

$$y_i = \frac{e^{z_i - m}}{\sum_{j=1}^K e^{z_j - m}}$$

Kết quả không thay đổi so với softmax ban đầu.

8.12.3 Vì sao không thay đổi?

Ta có:

$$\frac{e^{z_i - m}}{\sum_{j=1}^K e^{z_j - m}} = \frac{e^{z_i} e^{-m}}{\sum_{j=1}^K e^{z_j} e^{-m}}$$

Vì e^{-m} là hằng số chung, nó triệt tiêu:

$$= \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Ghi nhớ

Khi lập trình softmax, nên dùng phiên bản ổn định số học bằng cách trừ đi logit lớn nhất.

8.13 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
Classification	Bài toán phân loại dữ liệu vào các class rời rạc.	Chó, mèo, xe hơi.
Classifier	Bộ phân loại, nhận input và trả về class dự đoán.	$\hat{y} = f(x)$.
Probabilistic classifier	Bộ phân loại dựa trên xác suất.	$P(y x)$.
Class	Lớp dữ liệu cần dự đoán.	Chó, mèo, điều.
Label	Nhãn đúng của dữ liệu.	Ảnh chó có label chó.
One-hot encoding	Cách mã hóa nhãn bằng vector có một phần tử bằng 1.	$[0, 1, 0, 0]^T$.
Logits	Vector output thô trước softmax.	$z = [0.8, 1.4, -0.8, 0.2]^T$.
Softmax	Hàm biến logits thành vector xác suất.	$\frac{e^{z_i}}{\sum_j e^{z_j}}$.
Probability distribution	Phân bố xác suất, các phần tử không âm và tổng bằng 1.	$[0.2, 0.3, 0.5]$.
Max	Hàm trả về giá trị lớn nhất.	$\max([1, 4, 2]) = 4$.
Argmax	Hàm trả về vị trí của giá trị lớn nhất.	$\arg \max([1, 4, 2]) = 2$.
Cross entropy	Hàm mất mát thường dùng với softmax trong phân loại.	$-\sum t_i \log y_i$.
Output layer	Lớp cuối của mạng nơ-ron.	1000 output cho 1000 class.
Exponential function	Hàm mũ cơ số e .	e^{z_i} .
Numerical stability	Tính ổn định số học khi lập trình.	Trừ max logit trước softmax.

8.14 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

8.14.1 Các ý chính cần nhớ

1. Softmax thường được dùng ở output layer của bài toán phân loại nhiều lớp.
2. Mạng nơ-ron thường tạo ra logits trước khi đưa qua softmax.
3. Logits có thể là số âm, số dương và không phải xác suất.
4. Softmax biến logits thành vector xác suất.
5. Output softmax có các phần tử dương và tổng bằng 1.
6. Với K class, softmax output có K phần tử.
7. One-hot vector được dùng để biểu diễn label đúng.

8. Max trả về giá trị lớn nhất; argmax trả về vị trí của giá trị lớn nhất.
9. Softmax có thể hiểu là phiên bản mềm của argmax trong bài toán phân loại.
10. Class dự đoán cuối cùng thường được chọn bằng argmax trên output softmax.
11. Softmax thường đi cùng cross entropy loss.
12. Khi lập trình softmax, nên dùng phiên bản ổn định số học bằng cách trừ max logit.

8.14.2 Công thức quan trọng

Softmax

$$y_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Tổng xác suất

$$\sum_{i=1}^K y_i = 1$$

Dự đoán class

$$\hat{c} = \arg \max_i y_i$$

Cross entropy

$$L(t, y) = - \sum_{i=1}^K t_i \log(y_i)$$

Softmax ổn định số học

$$y_i = \frac{e^{z_i - m}}{\sum_{j=1}^K e^{z_j - m}}, \quad m = \max_j z_j$$

8.15 Review Questions – Câu hỏi ôn tập

1. Softmax thường được dùng ở đâu trong mạng nơ-ron?
2. Vì sao logits chưa thể xem là xác suất?
3. Công thức softmax là gì?
4. Vì sao output softmax luôn dương?

5. Vì sao tổng output softmax bằng 1?
6. One-hot vector là gì?
7. Với 1000 class, output softmax có bao nhiêu phần tử?
8. Max khác argmax như thế nào?
9. Với $z = [0.8, 1.4, -0.8, 0.2]^T$, argmax bằng bao nhiêu?
10. Softmax của vector trên xấp xỉ bằng bao nhiêu?
11. Vì sao softmax có thể xem như phiên bản mềm của argmax?
12. Softmax thường kết hợp với hàm loss nào trong phân loại?
13. Vì sao cần trừ max logit khi lập trình softmax?
14. Nếu output softmax là $[0.1, 0.7, 0.2]^T$, class dự đoán là class nào?

8.16 Practice Exercises – Bài tập vận dụng

Bài tập 1: Tính softmax

Cho logits:

$$z = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Hãy tính softmax của z .

Lời giải gợi ý

Tính các giá trị mũ:

$$e^1 \approx 2.718, \quad e^2 \approx 7.389, \quad e^3 \approx 20.086$$

Mẫu số:

$$S = 2.718 + 7.389 + 20.086 = 30.193$$

Softmax:

$$y_1 = \frac{2.718}{30.193} \approx 0.090$$

$$y_2 = \frac{7.389}{30.193} \approx 0.245$$

$$y_3 = \frac{20.086}{30.193} \approx 0.665$$

Vậy:

$$y \approx \begin{bmatrix} 0.090 \\ 0.245 \\ 0.665 \end{bmatrix}$$

Bài tập 2: Xác định class dự đoán

Cho output softmax:

$$y = \begin{bmatrix} 0.12 \\ 0.08 \\ 0.70 \\ 0.10 \end{bmatrix}$$

Hỏi mô hình dự đoán class nào?

Lời giải gợi ý

Giá trị lớn nhất là:

$$0.70$$

nằm ở vị trí thứ 3.

Vậy mô hình dự đoán class thứ 3.

Bài tập 3: One-hot vector

Một bài toán có 5 class:

{chó, mèo, chim, xe hơi, máy bay}

Hãy viết one-hot vector cho class “xe hơi”.

Lời giải gợi ý

Class “xe hơi” là class thứ 4, nên:

$$\text{xe hơi} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Bài tập 4: Cross entropy với softmax

Label đúng là class thứ 2:

$$t = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Output softmax:

$$y = \begin{bmatrix} 0.2 \\ 0.7 \\ 0.1 \end{bmatrix}$$

Tính cross entropy.

Lời giải gợi ý

Vì class đúng là class thứ 2:

$$L = -\log(0.7)$$

$$L \approx 0.357$$

Bài tập 5: Softmax ổn định số học

Cho:

$$z = \begin{bmatrix} 1000 \\ 1001 \\ 1002 \end{bmatrix}$$

Vì sao không nên tính trực tiếp $e^{1000}, e^{1001}, e^{1002}$? Hãy nêu cách xử lý.

Lời giải gợi ý

Các giá trị $e^{1000}, e^{1001}, e^{1002}$ rất lớn và có thể gây tràn số.

Ta trừ tất cả logits cho giá trị lớn nhất:

$$m = 1002$$

$$z' = \begin{bmatrix} 1000 - 1002 \\ 1001 - 1002 \\ 1002 - 1002 \end{bmatrix} = \begin{bmatrix} -2 \\ -1 \\ 0 \end{bmatrix}$$

Sau đó tính softmax trên z' .

Kết luận

Softmax là hàm rất quan trọng trong bài toán phân loại nhiều lớp. Mạng nơ-ron thường tạo ra một vector logits ở lớp cuối. Vector này chưa phải xác suất, vì nó có thể chứa số âm, số lớn hơn 1 và tổng không bằng 1. Softmax biến logits thành một vector xác suất hợp lệ, trong đó mỗi phần tử dương và tổng các phần tử bằng 1.

Trong bài toán phân loại, softmax giúp ta biết mô hình đang gán xác suất bao nhiêu cho từng class. Class dự đoán cuối cùng thường là class có xác suất lớn nhất. Vì vậy, softmax có thể được hiểu như một phiên bản mềm của argmax: nó vẫn nhấn mạnh class mạnh nhất, nhưng giữ lại thông tin về các class còn lại.

Softmax thường được dùng cùng cross entropy loss để huấn luyện mô hình phân loại nhiều lớp. Khi lập trình thực tế, cần chú ý đến ổn định số học bằng cách trừ đi logit lớn nhất trước khi tính hàm mũ.

Phần II

Mạng Nơ-ron Tích chập

Chương 1

CNN1: Convolutional Neural Networks – Mạng tích chập

Xem video minh họa CNN tại [video YouTube này](#).

CNN Explainer Learn Convolutional Neural Network (CNN) in your browser! [CNN Explainer](#).

Learning Objectives – Mục tiêu bài học

Bài học này mở đầu cho phần *Convolutional Neural Network*, viết tắt là CNN, hay còn gọi là mạng nơ-ron tích chập.

Sau khi học xong, người học cần nắm được:

1. CNN là gì và vì sao CNN quan trọng trong deep learning.
2. Vì sao CNN được thiết kế đặc biệt phù hợp cho dữ liệu hình ảnh.
3. Nhận biết cấu trúc tổng quát của một mạng CNN.
4. Phân biệt CNN với mạng nơ-ron fully connected thông thường.
5. Hiểu vì sao mạng fully connected gặp vấn đề khi xử lý ảnh lớn.
6. Hiểu hai ý tưởng quan trọng của CNN:
 - (a) *local connectivity* – kết nối cục bộ;
 - (b) *weight sharing* – trọng số dùng chung.
7. Hiểu kernel là gì.
8. Biết cách thực hiện phép convolution bằng cách trượt kernel trên ảnh.
9. Hiểu activation map là gì.
10. Biết cách nối phần convolution với fully connected layer trong ví dụ nhận dạng chữ X/O.

1.1 Convolutional Neural Network – Mạng nơ-ron tích chập

1.1.1 CNN là gì?

CNN là viết tắt của:

Convolutional Neural Network

Trong tiếng Việt, CNN thường được gọi là:

mạng nơ-ron tích chập

Nếu các mạng nơ-ron trước đây ta học là mạng nơ-ron thông thường, thì CNN là một kiểu mạng nơ-ron đặc biệt hơn, được thiết kế để xử lý tốt dữ liệu có cấu trúc không gian, đặc biệt là ảnh.

Ghi nhớ

CNN là một kiến trúc rất quan trọng trong deep learning, đặc biệt trong lĩnh vực thị giác máy tính.

1.1.2 CNN được thiết kế để giải quyết bài toán gì?

CNN ban đầu được thiết kế chủ yếu để xử lý các bài toán liên quan đến hình ảnh. Ví dụ:

- phân loại ảnh;
- nhận dạng khuôn mặt;
- nhận dạng vật thể;
- phát hiện vật thể trong ảnh;
- phân tích ảnh y tế;
- nhận dạng hành động;
- hỗ trợ xe tự lái.

Trong bài học này, ta tập trung vào bài toán:

ảnh đầu vào $\longrightarrow$ class dự đoán

Ví dụ:

ảnh $\longrightarrow$ chó, mèo, xe hơi, máy bay, ...

1.2 CNN Applications – Ứng dụng của CNN

1.2.1 Thị giác máy tính

CNN được sử dụng rất nhiều trong *computer vision* – thị giác máy tính. Một số bài toán tiêu biểu:

- nhận dạng khuôn mặt;
- nhận dạng đối tượng;
- phân loại ảnh;
- mô tả nội dung ảnh;
- nhận dạng hành động trong video.

Ví dụ minh họa

Nếu đưa vào một tấm ảnh, hệ thống có thể gán nhãn rằng đó là “một con chó đang chơi trên bãi cỏ” hoặc “một người đang lướt sóng ở bãi biển”.

1.2.2 Một số lĩnh vực khác

Ngoài ảnh, CNN cũng có thể được dùng trong các bài toán khác như:

- nhận dạng giọng nói;
- phân loại văn bản;
- xử lý tín hiệu;
- phân tích chuỗi thời gian.

Tuy nhiên, trong phần CNN đầu tiên này, ta chủ yếu khảo sát CNN trong bối cảnh xử lý ảnh.

1.3 General CNN Architecture – Cấu trúc tổng quát của CNN

1.3.1 Các thành phần chính

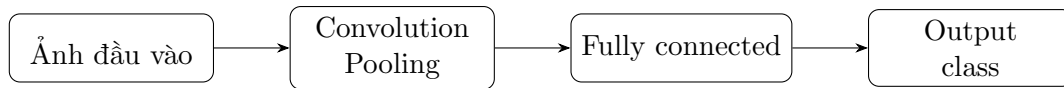
Một mạng CNN dùng cho phân loại ảnh thường có cấu trúc tổng quát như sau:

Input image → Convolution layers → Pooling layers → Fully connected layers → Output

Trong đó:

- **Convolution layer:** lớp tích chập, dùng để trích xuất đặc trưng từ ảnh.

- **Pooling layer:** lớp giảm kích thước dữ liệu, giúp giữ lại thông tin quan trọng.
- **Fully connected layer:** lớp kết nối đầy đủ, dùng để phân loại dựa trên các đặc trưng đã học.



Ghi nhớ

Trong CNN, phần đầu thường dùng convolution và pooling để học đặc trưng ảnh. Phần sau dùng fully connected layer để đưa ra quyết định phân loại.

1.3.2 Số lớp trong CNN

Trong hình minh họa, ta có thể thấy một vài convolution layer và pooling layer. Tuy nhiên, trong thực tế, số lớp có thể rất nhiều.

Một CNN có thể có:

- vài lớp convolution;
- vài chục lớp convolution;
- thậm chí hàng trăm lớp trong các mạng rất sâu.

Lưu ý

Hình vẽ trong slide thường chỉ minh họa ý tưởng. Một mạng CNN thực tế có thể phức tạp hơn nhiều.

1.4 Image Representation – Biểu diễn ảnh trong máy tính

1.4.1 Ảnh xám

Một ảnh xám có thể được biểu diễn bằng một ma trận.

Ví dụ, ảnh kích thước $H \times W$:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1W} \\ x_{21} & x_{22} & \cdots & x_{2W} \\ \vdots & \vdots & \ddots & \vdots \\ x_{H1} & x_{H2} & \cdots & x_{HW} \end{bmatrix}$$

Mỗi phần tử x_{ij} là một pixel.

Với ảnh 8-bit, giá trị pixel thường nằm trong khoảng:

$$0 \leq x_{ij} \leq 255$$

1.4.2 Ảnh màu

Ảnh màu thường có 3 kênh:

$$R, \quad G, \quad B$$

Do đó, ảnh màu kích thước $H \times W$ được biểu diễn dưới dạng:

$$H \times W \times 3$$

Ví dụ minh họa

Một ảnh màu kích thước 32×32 có số giá trị là:

$$32 \cdot 32 \cdot 3 = 3072$$

Tức là ảnh này có 3072 giá trị đầu vào.

1.5 Problem with Fully Connected Networks – Vấn đề của mạng fully connected khi xử lý ảnh

1.5.1 Flatten ảnh thành vector

Với mạng nơ-ron thông thường, khi xử lý ảnh, ta thường biến ảnh thành một vector dài.

Ví dụ, ảnh màu kích thước:

$$32 \times 32 \times 3$$

sẽ được biến thành vector có:

$$32 \cdot 32 \cdot 3 = 3072$$

phần tử.

Sau đó, mỗi neuron ở lớp kế tiếp sẽ nhận toàn bộ 3072 giá trị này.

1.5.2 Số trọng số tăng rất nhanh

Nếu một neuron nhận toàn bộ 3072 input, thì riêng neuron đó đã cần 3072 trọng số. Nếu ảnh lớn hơn, vấn đề nghiêm trọng hơn.

Ví dụ, ảnh màu kích thước:

$$200 \times 200 \times 3$$

có số input là:

$$200 \cdot 200 \cdot 3 = 120000$$

Như vậy, một neuron fully connected cần:

$$120000$$

trọng số.

Nếu lớp đó có 1000 neuron, số trọng số là:

$$120000 \cdot 1000 = 120000000$$

Đó mới chỉ là một lớp.

Cảnh báo

Khi dùng mạng fully connected cho ảnh lớn, số lượng tham số có thể trở nên khổng lồ. Điều này làm tăng chi phí tính toán và dễ dẫn đến overfitting.

1.5.3 Liên hệ với overfitting

Khi mô hình có quá nhiều tham số, nó có khả năng học quá sát dữ liệu huấn luyện. Điều này dễ dẫn đến:

overfitting

Tức là mô hình làm tốt trên tập huấn luyện nhưng làm kém trên dữ liệu mới.

1.6 Key Ideas of CNN – Hai ý tưởng quan trọng của CNN

1.6.1 Local connectivity – Kết nối cục bộ

Trong mạng fully connected, một neuron có thể nhận toàn bộ input.

Trong CNN, một neuron chỉ nhìn một vùng nhỏ của ảnh.

Ý tưởng này gọi là:

local connectivity

hay:

kết nối cục bộ

Ví dụ minh họa

Thay vì nhìn toàn bộ ảnh $200 \times 200 \times 3$, một kernel có thể chỉ nhìn một vùng nhỏ $3 \times 3 \times 3$.

Số trọng số khi đó giảm rất mạnh:

$$3 \cdot 3 \cdot 3 = 27$$

so với:

$$200 \cdot 200 \cdot 3 = 120000$$

Ghi nhớ

Local connectivity giúp CNN giảm số tham số bằng cách chỉ xử lý từng vùng nhỏ của ảnh.

1.6.2 Weight sharing – Trọng số dùng chung

Ý tưởng thứ hai là:

weight sharing

hay:

trọng số dùng chung

Thay vì mỗi vị trí trong ảnh có một bộ trọng số riêng, CNN dùng cùng một kernel để quét qua nhiều vị trí khác nhau.

Nếu kernel là:

$$K = \begin{bmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{bmatrix}$$

thì cùng bốn trọng số này được dùng ở tất cả các vị trí mà kernel trượt qua.

Ghi nhớ

Weight sharing giúp CNN tiếp tục giảm số lượng tham số vì cùng một bộ trọng số được tái sử dụng trên toàn ảnh.

1.6.3 Tác dụng của hai ý tưởng này

Hai ý tưởng này giúp CNN:

- giảm rất mạnh số lượng tham số;
- giảm nguy cơ overfitting;
- tận dụng cấu trúc không gian của ảnh;
- phát hiện cùng một đặc trưng ở nhiều vị trí khác nhau.

1.7 Kernel and Filter – Kernel và bộ lọc

1.7.1 Kernel là gì?

Kernel là một ma trận nhỏ chứa các trọng số được dùng để quét trên ảnh.

Ví dụ kernel kích thước 2×2 :

$$K = \begin{bmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{bmatrix}$$

Trong thực tế, kernel có thể có kích thước:

$$3 \times 3, \quad 5 \times 5, \quad 7 \times 7, \quad 11 \times 11$$

Lưu ý

Trong bài CNN1, kernel 2×2 được dùng để minh họa cho dễ tính tay. Trong mạng thực tế, kernel 3×3 rất phổ biến.

1.7.2 Kernel học được từ đâu?

Trong ví dụ, ta có thể thấy các giá trị kernel được cho sẵn. Tuy nhiên, trong mạng CNN thật, các giá trị này không phải do ta tự đặt thủ công.

Chúng được học thông qua quá trình huấn luyện:

$$\text{forward} \rightarrow \text{loss} \rightarrow \text{backpropagation} \rightarrow \text{update}$$

Ghi nhớ

Kernel cũng là tham số của mạng. Nó được học từ dữ liệu giống như trọng số trong mạng nơ-ron thông thường.

1.8 Convolution Forward Pass – Tính xuôi trong lớp tích chập

1.8.1 Ý tưởng trượt kernel

Để thực hiện convolution, ta làm như sau:

1. Đặt kernel lên một vùng nhỏ của ảnh.
2. Nhân từng phần tử tương ứng giữa kernel và vùng ảnh.
3. Cộng tất cả các tích lại.
4. Cộng thêm bias.
5. Đưa qua hàm kích hoạt.
6. Ghi kết quả vào vị trí tương ứng trong activation map.
7. Trượt kernel sang vị trí tiếp theo và lặp lại.

1.8.2 Công thức tại vị trí đầu tiên

Giả sử input là ma trận 3×3 :

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

Kernel là:

$$K = \begin{bmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{bmatrix}$$

Khi đặt kernel ở góc trên bên trái, vùng ảnh tương ứng là:

$$\begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{bmatrix}$$

Kết quả tại vị trí đầu tiên là:

$$a_{11} = f(k_{11}x_{11} + k_{12}x_{12} + k_{21}x_{21} + k_{22}x_{22} + b)$$

Trong đó:

- b là bias;
- f là hàm kích hoạt;
- a_{11} là phần tử đầu tiên của activation map.

1.8.3 Các vị trí còn lại

Với input 3×3 , kernel 2×2 , stride bằng 1 và không padding, output sẽ có kích thước:

$$2 \times 2$$

Activation map là:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

Trong đó:

$$a_{12} = f(k_{11}x_{12} + k_{12}x_{13} + k_{21}x_{22} + k_{22}x_{23} + b)$$

$$a_{21} = f(k_{11}x_{21} + k_{12}x_{22} + k_{21}x_{31} + k_{22}x_{32} + b)$$

$$a_{22} = f(k_{11}x_{22} + k_{12}x_{23} + k_{21}x_{32} + k_{22}x_{33} + b)$$

Ghi nhớ

Mỗi phần tử trong activation map tương ứng với một vị trí đặt kernel trên ảnh.

1.9 Activation Map – Bản đồ kích hoạt

1.9.1 Activation map là gì?

Activation map là kết quả thu được sau khi một kernel quét qua toàn bộ ảnh.

$$\text{Input image} \xrightarrow{\text{kernel}} \text{activation map}$$

Nếu một kernel phát hiện một loại đặc trưng nào đó, activation map cho biết đặc trưng đó xuất hiện mạnh ở vị trí nào.

1.9.2 Một kernel tạo ra một activation map

Nếu ta dùng một kernel, ta thu được một activation map.

Nếu ta dùng hai kernel, ta thu được hai activation map.

Tổng quát:

$$F \text{ kernels} \rightarrow F \text{ activation maps}$$

Ví dụ minh họa

Một kernel có thể học để phát hiện cạnh ngang. Một kernel khác có thể học để phát hiện cạnh dọc. Mỗi kernel tạo ra một activation map riêng.

1.9.3 Kích thước activation map

Khi một kernel trượt qua input, tại mỗi vị trí nó tạo ra một giá trị output. Tập hợp tất cả các giá trị output này tạo thành một **activation map**.

Nói cách khác:

$$\text{mỗi vị trí đặt kernel} \Rightarrow \text{một giá trị trong activation map}$$

Vì vậy, kích thước của activation map phụ thuộc vào:

- kích thước input;
- kích thước kernel;
- stride;
- padding.

Trường hợp đơn giản: stride bằng 1 và không padding

Nếu input có kích thước:

$$H \times W$$

trong đó:

$$H = \text{chiều cao input}$$

W = chiều rộng input

Kernel có kích thước:

$$K_h \times K_w$$

trong đó:

K_h = chiều cao kernel

K_w = chiều rộng kernel

Nếu stride bằng 1 và không padding, thì output có kích thước:

$$(H - K_h + 1) \times (W - K_w + 1)$$

Vì sao có công thức này?

Giả sử input có kích thước 3×3 :

$$I = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

và kernel có kích thước 2×2 .

Kernel 2×2 có thể đặt vào bốn vị trí khác nhau trên input.

Vị trí thứ nhất, góc trên bên trái:

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$$

Vị trí thứ hai, góc trên bên phải:

$$\begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}$$

Vị trí thứ ba, góc dưới bên trái:

$$\begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix}$$

Vị trí thứ tư, góc dưới bên phải:

$$\begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$$

Như vậy, theo chiều cao, kernel đặt được:

$$3 - 2 + 1 = 2$$

vị trí.

Theo chiều rộng, kernel cũng đặt được:

$$3 - 2 + 1 = 2$$

vị trí.

Do đó activation map có kích thước:

$$2 \times 2$$

Áp dụng công thức:

$$(H - K_h + 1) \times (W - K_w + 1)$$

với:

$$H = 3, \quad W = 3, \quad K_h = 2, \quad K_w = 2$$

ta có:

$$(3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2$$

Ví dụ tính activation map cụ thể

Giả sử input là:

$$I = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel là:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Bias:

$$b = 0$$

Ta không dùng hàm kích hoạt để tính cho đơn giản.

Tại vị trí thứ nhất:

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$$

ta tính:

$$1 \cdot 1 + 2 \cdot 0 + 4 \cdot 0 + 5 \cdot (-1)$$

$$= 1 - 5 = -4$$

Tại vị trí thứ hai:

$$\begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}$$

ta tính:

$$2 \cdot 1 + 3 \cdot 0 + 5 \cdot 0 + 6 \cdot (-1)$$

$$= 2 - 6 = -4$$

Tại vị trí thứ ba:

$$\begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix}$$

ta tính:

$$4 \cdot 1 + 5 \cdot 0 + 7 \cdot 0 + 8 \cdot (-1)$$

$$= 4 - 8 = -4$$

Tại vị trí thứ tư:

$$\begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$$

ta tính:

$$5 \cdot 1 + 6 \cdot 0 + 8 \cdot 0 + 9 \cdot (-1)$$

$$= 5 - 9 = -4$$

Vậy activation map thu được là:

$$\begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$$

Nếu sau đó dùng hàm kích hoạt ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

thì activation map sau ReLU là:

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Stride là gì?

Stride là số bước mà kernel dịch chuyển sau mỗi lần tính.

Nếu:

$$S = 1$$

thì kernel trượt từng ô một.

Nếu:

$$S = 2$$

thì kernel nhảy cách hai ô một lần.

Ví dụ, với input 4×4 :

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

và kernel 2×2 .

Nếu stride bằng 1, kernel có thể đi qua nhiều vị trí hơn, nên activation map lớn hơn.

Nếu stride bằng 2, kernel nhảy xa hơn, số vị trí đặt kernel ít hơn, nên activation map nhỏ hơn.

stride lớn $\Rightarrow$ kernel nhảy xa $\Rightarrow$ activation map nhỏ hơn

Padding là gì?

Padding là việc thêm các giá trị vào viền xung quanh input. Thông thường, giá trị được thêm vào là số 0, nên còn gọi là **zero padding**.

Ví dụ input 3×3 :

$$I = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Nếu padding bằng 1, ta thêm một lớp số 0 xung quanh input:

$$I_{pad} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 0 \\ 0 & 4 & 5 & 6 & 0 \\ 0 & 7 & 8 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Khi đó input từ kích thước:

$$3 \times 3$$

trở thành:

$$5 \times 5$$

Padding có hai tác dụng quan trọng.

Thứ nhất, padding giúp activation map không bị nhỏ quá nhanh sau mỗi lớp convolution.

Thứ hai, padding giúp các pixel ở rìa ảnh được kernel xét đến nhiều hơn. Nếu không padding, các pixel ở góc và cạnh ảnh thường được dùng ít hơn các pixel ở giữa ảnh.

Công thức tổng quát có stride và padding

Với input kích thước:

$$H \times W$$

kernel kích thước:

$$K_h \times K_w$$

padding:

$$P$$

stride:

$$S$$

thì kích thước activation map là:

$$H_{out} = \left\lfloor \frac{H + 2P - K_h}{S} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{W + 2P - K_w}{S} \right\rfloor + 1$$

Do đó:

$$\text{output size} = H_{out} \times W_{out}$$

Trong đó ký hiệu:

$$\lfloor x \rfloor$$

nghĩa là lấy phần nguyên xuống.

Ví dụ có padding

Giả sử input có kích thước:

$$5 \times 5$$

kernel có kích thước:

$$3 \times 3$$

stride:

$$S = 1$$

không padding:

$$P = 0$$

Khi đó:

$$H_{out} = \frac{5 + 2 \cdot 0 - 3}{1} + 1 = 3$$

$$W_{out} = \frac{5 + 2 \cdot 0 - 3}{1} + 1 = 3$$

Vậy output có kích thước:

$$3 \times 3$$

Nếu vẫn dùng input 5×5 , kernel 3×3 , stride 1, nhưng thêm padding:

$$P = 1$$

thì:

$$H_{out} = \frac{5 + 2 \cdot 1 - 3}{1} + 1 = 5$$

$$W_{out} = \frac{5 + 2 \cdot 1 - 3}{1} + 1 = 5$$

Vậy output có kích thước:

$$5 \times 5$$

Như vậy, padding giúp giữ nguyên kích thước không gian của activation map so với input ban đầu.

Tóm tắt

Stride là bước nhảy của kernel:

stride càng lớn $\Rightarrow$ activation map càng nhỏ

Padding là phần viền được thêm vào input:

padding càng lớn $\Rightarrow$ activation map càng lớn

Trong trường hợp đơn giản nhất, stride bằng 1 và không padding:

$$H_{out} = H - K_h + 1$$

$$W_{out} = W - K_w + 1$$

Còn trong trường hợp tổng quát:

$$H_{out} = \left\lfloor \frac{H + 2P - K_h}{S} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{W + 2P - K_w}{S} \right\rfloor + 1$$

1.10 Simple X/O Classification Example – Ví dụ phân loại chữ X/O

1.10.1 Mô tả bài toán

Trong video, ta khảo sát một ví dụ rất nhỏ:

input là ảnh 3×3

và chỉ có hai class:

chữ X hoặc chữ O

Nhiệm vụ của mạng là xác định ảnh đầu vào thuộc class nào.

1.10.2 Biểu diễn chữ X và chữ O

Để dễ tính, ta có thể biểu diễn pixel trắng là 1 và pixel đen là 0.
Chữ X có thể biểu diễn như sau:

$$X_{\text{letter X}} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Chữ O có thể biểu diễn như sau:

$$X_{\text{letter O}} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Lưu ý

Trong slide, pixel trắng có thể được biểu diễn bằng 255 và pixel đen bằng 0. Ở đây ta dùng 1 và 0 để việc tính tay đơn giản hơn.

1.10.3 One-hot output

Vì có hai class, output có hai phần tử.
Ta quy ước:

$$X = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$O = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Nếu đưa chữ X vào, ta mong muốn output là:

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Nếu đưa chữ O vào, ta mong muốn output là:

$$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

1.11 Convolution Example for Letter X – Ví dụ tích chập cho chữ X

1.11.1 Hai kernel minh họa

Giả sử ta dùng hai kernel đơn giản.

Kernel thứ nhất:

$$K_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Kernel thứ hai:

$$K_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Giả sử:

$$b = 0$$

và hàm kích hoạt là hàm đồng nhất:

$$f(u) = u$$

1.11.2 Input chữ X

$$X = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

1.11.3 Activation map với kernel thứ nhất

Tại vị trí trên trái:

$$a_{11}^{(1)} = 1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 2$$

Tại vị trí trên phải:

$$a_{12}^{(1)} = 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 = 0$$

Tại vị trí dưới trái:

$$a_{21}^{(1)} = 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 = 0$$

Tại vị trí dưới phải:

$$a_{22}^{(1)} = 1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 2$$

Vậy activation map thứ nhất là:

$$A_1 = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

1.11.4 Activation map với kernel thứ hai

Tương tự, với kernel:

$$K_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

ta thu được:

$$A_2 = \begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix}$$

Ghi nhớ

Với chữ X, hai kernel trên lần lượt phản ứng mạnh với hai hướng đường chéo khác nhau.

1.12 Flatten and Fully Connected Layer – Flatten và lớp kết nối đầy đủ

1.12.1 Flatten là gì?

Sau khi có các activation map, ta thường biến chúng thành một vector để đưa vào phần fully connected.

Thao tác này gọi là:

flatten

Ví dụ:

$$A_1 = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix}$$

Nếu đọc theo từng hàng, ta có vector:

$$v = \begin{bmatrix} 2 \\ 0 \\ 0 \\ 2 \\ 0 \\ 0 \\ 2 \\ 2 \\ 0 \end{bmatrix}$$

1.12.2 Fully connected layer

Vector sau flatten được đưa vào fully connected layer để phân loại.
Nếu có hai output neuron:

$$z_1 = \sum_{i=1}^m w_{1i}v_i + b_1$$
$$z_2 = \sum_{i=1}^m w_{2i}v_i + b_2$$

Trong bài toán X/O:

$z_1 \rightarrow$ điểm cho chữ X

$z_2 \rightarrow$ điểm cho chữ O

Sau đó ta có thể dùng hàm phù hợp, ví dụ softmax, để chuyển thành xác suất phân loại.

Ghi nhớ

Phần convolution trích xuất đặc trưng. Phần fully connected dùng các đặc trưng đó để quyết định class.

1.13 Where Do CNN Parameters Come From? – Các tham số trong CNN đến từ đâu?

1.13.1 Kernel, bias và trọng số

Trong ví dụ, kernel, bias và trọng số fully connected có thể được cho sẵn để ta tính tay.

Tuy nhiên, trong mạng thật, các tham số này được học từ dữ liệu.
Các tham số cần học gồm:

- các giá trị trong kernel;
- bias của convolution layer;
- trọng số fully connected;
- bias fully connected.

1.13.2 Huấn luyện CNN

Về nguyên tắc, CNN vẫn được huấn luyện theo quy trình quen thuộc:

forward $\rightarrow$ loss $\rightarrow$ backpropagation $\rightarrow$ update parameters

Tuy nhiên, vì CNN có weight sharing và convolution layer, backpropagation trong CNN sẽ hơi khác so với mạng fully connected thông thường.

Lưu ý

Bài CNN1 chủ yếu tập trung vào cách tính forward trong convolution layer. Phần backpropagation của CNN sẽ cần được học kỹ hơn ở các bài sau.

1.14 Why CNN Works Well for Images – Vì sao CNN phù hợp với ảnh?

1.14.1 Ảnh có tính cục bộ

Trong ảnh, nhiều đặc trưng quan trọng xuất hiện trong những vùng nhỏ. Ví dụ:

- cạnh ngang;
- cạnh dọc;
- góc;
- đường cong;
- họa tiết;
- mắt, mũi, miệng trong ảnh khuôn mặt.

Vì vậy, một kernel nhỏ vẫn có thể phát hiện được đặc trưng quan trọng.

1.14.2 Đặc trưng có thể xuất hiện ở nhiều vị trí

Một đặc trưng như cạnh, góc hoặc đường cong có thể xuất hiện ở bất kỳ vị trí nào trong ảnh.

Nếu một kernel học được cách phát hiện cạnh dọc, ta có thể dùng kernel đó quét trên toàn ảnh để tìm cạnh dọc ở nhiều vị trí.

Đây là lý do weight sharing phù hợp với ảnh.

1.14.3 CNN giảm overfitting

Vì CNN giảm số lượng tham số so với fully connected network, nên CNN thường giảm nguy cơ overfitting khi xử lý ảnh.

Ghi nhớ

CNN hiệu quả vì nó tận dụng hai đặc điểm quan trọng của ảnh: thông tin cục bộ và sự lặp lại của đặc trưng theo vị trí.

1.15 ANN vs CNN – So sánh ANN thông thường và CNN

Tiêu chí	ANN fully connected	CNN
Cách xử lý ảnh	Flatten ảnh thành vector	Giữ cấu trúc không gian của ảnh
Kết nối	Mỗi neuron nhận toàn bộ input	Mỗi neuron nhìn một vùng cục bộ
Trọng số	Mỗi neuron có bộ trọng số riêng	Kernel được dùng chung nhiều vị trí
Số tham số	Rất lớn với ảnh lớn	Nhỏ hơn nhiều
Phù hợp với ảnh	Kém hơn khi ảnh lớn/phức tạp	Rất phù hợp
Nguy cơ overfitting	Cao nếu dữ liệu không đủ	Thấp hơn do ít tham số hơn
Đặc trưng học được	Ít khai thác cấu trúc không gian	Học cạnh, góc, pattern cục bộ

1.16 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
CNN	Convolutional Neural Network, mạng nơ-ron tích chập.	Mạng phân loại ảnh.
Convolution	Phép tích chập, quét kernel trên ảnh để tạo đặc trưng.	Kernel 3×3 .
Kernel	Ma trận trọng số nhỏ dùng trong convolution.	2×2 , 3×3 .
Filter	Bộ lọc; thường được dùng gần nghĩa với kernel.	Bộ lọc cạnh.
Activation map	Ma trận kết quả sau khi kernel quét trên ảnh.	Output 2×2 .
Feature map	Cách gọi khác của activation map.	Bản đồ đặc trưng.
Local connectivity	Kết nối cục bộ, neuron chỉ nhìn một vùng nhỏ.	Kernel nhìn vùng 3×3 .
Weight sharing	Trọng số dùng chung ở nhiều vị trí.	Một kernel quét toàn ảnh.
Fully connected layer	Lớp kết nối đầy đủ như ANN truyền thống.	Lớp phân loại cuối.
Flatten	Biến ma trận hoặc tensor thành vector.	2×2 thành vector 4 phần tử.
Pixel	Điểm ảnh.	Giá trị từ 0 đến 255.
Channel	Kênh của ảnh.	R, G, B.
RGB	Ảnh màu có 3 kênh đỏ, xanh lá, xanh dương.	$H \times W \times 3$.
Pooling	Lớp giảm kích thước đặc trưng.	Max pooling.
Stride	Bước trượt của kernel.	Stride bằng 1.

Thuật ngữ	Giải thích	Ví dụ
Padding	Thêm viền quanh ảnh để điều chỉnh kích thước output.	Zero padding.
Overfitting	Mô hình quá khớp dữ liệu huấn luyện.	Quá nhiều tham số.
Backpropagation	Lan truyền ngược để cập nhật tham số.	Học kernel.

1.17 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

1.17.1 Các ý chính cần nhớ

1. CNN là mạng nơ-ron tích chập, rất quan trọng trong xử lý ảnh.
2. CNN thường gồm phần convolution/pooling và phần fully connected.
3. Mạng fully connected gặp vấn đề lớn khi xử lý ảnh vì số lượng tham số tăng rất nhanh.
4. Ảnh $32 \times 32 \times 3$ có 3072 giá trị.
5. Ảnh $200 \times 200 \times 3$ có 120000 giá trị.
6. Một neuron fully connected nhận toàn bộ input nên cần rất nhiều trọng số.
7. CNN giảm số tham số bằng local connectivity và weight sharing.
8. Kernel là một ma trận trọng số nhỏ trượt trên ảnh.
9. Mỗi lần đặt kernel lên một vùng ảnh, ta nhân tương ứng, cộng lại, cộng bias và đưa qua hàm kích hoạt.
10. Kết quả khi một kernel quét toàn ảnh là một activation map.
11. Nhiều kernel tạo ra nhiều activation map.
12. Sau convolution, activation map có thể được flatten rồi đưa vào fully connected layer.
13. Kernel và trọng số fully connected được học bằng backpropagation.
14. CNN hiệu quả vì ảnh có tính cục bộ và đặc trưng có thể xuất hiện ở nhiều vị trí.

1.17.2 Công thức quan trọng

Số giá trị của ảnh RGB

$$H \cdot W \cdot 3$$

Số trọng số của một neuron fully connected

$$H \cdot W \cdot C$$

Số trọng số của một kernel

$$K_h \cdot K_w \cdot C$$

Convolution với kernel 2×2

$$a_{ij} = f(k_{11}x_{ij} + k_{12}x_{i,j+1} + k_{21}x_{i+1,j} + k_{22}x_{i+1,j+1} + b)$$

Kích thước output không padding, stride 1

$$(H - K_h + 1) \times (W - K_w + 1)$$

1.18 Review Questions – Câu hỏi ôn tập

1. CNN là viết tắt của cụm từ gì?
2. CNN được thiết kế ban đầu chủ yếu cho loại dữ liệu nào?
3. Một ảnh màu 32×32 có bao nhiêu giá trị input?
4. Một ảnh màu 200×200 có bao nhiêu giá trị input?
5. Vì sao mạng fully connected dễ có quá nhiều tham số khi xử lý ảnh?
6. Local connectivity là gì?
7. Weight sharing là gì?
8. Kernel là gì?
9. Activation map là gì?
10. Một kernel tạo ra bao nhiêu activation map?
11. Nếu có 10 kernel thì có bao nhiêu activation map?
12. Với input 3×3 và kernel 2×2 , stride 1, không padding, output có kích thước bao nhiêu?
13. Flatten là gì?
14. Vì sao CNN thường giảm overfitting tốt hơn fully connected layer khi xử lý ảnh?
15. Các kernel trong CNN thật được lấy từ đâu?

1.19 Practice Exercises – Bài tập vận dụng

Bài tập 1: Tính số input

Một ảnh màu có kích thước:

$$64 \times 64 \times 3$$

Hỏi ảnh này có bao nhiêu giá trị pixel?

Lời giải gợi ý

$$64 \cdot 64 \cdot 3 = 12288$$

Vậy ảnh có 12288 giá trị.

Bài tập 2: So sánh số trọng số

Một ảnh màu kích thước:

$$100 \times 100 \times 3$$

1. Một neuron fully connected cần bao nhiêu trọng số?
2. Một kernel 3×3 cần bao nhiêu trọng số?

Lời giải gợi ý

Neuron fully connected:

$$100 \cdot 100 \cdot 3 = 30000$$

Kernel 3×3 :

$$3 \cdot 3 \cdot 3 = 27$$

Bài tập 3: Tính convolution

Cho input:

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}$$

Kernel:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Giả sử $b = 0$ và $f(u) = u$. Tính activation map.

Lời giải gợi ý

$$a_{11} = 1 \cdot 1 + 0 \cdot 2 + 0 \cdot 0 + 1 \cdot 1 = 2$$

$$a_{12} = 1 \cdot 2 + 0 \cdot 0 + 0 \cdot 1 + 1 \cdot 3 = 5$$

$$a_{21} = 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 2 + 1 \cdot 1 = 1$$

$$a_{22} = 1 \cdot 1 + 0 \cdot 3 + 0 \cdot 1 + 1 \cdot 0 = 1$$

Vậy:

$$A = \begin{bmatrix} 2 & 5 \\ 1 & 1 \end{bmatrix}$$

Bài tập 4: Kích thước output

Input có kích thước:

$$10 \times 10$$

Kernel có kích thước:

$$3 \times 3$$

Stride bằng 1, không padding. Hỏi activation map có kích thước bao nhiêu?

Lời giải gợi ý

$$(10 - 3 + 1) \times (10 - 3 + 1) = 8 \times 8$$

Bài tập 5: Giải thích trực giác

Giải thích ngắn gọn vì sao CNN dùng local connectivity và weight sharing lại phù hợp với ảnh.

Lời giải gợi ý

Ảnh có cấu trúc không gian. Các đặc trưng như cạnh, góc, đường cong thường xuất hiện trong những vùng nhỏ. Vì vậy, local connectivity giúp kernel tập trung vào vùng cục bộ. Ngoài ra, cùng một đặc trưng có thể xuất hiện ở nhiều vị trí khác nhau trong ảnh, nên weight sharing cho phép cùng một kernel phát hiện đặc trưng đó ở mọi nơi. Hai cơ chế này giúp giảm số tham số và giảm nguy cơ overfitting.

Conclusion – Kết luận

CNN1 giới thiệu mạng nơ-ron tích chập, một kiến trúc rất quan trọng trong deep learning, đặc biệt với các bài toán hình ảnh. Khác với mạng fully connected truyền thống, CNN không biến ảnh thành một vector rồi kết nối mọi pixel với mọi neuron ngay từ đầu. Thay vào đó, CNN dùng các kernel nhỏ trượt trên ảnh để trích xuất đặc trưng cục bộ.

Hai ý tưởng cốt lõi của CNN là *local connectivity* và *weight sharing*. Local connectivity giúp mỗi kernel chỉ nhìn một vùng nhỏ của ảnh, còn weight sharing giúp cùng một bộ trọng số được dùng lại ở nhiều vị trí. Nhờ đó, CNN giảm mạnh số lượng tham số, khai thác tốt cấu trúc không gian của ảnh và giảm nguy cơ overfitting.

Bài học cũng minh họa cách tính convolution forward bằng kernel 2×2 trên ảnh 3×3 , cách tạo activation map, cách flatten kết quả và đưa vào fully connected layer để phân loại. Các tham số như kernel, bias và trọng số fully connected không phải tự chọn bằng tay trong mạng thật, mà được học thông qua quá trình huấn luyện và backpropagation.

Chương 2

CNN2: Multichannel Convolution, Pooling, and Padding – Tích chập nhiều kênh, pooling và padding

Learning Objectives – Mục tiêu bài học

Bài học này tiếp tục phần CNN, tập trung vào cách xử lý ảnh màu, convolution nhiều kênh, pooling layer, việc lặp lại nhiều lớp tích chập và kỹ thuật zero padding.

Sau khi học xong, người học cần nắm được:

1. Hiểu sự khác nhau giữa convolution trên ảnh xám và ảnh màu.
2. Biết vì sao kernel phải có số layer tương ứng với số channel của input.
3. Hiểu cách tính convolution khi input có nhiều channel.
4. Biết nhiều kernel tạo ra nhiều activation map.
5. Hiểu pooling layer là gì.
6. Phân biệt max pooling và average pooling.
7. Hiểu pooling là một dạng downsampling.
8. Biết mục tiêu chính của pooling là giảm kích thước dữ liệu và giảm số tham số.
9. Hiểu cách các lớp convolution và pooling được lặp lại trong CNN.
10. Hiểu khi input của convolution layer có nhiều activation map thì kernel cũng phải có cùng số layer.
11. Hiểu ý nghĩa của flatten trước khi đưa vào fully connected layer.
12. Hiểu zero padding là gì và vì sao cần dùng padding.

2.1 Review of Simple Convolution – Ôn lại phép tích chập đơn giản

2.1.1 Trường hợp ảnh xám một kênh

Trong bài trước, ta đã khảo sát trường hợp đơn giản nhất: input là một ảnh xám rất nhỏ.

Ví dụ input là ma trận 3×3 :

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

Vì ảnh xám chỉ có một kênh, nên input chỉ là một ma trận 2D.

Kernel cũng là một ma trận 2D, ví dụ kernel 2×2 :

$$K = \begin{bmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{bmatrix}$$

Khi đặt kernel lên một vùng ảnh, ta nhân từng phần tử tương ứng, cộng lại, cộng bias, rồi đưa qua hàm kích hoạt.

$$a_{11} = f(k_{11}x_{11} + k_{12}x_{12} + k_{21}x_{21} + k_{22}x_{22} + b)$$

Ghi nhớ

Với ảnh một kênh, kernel là một ma trận 2D. Với ảnh nhiều kênh, kernel không còn chỉ là một ma trận 2D nữa.

2.1.2 Kích thước kernel

Trong ví dụ đơn giản, ta dùng kernel 2×2 để dễ tính tay.

Trong thực tế, kernel có thể có nhiều kích thước khác nhau:

$$2 \times 2, \quad 3 \times 3, \quad 5 \times 5, \quad 7 \times 7, \quad 11 \times 11$$

Kernel 3×3 là một lựa chọn rất phổ biến trong nhiều mạng CNN hiện đại.

2.2 Color Image Convolution – Tích chập trên ảnh màu

2.2.1 Ảnh màu có nhiều channel

Ảnh màu RGB có ba kênh màu:

$$R, \quad G, \quad B$$

Có thể hiểu mỗi kênh màu là một ma trận riêng.

Vì vậy, một ảnh màu có kích thước:

$$H \times W \times 3$$

Trong đó:

- H là chiều cao ảnh;
- W là chiều rộng ảnh;
- 3 là số channel màu RGB.

Ví dụ minh họa

Một ảnh màu 224×224 không chỉ là một ma trận 224×224 , mà là một khối dữ liệu:

$$224 \times 224 \times 3$$

gồm ba ma trận tương ứng với ba kênh màu.

2.2.2 Kernel cũng phải có cùng số layer với input

Nếu input có 3 channel, kernel cũng phải có 3 layer tương ứng.

Ví dụ, nếu input là ảnh RGB:

$$X \in \mathbb{R}^{H \times W \times 3}$$

thì một kernel kích thước không gian 3×3 phải có dạng:

$$K \in \mathbb{R}^{3 \times 3 \times 3}$$

Tức là kernel gồm 3 ma trận 3×3 :

$$K_R, \quad K_G, \quad K_B$$

Mỗi layer của kernel áp lên một channel tương ứng của ảnh.

$$K_R \text{ áp lên kênh } R$$

$$K_G \text{ áp lên kênh } G$$

$$K_B \text{ áp lên kênh } B$$

Ghi nhớ

Độ sâu của kernel phải bằng số channel của input. Nếu input có 3 channel thì kernel cũng phải có 3 layer.

2.3 Multi-channel Convolution Formula – Công thức tích chập nhiều kênh

2.3.1 Ý tưởng tính toán

Giả sử input có 3 channel:

$$X_R, \quad X_G, \quad X_B$$

và kernel có 3 layer:

$$K_R, \quad K_G, \quad K_B$$

Khi đặt kernel lên một vùng ảnh, ta làm như sau:

1. Nhân tương ứng K_R với vùng ảnh trên channel R .
2. Nhân tương ứng K_G với vùng ảnh trên channel G .
3. Nhân tương ứng K_B với vùng ảnh trên channel B .
4. Cộng tất cả các tích của cả ba channel.
5. Cộng bias.
6. Đưa qua hàm kích hoạt.

2.3.2 Ví dụ kernel $3 \times 3 \times 3$

Nếu kernel có kích thước:

$$3 \times 3 \times 3$$

thì số phép nhân tại một vị trí là:

$$3 \cdot 3 \cdot 3 = 27$$

Sau đó cộng 27 tích lại, cộng thêm bias và đưa qua hàm kích hoạt.

$$a_{ij} = f \left(\sum_{c=1}^3 \sum_{u=1}^3 \sum_{v=1}^3 K_{u,v,c} X_{i+u-1, j+v-1, c} + b \right)$$

Trong đó:

- c là chỉ số channel;
- u, v là chỉ số trong kernel;
- b là bias;
- f là hàm kích hoạt;
- a_{ij} là một giá trị trong activation map.

Ghi nhớ

Trong convolution nhiều kênh, tại mỗi vị trí ta cộng tất cả các tích trên toàn bộ chiều rộng, chiều cao và chiều sâu của kernel.

2.3.3 Một kernel vẫn tạo ra một activation map

Dù kernel có nhiều layer, ví dụ $3 \times 3 \times 3$, thì toàn bộ kernel đó vẫn tạo ra một activation map duy nhất.

$$1 \text{ kernel } 3 \times 3 \times 3 \rightarrow 1 \text{ activation map}$$

Lý do là tại mỗi vị trí trượt, sau khi cộng tất cả các tích trên mọi channel, ta chỉ thu được một giá trị output.

2.4 Multiple Kernels – Nhiều kernel trong một lớp convolution

2.4.1 Một lớp convolution thường có nhiều kernel

Trong thực tế, một convolution layer thường không chỉ có một kernel. Ta có thể có:

$$K \text{ kernels}$$

Mỗi kernel học một kiểu đặc trưng khác nhau.
Ví dụ:

- một kernel học cạnh ngang;
- một kernel học cạnh dọc;
- một kernel học đường chéo;
- một kernel học vùng màu hoặc texture.

2.4.2 Số kernel bằng số activation map đầu ra

Nếu có K kernel, ta thu được K activation map.

$$K \text{ kernels} \rightarrow K \text{ activation maps}$$

Nếu một lớp convolution có 64 kernel, output của lớp đó sẽ có 64 activation map.

Ghi nhớ

Số activation map đầu ra của một convolution layer bằng số kernel của layer đó.

2.4.3 Output của convolution layer là một khối dữ liệu

Nếu mỗi activation map có kích thước:

$$H' \times W'$$

và có K activation map, thì output của convolution layer có kích thước:

$$H' \times W' \times K$$

Tức là output cũng là một tensor 3D.

2.5 Pooling Layer – Lớp pooling

2.5.1 Pooling layer là gì?

Ngoài convolution layer, CNN thường có thêm một loại layer rất quan trọng là pooling layer.

Pooling layer nhận input là activation map và tạo ra output có kích thước nhỏ hơn.

Ý tưởng là chọn một cửa sổ nhỏ, ví dụ:

$$2 \times 2$$

rồi trượt cửa sổ đó trên activation map.

Ghi nhớ

Pooling là kỹ thuật giảm kích thước không gian của activation map.

2.5.2 Pooling window và stride

Pooling layer cần hai thông số quan trọng:

- **window size**: kích thước vùng pooling;
- **stride**: bước nhảy khi trượt cửa sổ.

Một lựa chọn phổ biến là:

$$\text{window size} = 2 \times 2$$

$$\text{stride} = 2$$

Khi đó, các vùng pooling thường không chồng lấn lên nhau.

2.5.3 Ví dụ cửa sổ pooling 2×2

Giả sử ta có activation map:

$$A = \begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 3 & 1 \\ 0 & 4 & 5 & 8 \end{bmatrix}$$

Với pooling window 2×2 và stride 2, ta chia thành bốn vùng:

$$\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix}, \quad \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$$
$$\begin{bmatrix} 7 & 2 \\ 0 & 4 \end{bmatrix}, \quad \begin{bmatrix} 3 & 1 \\ 5 & 8 \end{bmatrix}$$

2.6 Max Pooling – Max pooling

2.6.1 Ý tưởng max pooling

Max pooling lấy giá trị lớn nhất trong mỗi vùng pooling.
Với vùng:

$$\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix}$$

giá trị max là:

$$6$$

2.6.2 Ví dụ max pooling

Với activation map:

$$A = \begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 3 & 1 \\ 0 & 4 & 5 & 8 \end{bmatrix}$$

Max pooling 2×2 , stride 2 cho kết quả:

$$P = \begin{bmatrix} 6 & 4 \\ 7 & 8 \end{bmatrix}$$

Vì:

$$\max \begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix} = 6$$

$$\max \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix} = 4$$

$$\max \begin{bmatrix} 7 & 2 \\ 0 & 4 \end{bmatrix} = 7$$

$$\max \begin{bmatrix} 3 & 1 \\ 5 & 8 \end{bmatrix} = 8$$

Ghi nhớ

Max pooling giữ lại tín hiệu mạnh nhất trong mỗi vùng. Đây là kỹ thuật pooling rất phổ biến.

2.7 Average Pooling – Average pooling

2.7.1 Ý tưởng average pooling

Average pooling lấy giá trị trung bình trong mỗi vùng pooling. Với vùng:

$$\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix}$$

giá trị average pooling là:

$$\frac{1 + 3 + 5 + 6}{4} = 3.75$$

2.7.2 Ví dụ average pooling

Với activation map:

$$A = \begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 3 & 1 \\ 0 & 4 & 5 & 8 \end{bmatrix}$$

Average pooling 2×2 , stride 2 cho kết quả:

$$P = \begin{bmatrix} 3.75 & 2.25 \\ 3.25 & 4.25 \end{bmatrix}$$

Vì:

$$\frac{1 + 3 + 5 + 6}{4} = 3.75$$

$$\frac{2 + 4 + 1 + 2}{4} = 2.25$$

$$\frac{7 + 2 + 0 + 4}{4} = 3.25$$

$$\frac{3 + 1 + 5 + 8}{4} = 4.25$$

Lưu ý

Max pooling thường được dùng phổ biến hơn trong nhiều CNN cổ điển, nhưng average pooling cũng xuất hiện trong một số kiến trúc.

2.8 Pooling as Downsampling – Pooling như một dạng downsampling

2.8.1 Downsampling là gì?

Downsampling là quá trình giảm kích thước dữ liệu.
 Trong ảnh, downsampling có thể hiểu là làm ảnh nhỏ lại.
 Pooling cũng có tác dụng tương tự: nó làm activation map nhỏ lại.
 Ví dụ:

$$10 \times 10 \xrightarrow{\text{pooling } 2 \times 2, \text{ stride } 2} 5 \times 5$$

2.8.2 Mục tiêu của pooling

Pooling có các mục tiêu chính:

1. Giảm kích thước activation map.
2. Giảm số lượng tính toán.
3. Giảm số tham số ở các lớp phía sau.
4. Giúp mô hình bớt nhạy với thay đổi nhỏ về vị trí.
5. Góp phần giảm overfitting.

Ghi nhớ

Mục tiêu lớn nhất của pooling là giảm dung lượng dữ liệu và giảm độ phức tạp của hệ thống.

2.8.3 Pooling có làm mất thông tin không?

Có. Khi giảm kích thước, pooling có thể làm mất một phần thông tin.
 Tuy nhiên, nếu hệ thống được thiết kế hợp lý, những thông tin quan trọng vẫn được giữ lại ở mức đủ dùng.

Lưu ý

Việc giảm kích thước ảnh hoặc activation map có thể làm mất thông tin. Tuy nhiên, trong nhiều bài toán, điều quan trọng là giữ đặc trưng lớn và tổng quát, không phải giữ mọi chi tiết nhỏ.

2.9 Repeated Convolution and Pooling – Lặp lại convolution và pooling

2.9.1 CNN thường có nhiều lớp

Trong thực tế, CNN hiếm khi chỉ có một convolution layer duy nhất. Thông thường, ta lặp lại nhiều lần các khối:

convolution $\rightarrow$ pooling

hoặc:

convolution $\rightarrow$ convolution $\rightarrow$ pooling

Tùy kiến trúc, số lớp có thể là:

- vài lớp;
- vài chục lớp;
- thậm chí hàng trăm lớp.

2.9.2 Output của lớp trước là input của lớp sau

Sau convolution và pooling, ta thu được một tensor mới. Tensor này trở thành input cho convolution layer tiếp theo. Ví dụ:

$$X \rightarrow \text{Conv}_1 \rightarrow A^{(1)} \rightarrow \text{Pooling} \rightarrow P^{(1)} \rightarrow \text{Conv}_2$$

Ghi nhớ

Trong CNN, các activation map của lớp trước trở thành các channel đầu vào cho lớp convolution tiếp theo.

2.10 Convolution on Multiple Activation Maps – Tích chập trên nhiều activation map

2.10.1 Input của lớp convolution sau có nhiều layer

Giả sử sau lớp convolution và pooling đầu tiên, ta thu được 4 activation map. Khi đó input cho lớp convolution tiếp theo có dạng:

$$H \times W \times 4$$

Tức là input có 4 channel.

2.10.2 Kernel của lớp sau cũng phải có 4 layer

Vì input có 4 channel, kernel của lớp convolution tiếp theo cũng phải có 4 layer. Nếu kernel có kích thước không gian 3×3 , thì kernel đầy đủ có kích thước:

$$3 \times 3 \times 4$$

Tức là nó gồm 4 ma trận 3×3 .

$$K^{(1)}, K^{(2)}, K^{(3)}, K^{(4)}$$

Mỗi layer của kernel áp lên một activation map tương ứng.

2.10.3 Một kernel nhiều layer vẫn tạo ra một activation map

Tương tự ảnh RGB, nếu input có 4 channel và kernel là $3 \times 3 \times 4$, thì một kernel vẫn chỉ tạo ra một activation map.

Nếu lớp đó có K kernel thì sẽ tạo ra K activation map.

$$K \text{ kernels} \rightarrow K \text{ output maps}$$

Ghi nhớ

Ở bất kỳ convolution layer nào, độ sâu của kernel phải bằng độ sâu của input.

2.11 Flatten and Fully Connected Layers – Flatten và các lớp fully connected

2.11.1 Khi nào cần flatten?

Sau nhiều lần convolution và pooling, ta thu được một tensor đặc trưng.

Ví dụ:

$$H' \times W' \times C'$$

Để đưa tensor này vào mạng fully connected, ta cần biến nó thành một vector.

Thao tác này gọi là:

flatten

2.11.2 Ví dụ flatten

Nếu tensor có kích thước:

$$5 \times 5 \times 16$$

thì sau flatten, vector có số phần tử:

$$5 \cdot 5 \cdot 16 = 400$$

Vector này được đưa vào fully connected layer.

2.11.3 Fully connected layer ở cuối CNN

Phần cuối của CNN thường gồm một hoặc vài fully connected layer.

Ví dụ:

$$\text{Flatten} \rightarrow \text{FC}_1 \rightarrow \text{FC}_2 \rightarrow \text{Output}$$

Trong bài toán phân loại, số output cuối cùng thường bằng số class.

Ví dụ minh họa

Nếu bài toán có 1000 class, output cuối cùng thường có 1000 phần tử.

2.12 Sliding Order – Thứ tự trượt kernel

2.12.1 Trượt ngang trước hay dọc trước có khác không?

Khi thực hiện convolution, ta thường minh họa bằng cách trượt kernel từ trái sang phải, rồi từ trên xuống dưới.

Tuy nhiên, về bản chất, ta có thể duyệt các vị trí theo thứ tự khác.

Ví dụ:

- trượt ngang trước rồi xuống dòng;
- trượt dọc trước rồi sang cột;
- duyệt theo bất kỳ thứ tự nào.

Kết quả cuối cùng vẫn giống nhau nếu ta ghi giá trị output vào đúng vị trí tương ứng.

Ghi nhớ

Thứ tự duyệt các vùng không quan trọng. Điều quan trọng là mỗi kết quả phải được ghi đúng vị trí trong activation map.

2.12.2 Sai lầm cần tránh

Nếu trượt kernel ở vị trí bên phải nhưng lại ghi kết quả xuống dòng dưới, hoặc trượt xuống dưới nhưng lại ghi kết quả sang bên phải, thì activation map sẽ bị sai.

Cảnh báo

Khi tính convolution bằng tay, cần chú ý ánh xạ đúng giữa vị trí kernel trên input và vị trí output trong activation map.

2.13 Output Size Reduction – Kích thước output giảm dần

2.13.1 Vì sao output có thể nhỏ dần?

Nếu convolution không dùng padding, kích thước output thường nhỏ hơn input.
Ví dụ input:

$$3 \times 3$$

kernel:

$$2 \times 2$$

stride bằng 1, không padding, output là:

$$2 \times 2$$

Vì kernel 2×2 chỉ có thể đặt được ở 4 vị trí hợp lệ trên ảnh 3×3 .

2.13.2 Pooling cũng làm kích thước nhỏ lại

Pooling thường làm kích thước giảm mạnh hơn.
Ví dụ:

$$10 \times 10 \xrightarrow{\text{pooling } 2 \times 2, \text{ stride } 2} 5 \times 5$$

Vì vậy, nếu lặp quá nhiều convolution và pooling, kích thước không gian của feature map có thể trở nên rất nhỏ.

Lưu ý

Trong thực tế, kiến trúc CNN phải được thiết kế hợp lý để kích thước feature map không giảm quá nhanh hoặc quá mức cần thiết.

2.14 Zero Padding – Đệm số 0

2.14.1 Zero padding là gì?

Zero padding là kỹ thuật thêm một hoặc nhiều lớp số 0 xung quanh input.

Ví dụ, input 3×3 :

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

Nếu thêm padding một lớp số 0 xung quanh, ta được:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & x_{11} & x_{12} & x_{13} & 0 \\ 0 & x_{21} & x_{22} & x_{23} & 0 \\ 0 & x_{31} & x_{32} & x_{33} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2.14.2 Mục đích của padding

Zero padding được dùng để:

1. kiểm soát kích thước output;
2. giúp feature map không bị nhỏ quá nhanh;
3. cho phép kernel xử lý tốt hơn các pixel ở biên ảnh;
4. giữ lại thông tin ở vùng rìa ảnh.

Ghi nhớ

Zero padding là cách thêm số 0 quanh input để điều chỉnh kích thước output sau convolution.

2.14.3 Công thức kích thước output

Với input kích thước $H \times W$, kernel $F \times F$, padding P , stride S , kích thước output là:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - F}{S} \right\rfloor + 1$$
$$W_{\text{out}} = \left\lfloor \frac{W + 2P - F}{S} \right\rfloor + 1$$

Nếu muốn giữ kích thước output bằng input, ta cần chọn padding phù hợp.

2.14.4 Lưu ý với kernel chẵn

Với kernel có kích thước chẵn, ví dụ 2×2 , việc giữ nguyên chính xác kích thước input bằng padding đối xứng không phải lúc nào cũng đơn giản.

Ví dụ, input 3×3 , kernel 2×2 , stride 1:

$$H_{\text{out}} = H$$

sẽ cần:

$$\frac{H + 2P - 2}{1} + 1 = H$$

Suy ra:

$$2P = 1$$

Tức là padding đối xứng nguyên không thực hiện được. Khi đó, một số hệ thống có thể dùng padding bất đối xứng hoặc quy ước riêng.

Lưu ý

Ý chính cần nhớ là padding giúp kiểm soát kích thước output. Với kernel lẻ như 3×3 , padding đối xứng để dùng hơn, ví dụ padding $P = 1$ giúp giữ kích thước khi stride bằng 1.

2.15 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
Channel	Kênh dữ liệu của ảnh hoặc feature map.	RGB có 3 channel.
RGB	Hệ màu gồm ba kênh đỏ, xanh lá, xanh dương.	Ảnh $224 \times 224 \times 3$.
Kernel	Bộ trọng số nhỏ dùng để tích chập trên input.	$3 \times 3 \times 3$.
Kernel depth	Độ sâu của kernel, phải bằng số channel input.	Input 4 channel thì kernel depth 4.
Activation map	Ma trận output do một kernel tạo ra.	Output 10×10 .
Feature map	Cách gọi khác của activation map.	Bản đồ đặc trưng.
Convolution layer	Lớp tích chập dùng kernel để trích xuất đặc trưng.	Conv 3×3 .
Pooling layer	Lớp giảm kích thước activation map.	Max pooling.
Max pooling	Lấy giá trị lớn nhất trong mỗi vùng pooling.	Max của vùng 2×2 .
Average pooling	Lấy giá trị trung bình trong mỗi vùng pooling.	Average của vùng 2×2 .
Window size	Kích thước cửa sổ pooling.	2×2 .
Stride	Bước nhảy khi trượt kernel hoặc pooling window.	Stride 2.
Downsampling	Giảm kích thước dữ liệu.	10×10 thành 5×5 .
Flatten	Biến tensor nhiều chiều thành vector.	$5 \times 5 \times 16$ thành vector 400.
Fully connected layer	Lớp kết nối đầy đủ ở cuối CNN.	Lớp phân loại.
Zero padding	Thêm viền số 0 quanh input.	Padding $P = 1$.

Thuật ngữ	Giải thích	Ví dụ
Padding	Đệm thêm quanh input để điều chỉnh output size.	Same padding.
Output size	Kích thước đầu ra sau convolution hoặc pooling.	$H_{\text{out}} \times W_{\text{out}}$.

2.16 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

2.16.1 Các ý chính cần nhớ

1. Ảnh màu RGB có 3 channel.
2. Khi input có nhiều channel, kernel cũng phải có cùng số layer với input.
3. Với input RGB, một kernel 3×3 thực chất có kích thước $3 \times 3 \times 3$.
4. Một kernel nhiều layer vẫn chỉ tạo ra một activation map.
5. Nếu một convolution layer có K kernel, nó tạo ra K activation map.
6. Pooling layer dùng để giảm kích thước activation map.
7. Max pooling lấy giá trị lớn nhất trong mỗi vùng.
8. Average pooling lấy giá trị trung bình trong mỗi vùng.
9. Pooling là một dạng downsampling.
10. Pooling giúp giảm dung lượng dữ liệu, giảm tính toán và giảm số tham số.
11. CNN thường lặp lại nhiều lần convolution và pooling.
12. Output của layer trước trở thành input nhiều channel cho layer sau.
13. Trước khi đưa vào fully connected layer, tensor thường được flatten thành vector.
14. Zero padding thêm viền số 0 quanh input để kiểm soát kích thước output.
15. Thứ tự trượt kernel không quan trọng nếu ghi output đúng vị trí.

2.16.2 Công thức quan trọng

Số tham số của một kernel nhiều kênh

$$K_h \cdot K_w \cdot C$$

Trong đó C là số channel của input.

Convolution nhiều kênh

$$a_{ij} = f \left(\sum_{c=1}^C \sum_{u=1}^{K_h} \sum_{v=1}^{K_w} K_{u,v,c} X_{i+u-1,j+v-1,c} + b \right)$$

Pooling 2×2 , stride 2

$$H \times W \rightarrow \frac{H}{2} \times \frac{W}{2}$$

nếu H, W chia hết cho 2.

Kích thước output sau convolution

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - F}{S} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W + 2P - F}{S} \right\rfloor + 1$$

2.17 Review Questions – Câu hỏi ôn tập

1. Ảnh màu RGB có bao nhiêu channel?
2. Nếu input có 3 channel, kernel phải có bao nhiêu layer?
3. Một kernel $3 \times 3 \times 3$ có bao nhiêu trọng số?
4. Một kernel tạo ra bao nhiêu activation map?
5. Nếu một convolution layer có 32 kernel, output có bao nhiêu activation map?
6. Pooling layer dùng để làm gì?
7. Max pooling khác average pooling như thế nào?
8. Với activation map 10×10 , pooling 2×2 , stride 2 tạo ra output kích thước bao nhiêu?
9. Downsampling là gì?
10. Vì sao pooling có thể làm mất thông tin?
11. Vì sao vẫn dùng pooling dù nó có thể làm mất thông tin?
12. Khi output của layer trước có 4 activation map, kernel của layer sau phải có depth bằng bao nhiêu?
13. Flatten là gì?
14. Zero padding là gì?

15. Vì sao cần zero padding?
16. Thứ tự trượt kernel có làm thay đổi kết quả không nếu ghi output đúng vị trí?

2.18 Practice Exercises – Bài tập vận dụng

Bài tập 1: Kích thước kernel nhiều kênh

Input là ảnh RGB. Ta dùng kernel có kích thước không gian 5×5 .
Hỏi kernel đầy đủ có kích thước bao nhiêu và có bao nhiêu trọng số?

Lời giải gợi ý

Ảnh RGB có 3 channel, nên kernel phải có depth bằng 3.
Kích thước kernel là:

$$5 \times 5 \times 3$$

Số trọng số:

$$5 \cdot 5 \cdot 3 = 75$$

Bài tập 2: Số activation map

Một convolution layer có 64 kernel. Hỏi output của layer này có bao nhiêu activation map?

Lời giải gợi ý

Mỗi kernel tạo ra một activation map.
Do đó 64 kernel tạo ra:

$$64$$

activation map.

Bài tập 3: Max pooling

Cho activation map:

$$A = \begin{bmatrix} 1 & 4 & 2 & 3 \\ 5 & 6 & 1 & 0 \\ 2 & 8 & 7 & 1 \\ 3 & 4 & 5 & 9 \end{bmatrix}$$

Dùng max pooling 2×2 , stride 2. Tính output.

Lời giải gợi ý

Các vùng pooling là:

$$\begin{bmatrix} 1 & 4 \\ 5 & 6 \end{bmatrix} \Rightarrow 6$$

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \Rightarrow 3$$

$$\begin{bmatrix} 2 & 8 \\ 3 & 4 \end{bmatrix} \Rightarrow 8$$

$$\begin{bmatrix} 7 & 1 \\ 5 & 9 \end{bmatrix} \Rightarrow 9$$

Vậy output là:

$$P = \begin{bmatrix} 6 & 3 \\ 8 & 9 \end{bmatrix}$$

Bài tập 4: Average pooling

Dùng lại activation map ở bài tập 3. Tính average pooling 2×2 , stride 2.

Lời giải gợi ý

$$\frac{1 + 4 + 5 + 6}{4} = 4$$

$$\frac{2 + 3 + 1 + 0}{4} = 1.5$$

$$\frac{2 + 8 + 3 + 4}{4} = 4.25$$

$$\frac{7 + 1 + 5 + 9}{4} = 5.5$$

Vậy output là:

$$P = \begin{bmatrix} 4 & 1.5 \\ 4.25 & 5.5 \end{bmatrix}$$

Bài tập 5: Kích thước output sau convolution

Input có kích thước 32×32 , kernel 3×3 , stride 1, padding 1.
Tính kích thước output.

Lời giải gợi ý

$$H_{\text{out}} = \left\lfloor \frac{32 + 2 \cdot 1 - 3}{1} \right\rfloor + 1$$

$$H_{\text{out}} = 32$$

Tương tự:

$$W_{\text{out}} = 32$$

Vậy output có kích thước:

$$32 \times 32$$

Bài tập 6: Flatten

Sau nhiều lớp CNN, ta thu được tensor kích thước:

$$7 \times 7 \times 128$$

Hỏi sau flatten, vector có bao nhiêu phần tử?

Lời giải gợi ý

$$7 \cdot 7 \cdot 128 = 6272$$

Vector sau flatten có 6272 phần tử.

Conclusion – Kết luận

CNN2 mở rộng phép convolution từ trường hợp ảnh xám một kênh sang ảnh màu nhiều kênh. Khi input có nhiều channel, kernel cũng phải có cùng số layer tương ứng. Ví dụ, ảnh RGB có 3 channel thì kernel 3×3 thực chất là kernel $3 \times 3 \times 3$. Tại mỗi vị trí, ta nhân tương ứng trên tất cả các channel, cộng toàn bộ các tích, cộng bias và đưa qua hàm kích hoạt để tạo ra một giá trị trong activation map.

Bài học cũng giới thiệu pooling layer, một thành phần quan trọng trong CNN. Max pooling lấy giá trị lớn nhất trong mỗi vùng, còn average pooling lấy giá trị trung bình. Pooling là một dạng downsampling, giúp giảm kích thước dữ liệu, giảm số lượng tính toán và giảm số tham số ở các lớp phía sau.

Trong CNN thực tế, convolution và pooling thường được lặp lại nhiều lần. Output của lớp trước trở thành input nhiều channel cho lớp sau, nên kernel ở lớp sau cũng phải có depth bằng số channel của input. Cuối cùng, tensor đặc trưng được flatten thành vector rồi đưa vào fully connected layer để phân loại.

Ngoài ra, bài học cũng nhắc đến zero padding, tức là thêm viền số 0 quanh input để kiểm soát kích thước output và giúp feature map không bị nhỏ quá nhanh sau nhiều lớp convolution.

Chương 3

CNN3: Convolution, Correlation và xử lý ảnh nhiều kênh trong CNN

Learning Objectives – Mục tiêu bài học

Bài học này review lại một số khái niệm nền tảng trước khi tiếp tục đi sâu vào *Convolutional Neural Network*, viết tắt là CNN.

Sau khi học xong, người học cần nắm được:

1. Phân biệt được *convolution* – tích chập và *correlation* – tương quan.
2. Hiểu *cross-correlation* – tương quan chéo và *auto-correlation* – tự tương quan.
3. Biết điểm khác nhau quan trọng giữa convolution và correlation: trong convolution có thao tác lật hàm hoặc lật kernel.
4. Hiểu trực giác của tích chập thông qua ví dụ hai xung chữ nhật.
5. Biết vì sao ví dụ với hàm $f(t)$ và $g(t)$ là tích chập một chiều.
6. Hiểu ảnh xám được xem như dữ liệu hai chiều theo không gian.
7. Hiểu ảnh RGB có ba kênh màu nhưng trong CNN cơ bản không đơn giản xem là một phép tích chập 3D.
8. Biết cách CNN xử lý ảnh RGB: tính riêng từng kênh màu rồi cộng kết quả lại.

3.1 Convolution and Correlation – Tích chập và tương quan

3.1.1 Các thuật ngữ cần phân biệt

Trong phần CNN, ta thường gặp các thuật ngữ gần giống nhau:

- **Convolution** – tích chập;
- **Correlation** – tương quan;
- **Cross-correlation** – tương quan chéo;

- **Auto-correlation** – tự tương quan.

Các khái niệm này có hình thức tính toán khá gần nhau, nên dễ gây nhầm lẫn.

Ghi nhớ

Điểm khác quan trọng nhất: trong *convolution*, một hàm hoặc kernel bị lật trước khi trượt. Trong *correlation*, kernel không bị lật.

3.1.2 Cross-correlation – Tương quan chéo

Giả sử có hai hàm f và g . Tương quan chéo giữa hai hàm này đo mức độ giống nhau giữa chúng khi ta trượt một hàm qua hàm còn lại.

Một cách viết phổ biến của cross-correlation là:

$$(f \star g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(\tau + t)d\tau$$

Trong đó:

- f được giữ nguyên;
- g cũng được giữ nguyên;
- g được trượt theo t ;
- tại mỗi vị trí, ta nhân hai hàm rồi lấy tích phân.

Ghi nhớ

Cross-correlation dùng để đo mức tương đồng giữa hai hàm khác nhau.

3.1.3 Auto-correlation – Tự tương quan

Auto-correlation là trường hợp đặc biệt của correlation, khi một hàm được tương quan với chính nó.

Ví dụ:

$$(f \star f)(t)$$

hoặc:

$$(g \star g)(t)$$

Ví dụ minh họa

Nếu f là tín hiệu âm thanh, auto-correlation có thể dùng để xem tín hiệu đó giống với chính nó như thế nào sau khi bị dịch đi một khoảng thời gian.

3.1.4 Convolution – Tích chập

Tích chập giữa hai hàm f và g được định nghĩa:

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(t - \tau)d\tau$$

Điểm đáng chú ý là biểu thức:

$$g(t - \tau)$$

tương ứng với việc hàm g bị lật rồi dịch chuyển.

Lưu ý

Trong convolution, ta không chỉ trượt kernel. Ta lật kernel trước, sau đó mới trượt.

3.2 Convolution vs Correlation – So sánh tích chập và tương quan

3.2.1 Correlation không lật hàm

Trong correlation, hàm hoặc kernel được giữ nguyên.

Ta có thể hiểu đơn giản:

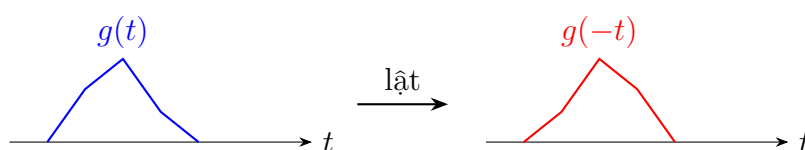
Correlation = giữ nguyên kernel + trượt

3.2.2 Convolution lật hàm trước khi trượt

Trong convolution, hàm hoặc kernel được lật trước.

Ta có thể hiểu:

Convolution = lật kernel + trượt



Ghi nhớ

Nếu chỉ nhớ một câu: correlation là trượt trực tiếp, còn convolution là lật rồi trượt.

3.2.3 Trong CNN thực tế

Trong nhiều thư viện deep learning, phép toán được gọi là convolution thường được cài đặt giống cross-correlation, tức là kernel không bị lật.

Điều này thường không gây vấn đề trong CNN vì các giá trị kernel là tham số được học từ dữ liệu.

Lưu ý

Về mặt toán học, convolution và correlation khác nhau. Nhưng trong thực hành CNN, nhiều thư viện vẫn gọi phép cross-correlation là convolution.

3.3 How to Compute 1D Convolution – Cách tính tích chập một chiều

3.3.1 Công thức tích chập liên tục

Tích chập một chiều giữa hai hàm f và g :

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(t - \tau)d\tau$$

Có thể hiểu theo các bước:

1. Giữ nguyên hàm f .
2. Lật hàm g theo trục thời gian.
3. Dịch chuyển hàm g đã lật theo t .
4. Nhân hai hàm tại từng vị trí.
5. Lấy tích phân của tích đó.

3.3.2 Ý nghĩa trực giác

Tại mỗi vị trí trượt, ta xét phần chồng lấp giữa f và phiên bản đã lật, đã dịch của g . Tuy nhiên, giá trị tích chập không đơn giản chỉ là diện tích phần giao nhau. Tổng quát hơn:

giá trị tích chập = tổng có trọng số của phần chồng lấp

Nghĩa là giá trị của f tại từng điểm phải được nhân với giá trị tương ứng của g .

Lưu ý

Chỉ trong trường hợp đặc biệt khi g có biên độ bằng 1 trên vùng đang xét, giá trị tích chập mới bằng diện tích phần giao nhau.

3.4 Rectangular Pulse Example – Ví dụ hai xung chữ nhật

3.4.1 Mô tả ví dụ

Giả sử ta có hai hàm dạng xung chữ nhật:

$$f(t)$$

và:

$$g(t)$$

Trong ví dụ minh họa, f là một xung chữ nhật màu xanh, còn g là một xung chữ nhật màu đỏ.

Giả sử biên độ của g bằng 1.

3.4.2 Khi hai hàm chưa giao nhau

Ban đầu, g nằm xa f , hai hàm không có phần giao nhau.

Do đó:

$$(f * g)(t) = 0$$

3.4.3 Khi bắt đầu giao nhau

Khi g bắt đầu trượt vào vùng có f , phần giao nhau bắt đầu xuất hiện.

Khi đó giá trị tích chập bắt đầu tăng:

$$0 \rightarrow \text{tăng dần}$$

3.4.4 Khi phần giao nhau lớn nhất

Khi g chồng lên f nhiều nhất, tích chập đạt giá trị lớn nhất.

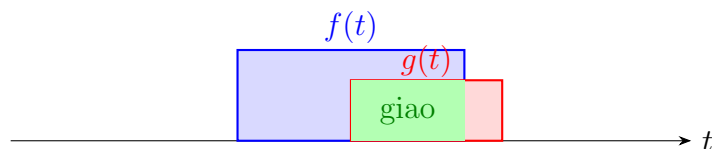
Trong trường hợp g có biên độ bằng 1, giá trị này đúng bằng diện tích phần giao nhau dưới f .

3.4.5 Khi trượt ra khỏi nhau

Sau khi g tiếp tục trượt đi, phần giao nhau giữa f và g giảm dần.

Do đó tích chập cũng giảm dần:

$$\text{giảm dần} \rightarrow 0$$



Ghi nhớ

Trong ví dụ hai xung chữ nhật, tích chập bằng 0 khi không giao nhau, tăng dần khi bắt đầu giao nhau, đạt cực đại khi giao nhau nhiều nhất, rồi giảm dần về 0.

3.5 1D and 2D Convolution – Tích chập một chiều và hai chiều

3.5.1 Ví dụ với $f(t)$ và $g(t)$ là 1D

Trong ví dụ trước, ta xét hai hàm:

$$f(t), \quad g(t)$$

Hai hàm này chỉ phụ thuộc vào một biến t .

Do đó đây là tích chập một chiều:

convolution 1D

Ghi nhớ

Nếu hàm chỉ phụ thuộc vào một biến, ví dụ thời gian t , thì ta đang xét tích chập một chiều.

3.5.2 Ảnh xám là dữ liệu 2D

Một ảnh xám không chỉ phụ thuộc vào một biến. Nó phụ thuộc vào hai tọa độ không gian:

$$I(x, y)$$

Trong đó:

x = tọa độ ngang

y = tọa độ dọc

Vì vậy, ảnh xám được xem là dữ liệu hai chiều.

ảnh xám $\Rightarrow$ dữ liệu 2D

3.5.3 Tích chập 2D trong ảnh

Với ảnh xám, kernel cũng là một ma trận 2D.

Ví dụ:

$$I = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}$$

Kernel:

$$K = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix}$$

Tại góc trên bên trái, vùng ảnh tương ứng là:

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

Ta nhân từng phần tử rồi cộng:

$$1 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) + 1 \cdot 2$$

$$= 1 + 0 + 0 + 2 = 3$$

Nếu bias là:

$$b = 0.5$$

thì:

$$z = 3 + 0.5 = 3.5$$

Nếu dùng ReLU:

$$a = \text{ReLU}(3.5) = 3.5$$

Ví dụ minh họa

Ở ảnh xám, kernel trượt theo hai chiều không gian: ngang và dọc. Vì vậy ta gọi đây là tích chập 2D.

3.6 RGB Images in CNN – Ảnh RGB trong CNN

3.6.1 Ảnh RGB có ba kênh

Ảnh màu RGB gồm ba kênh:

$$R, \quad G, \quad B$$

Trong đó:

- R là kênh đỏ;
- G là kênh xanh lá;
- B là kênh xanh dương.

Một ảnh RGB kích thước $H \times W$ có thể được biểu diễn dưới dạng:

$$H \times W \times 3$$

3.6.2 Có phải tích chập 3D không?

Trong bài học, cần chú ý rằng ta không đơn giản nói ảnh RGB là đang tính một phép tích chập 3D theo nghĩa không gian ba chiều.

Lý do là CNN không trượt kernel theo chiều kênh màu giống như trượt theo chiều ngang và chiều dọc.

Thay vào đó, với ảnh RGB, một filter sẽ có ba kernel con tương ứng với ba kênh:

$$K_R, \quad K_G, \quad K_B$$

Lưu ý

Trong CNN cơ bản, kernel trượt theo hai chiều không gian. Chiều kênh màu được xử lý bằng cách tính riêng từng kênh rồi cộng lại, không phải trượt theo chiều kênh như một chiều không gian thứ ba.

3.6.3 Cách tính trên ảnh RGB

Giả sử input có ba kênh:

$$R, \quad G, \quad B$$

Filter cũng có ba phần:

$$K_R, \quad K_G, \quad K_B$$

Ta tính riêng từng kênh:

$$R * K_R$$

$$G * K_G$$

$$B * K_B$$

Sau đó cộng lại và cộng bias:

$$z = (R * K_R) + (G * K_G) + (B * K_B) + b$$

Cuối cùng đưa qua hàm kích hoạt:

$$a = f(z)$$

Ghi nhớ

Với ảnh RGB, mỗi filter nhìn đồng thời cả ba kênh màu, nhưng kết quả cuối cùng tại mỗi vị trí vẫn là một giá trị.

3.7 RGB Convolution Example – Ví dụ tính trên ảnh RGB

3.7.1 Vùng ảnh đang xét

Giả sử ta xét một vùng ảnh RGB kích thước 2×2 .

Kênh đỏ:

$$R = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

Kênh xanh lá:

$$G = \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}$$

Kênh xanh dương:

$$B = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}$$

3.7.2 Filter tương ứng

Kernel cho kênh đỏ:

$$K_R = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix}$$

Kernel cho kênh xanh lá:

$$K_G = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Kernel cho kênh xanh dương:

$$K_B = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

Bias:

$$b = 0.5$$

3.7.3 Tính trên kênh R

$$R * K_R = 1 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) + 1 \cdot 2$$

$$R * K_R = 3$$

3.7.4 Tính trên kênh G

$$G * K_G = 0 \cdot 0 + 1 \cdot 1 + 2 \cdot 1 + 1 \cdot 0$$

$$G * K_G = 3$$

3.7.5 Tính trên kênh B

$$B * K_B = 2 \cdot 1 + 0 \cdot (-1) + 1 \cdot 0 + 3 \cdot 1$$

$$B * K_B = 5$$

3.7.6 Cộng các kênh và bias

$$z = 3 + 3 + 5 + 0.5$$

$$z = 11.5$$

Nếu dùng ReLU:

$$a = \text{ReLU}(11.5) = 11.5$$

Vậy tại vị trí đang xét, output của filter là:

$$11.5$$

Ví dụ minh họa

Một filter trên ảnh RGB có thể học cách kết hợp thông tin màu. Ví dụ, nó có thể phản ứng mạnh khi kênh đỏ, xanh lá và xanh dương tạo ra một kiểu cấu trúc cụ thể nào đó.

3.8 Activation Map from RGB Filter – Activation map từ filter RGB

3.8.1 Một filter tạo ra một activation map

Khi một filter trượt qua toàn bộ ảnh RGB, tại mỗi vị trí nó tạo ra một giá trị. Các giá trị đó ghép lại thành một activation map.

$$1 \text{ filter} \Rightarrow 1 \text{ activation map}$$

Nếu có F filter:

$$F \text{ filters} \Rightarrow F \text{ activation maps}$$

Ghi nhớ

Số activation map bằng số filter, không phải bằng số kênh màu.

3.8.2 Số trọng số của filter RGB

Nếu input là ảnh RGB và kernel không gian có kích thước:

$$3 \times 3$$

thì một filter có kích thước:

$$3 \times 3 \times 3$$

Số trọng số là:

$$3 \cdot 3 \cdot 3 = 27$$

Cộng thêm một bias:

$$27 + 1 = 28$$

Nếu có 16 filter, tổng số tham số là:

$$16 \cdot 28 = 448$$

Ví dụ minh họa

Một convolution layer có 16 filter 3×3 nhận input RGB sẽ tạo ra 16 activation map và có 448 tham số.

3.9 Technical Glossary – Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
Convolution	Tích chập; phép toán lật kernel rồi trượt qua tín hiệu hoặc ảnh.	$f * g$
Correlation	Tương quan; phép đo độ giống nhau khi trượt mà không lật.	$f \star g$
Cross-correlation	Tương quan chéo giữa hai hàm khác nhau.	$f \star g$
Auto-correlation	Tự tương quan của một hàm với chính nó.	$f \star f$
Kernel	Ma trận trọng số nhỏ dùng để quét trên ảnh.	3×3
Filter	Bộ lọc; trong CNN thường gồm kernel theo toàn bộ số kênh input.	$3 \times 3 \times 3$
Activation map	Bản đồ kích hoạt sau khi một filter quét qua input.	Một map $H_{out} \times W_{out}$
RGB	Ảnh màu gồm ba kênh đỏ, xanh lá, xanh dương.	$H \times W \times 3$
Channel	Kênh dữ liệu trong ảnh hoặc feature map.	R, G, B

Thuật ngữ	Giải thích	Ví dụ
1D convolution	Tích chập trên tín hiệu một chiều.	Tín hiệu theo thời gian
2D convolution	Tích chập trên dữ liệu hai chiều.	Ảnh xám
Bias	Hằng số cộng thêm sau phép nhân và cộng.	$z = \sum xw + b$
ReLU	Hàm kích hoạt lấy giá trị lớn hơn giữa 0 và input.	$\max(0, x)$

3.10 Key Concepts Summary – Tóm tắt kiến thức trọng tâm

3.10.1 Các ý chính cần nhớ

1. Convolution và correlation là hai khái niệm gần nhau nhưng không giống nhau.
2. Trong convolution, kernel bị lật trước khi trượt.
3. Trong correlation, kernel không bị lật.
4. Cross-correlation là tương quan giữa hai hàm khác nhau.
5. Auto-correlation là tương quan của một hàm với chính nó.
6. Tích chập liên tục được biểu diễn bằng tích phân.
7. Giá trị tích chập tổng quát không chỉ là diện tích giao nhau, mà là tổng có trọng số.
8. Ví dụ với $f(t)$ và $g(t)$ là tích chập một chiều.
9. Ảnh xám là dữ liệu hai chiều vì phụ thuộc vào hai tọa độ không gian.
10. Ảnh RGB có ba kênh màu: R, G, B.
11. Với ảnh RGB, CNN tính riêng từng kênh màu rồi cộng kết quả lại.
12. Một filter tạo ra một activation map.
13. Số activation map bằng số filter.

3.10.2 Công thức quan trọng

Convolution liên tục

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(t - \tau)d\tau$$

Cross-correlation liên tục

$$(f \star g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(\tau + t)d\tau$$

Tính trên ảnh RGB

$$z = (R * K_R) + (G * K_G) + (B * K_B) + b$$

$$a = f(z)$$

Số trọng số của filter RGB

$$K_h \cdot K_w \cdot 3$$

3.11 Review Questions – Câu hỏi ôn tập

1. Convolution khác correlation ở điểm nào?
2. Cross-correlation là gì?
3. Auto-correlation là gì?
4. Trong convolution, kernel có bị lật không?
5. Trong correlation, kernel có bị lật không?
6. Vì sao ví dụ với $f(t)$ và $g(t)$ là tích chập 1D?
7. Ảnh xám là dữ liệu mấy chiều?
8. Ảnh RGB gồm những kênh nào?
9. Với ảnh RGB, CNN xử lý từng kênh màu như thế nào?
10. Một filter tạo ra bao nhiêu activation map?
11. Nếu có 32 filter thì có bao nhiêu activation map?
12. Một filter 3×3 trên ảnh RGB có bao nhiêu trọng số?

3.12 Practice Exercises – Bài tập vận dụng

Bài tập 1: Phân biệt convolution và correlation

Điền vào chỗ trống:

1. Trong convolution, kernel được _____ trước khi trượt.
2. Trong correlation, kernel _____ trước khi trượt.

Lời giải gợi ý

1. lật.
2. không bị lật.

Bài tập 2: Tính số trọng số

Một filter có kích thước không gian:

$$5 \times 5$$

và input có 3 kênh màu. Hỏi filter có bao nhiêu trọng số? Nếu cộng thêm bias thì có bao nhiêu tham số?

Lời giải gợi ý

Số trọng số:

$$5 \cdot 5 \cdot 3 = 75$$

Cộng thêm một bias:

$$75 + 1 = 76$$

Vậy filter có 76 tham số.

Bài tập 3: Số activation map

Một convolution layer có 20 filter. Hỏi layer này tạo ra bao nhiêu activation map?

Lời giải gợi ý

Mỗi filter tạo ra một activation map.

$$20 \text{ filter} \Rightarrow 20 \text{ activation map}$$

Bài tập 4: Tính trên ảnh RGB

Cho kết quả tính riêng từng kênh là:

$$R * K_R = 2$$

$$G * K_G = -1$$

$$B * K_B = 4$$

Bias:

$$b = 0.5$$

Tính z trước hàm kích hoạt.

Lời giải gợi ý

$$z = 2 + (-1) + 4 + 0.5$$

$$z = 5.5$$

Conclusion – Kết luận

CNN3 review lại nền tảng của tích chập và tương quan để chuẩn bị cho việc hiểu sâu hơn về convolution layer trong CNN. Convolution khác correlation ở thao tác lật kernel trước khi trượt. Trong khi đó, correlation giữ nguyên kernel và chỉ trượt trực tiếp.

Bài học cũng nhấn mạnh rằng ví dụ với hai hàm $f(t)$ và $g(t)$ là tích chập một chiều. Khi chuyển sang ảnh xám, ta làm việc với dữ liệu hai chiều theo không gian. Với ảnh RGB, input có ba kênh màu R, G, B. CNN xử lý từng kênh bằng kernel tương ứng, cộng các kết quả lại, cộng bias rồi đưa qua hàm kích hoạt.

Điểm quan trọng cần nhớ là: một filter tạo ra một activation map. Nếu convolution layer có nhiều filter, output sẽ có nhiều activation map tương ứng.